An E-Commerce Recommender System: Mathematical Specification Using Coveo SIGIR eCommerce Data

Interaction Graph, Item Features,

and Session Search Context

Version 2.10.

Demo service requirements clarified on October 8, 2026.

Abstract

The initial version uses a transition graph, provided description and image vectors, search, and LambdaMART ranking. A neural extension learns item representations and session intents. Each stage specifies its inputs, training procedure, validation, and conditions for proceeding to the next stage. The business objective is to increase the number of sessions with a purchase per 1000 eligible sessions. The first Coveo evaluation measures whether the first recommendation list includes a recorded future purchase among sessions with at least one such purchase. Telemetry completeness remains unknown; evaluation over all sessions requires it to be confirmed. The data support measuring agreement with historical purchases; causal effects require a randomized experiment with recommendation impressions. This specification contains no training results or load measurements.

Implementation sequence

Stage	Sections and outcome
Baseline system	Part I, recipe H.1, and tests K.2. The first report uses C_{smoke}; no configuration tuning or access to D.
Selection and final evaluation	Period and selection sequence: Section 13.5; result states: Section 20.
Neural model	Part II, parameters H.2, tests K.3. Requires a reproducible baseline, diagnostics of its losses, and measured cost.
Separate studies	Sequence B, graph updates C, SID D, other models E. Each requires a predefined question, control, and budget.
Experiment with impressions	Protocol F, after serving validation. Measures the causal effect.

Conditions for moving between stages are specified in the run manifest H.4. Parameter values are in Appendix H; report contents are in I.2. If D has already been used to evaluate the baseline, the next extension requires a new final evaluation period.

The current implementation goal is local historical evaluation through D under the `observed_purchases_v1` protocol (Section 14.1) in `ideal_deterministic` mode. It defines common rules for recorded history, targets, metrics, and acceptance. Estimating purchase frequency over all sessions would require confirmed observation completeness and a separate formulation; it is outside

the current scope. Service readiness is checked separately and is not a prerequisite for this evaluation. Neural studies, the service, and A/B testing remain later stages subject to all requirements of their respective sections.

Contents

Implementation sequence **1**

I Core system: data, training, and evaluation 7

1 Purpose and chosen formulation **7**

2 Notation and units **7**

3 Coveo data and observation limits **8**
3.1 Observed objects . . . 8
3.2 What cannot be recovered from the dataset . . . 9
3.3 Snapshot and reproducible subsample . . . 9

4 Time, cleaning, and available history **10**
4.1 Event ordering . . . 10
4.2 Information at decision time . . . 10
4.3 Observed cart state . . . 11

5 Requests, eligible items, and future targets **11**
5.1 Observation start and schedule . . . 11
5.2 Recorded target window . . . 11
5.3 Catalog and labels . . . 11

6 Primary business criterion **12**
6.1 Purchasing sessions and causal lift . . . 12
6.2 Why a purchase label does not solve the causal problem . . . 13
6.3 Monetary extension when the required data become available . . . 13
6.4 Levels of evidence . . . 14

7 Item interaction graph **14**
7.1 Vertices, relations, and one session's contribution . . . 14
7.2 Sparsification and graph-row regularization . . . 15
7.3 Construction and request cost . . . 16

8 Content features and search context **16**
8.1 Separate representation spaces . . . 16
8.2 Latest available search query . . . 17
8.3 Categories and price bucket . . . 17

9 Candidate set construction **18**
9.1 Sources and quotas . . . 18
9.2 Neighbor search in vector space . . . 19
9.3 Candidate provenance features . . . 19

10 Request–item pair features **19**

11 Learning to rank future purchases **21**
11.1 Labels, groups, and weights . . . 21
11.2 Pairwise logistic loss . . . 21
11.3 Greater weight for errors near the top of the list . . . 22
11.4 Tree ensemble and regularization . . . 22
11.5 Executable contract for custom derivatives . . . 23

11.6 Negative examples and sparse purchases . . . 24
11.7 Limits on score interpretation . . . 25

12 Final recommendation list construction 25
12.1 Primary recommendation method . . . 25
12.2 Fallback response . . . 25

13 Temporal protocol for training and evaluation 26
13.1 Periods and their roles . . . 26
13.2 Sequence of the first run and final evaluation . . . 26
13.3 Calendar admission and complete reading . . . 27
13.4 Model versions and final freeze . . . 28
13.5 One primary final comparison . . . 28

14 Evaluation on the historical log 28
14.1 Session population and primary metric . . . 29
14.2 Exact upper bound for candidate retrieval . . . 29
14.3 Mandatory decomposition of coverage losses . . . 30
14.4 Source contribution and overlap . . . 30
14.5 Choosing the next change . . . 31
14.6 Auxiliary metrics and slices . . . 31

15 Uncertainty in historical comparisons 32
15.1 Paired session comparison . . . 32
15.2 Planning precision for period D . . . 33
15.3 Temporal dependence and interpretation limits . . . 34
15.4 Information sufficiency and degenerate intervals . . . 35

16 Feature storage and updates 36
16.1 Training–serving consistency . . . 36
16.2 Numerical stability . . . 36
16.3 Session state updates . . . 37
16.4 State storage contract . . . 38
16.5 Requirements for request replay . . . 38
16.6 Arrival time and late events . . . 39
16.7 Partial data availability . . . 39
16.8 Trust boundaries, input limits, and trace retention . . . 40

17 Computational budget and serving 42
17.1 Preparation, training, and request processing . . . 42
17.2 Version contents . . . 42
17.3 Memory and operation counts . . . 43
17.4 Latency and fallback branches . . . 44
17.5 Reproducible load profile . . . 45
17.6 Recovery and rollback . . . 47

18 Monitoring quality and data changes 47
18.1 Immediately observable metrics . . . 47
18.2 Distribution changes . . . 48
18.3 Metrics with delayed outcomes . . . 49
18.4 Model updates . . . 49

19 Explaining results and limits of interpretation 49

20 Result completeness and acceptance criteria 50
20.1 Result states . . . 50
20.2 Single-request result . . . 51
20.3 Complete computational formulation . . . 52

II Neural extension 53

21 Rationale and sequence for the neural extension 53
21.1 Relationship to work from 2024–2026 . . . 53
21.2 Configuration and training chronology . . . 53
21.3 Implementation order and controlled extensions . . . 53
21.4 Hypotheses following diagnostics . . . 54

22 Learned model of graph, content, and session intents 54
22.1 What is learned and when . . . 54
22.2 Shortest paths in the fixed graph . . . 55
22.3 Learned links and separate item representations . . . 55
22.4 Multiple intents and selection by search context . . . 56
22.5 Candidate retrieval and ranking features . . . 58
22.6 Primary training objective: recorded future purchases . . . 59
22.7 Auxiliary next-view prediction . . . 59
22.8 Contrastive alignment of two representations . . . 60
22.9 Regularization and final training objective . . . 61
22.10 Reproducible training recipe for extensions . . . 61
22.11 Checking intent distinctness . . . 62
22.12 Search-signal robustness . . . 63
22.13 Cost and failure behavior . . . 64

III Appendices: computational variants and additional studies 65

A Approximate training with item sampling 65
A.1 Positive items and sampling distribution . . . 65
A.2 Denominator estimation and derivatives . . . 66
A.3 Training integration and cost . . . 67
A.4 Computational checks . . . 67

B Sequential model for a controlled comparison 68
B.1 Identical inputs and groups of simultaneous events . . . 68
B.2 A single causal attention block . . . 68
B.3 Training, candidates, and comparison conditions . . . 69
B.4 Cost and testable properties . . . 70

C Aging of relations and an updated graph 70
C.1 Two distinct time scales . . . 70
C.2 Publishing a consistent version . . . 71
C.3 Comparison protocol and checks . . . 71

D Candidate generator based on semantic codes 71

D.1 Training data and fixed representations . . . 72
D.2 Soft assignments and the hard code . . . 72
D.3 Losses and gradient flow to assignments . . . 73
D.4 Autoregressive decoder and training order . . . 74
D.5 Valid sequences, collisions, and output size . . . 74
D.6 Diagnostics, comparisons, and computational limits . . . 75

E Additional retrieval and slate construction methods 76
E.1 Downweighting edges supported by few sessions . . . 76
E.2 Additional slate diversity . . . 76
E.3 Why the sum of item probabilities is not conversion . . . 77
E.4 Multiple graph steps . . . 77
E.5 Behavioral item vectors . . . 78

F Experiment with real impressions 78
F.1 Assignment unit and causal effect . . . 79
F.2 Interval for the business effect . . . 80
F.3 Business result acceptance rule . . . 81
F.4 Experiment size and duration . . . 81
F.5 Assignment check and stopping rule . . . 82
F.6 Incomplete outcomes and safety measures . . . 83

G Why historical data do not identify an increase in purchases 83
G.1 Conditions for evaluating another policy from an impression log . . . 83
G.2 What event replay can establish . . . 84

H Run configurations and parameter registry 84
H.1 Fixed configuration for the first run . . . 84
H.2 Neural extension parameters . . . 85
H.3 Initial settings for the deferred SID study . . . 86
H.4 Run manifest and stage checks . . . 87
H.5 Preliminary calendar plan . . . 89
H.6 Parameter sources . . . 89
H.7 Period sizes and data sufficiency . . . 90

I Comparison order and isolating component contributions 91
I.1 Required metadata sensitivity analysis . . . 92
I.2 Required report for the first run and comparisons . . . 93

J Numerical example 94

K Testable implementation properties 94
K.1 Requirement mapping and check order . . . 95
K.2 Baseline system . . . 95
K.3 Neural extension . . . 98
K.4 Additional variants . . . 99

L Contents of the final research deliverable 100

References and documentation 101

Part I

Core system: data, training, and evaluation

1 Purpose and chosen formulation

Recommendations use the current session without a persistent customer identifier. Inputs are the available history, the latest search query, and the item feature version. The response contains an ordered list, the model version, candidate sources, and computation status.

The first implementation is policy π_{base}: graph transitions, similarity of the provided item vectors, search context, and LambdaMART ranking. This policy is called GCS+Rk: G denotes the graph, C the description and image features, S search, and Rk the ranker. The first run is defined in Part I, neural representations in Part II, and additional studies in the appendices.

Initial settings and load targets define requirements; performance and business impact remain to be measured.

Object	Primary variant
Deployment objective	Increase the fraction of eligible sessions with a purchase, subject to latency, error, and cost constraints.
Outcome unit	A session admitted under rules checked before the first experimental recommendation impression.
Item retrieval in the first run	One-step transitions on the historical graph, content and search candidates, recent items, and popularity.
Ranking	Trees with pairwise training; the primary label is a subsequent purchase of the item. Clicks and cart actions are considered separately.
Historical evaluation	Common protocol, sessions, and requests across methods; the first Coveo evaluation follows Section 14.1.
Evidence of impact	A randomized experiment counting all assigned sessions, including serving failures.
Prototype subsample	Complete sessions from a predefined set; the result is not claimed as an official competition result.

Invariant 1.1 (Three distinct quantities)**.** Model scores, agreement of recommendations with historical purchases, and changes in conversion after deployment are different quantities. They are not equated without additional data and assumptions.

2 Notation and units

$\mathbf{1}\{A\}$ is the indicator of condition A, $\mathbb{E}$ denotes expectation, and $\mathbb{P}$ probability. Top_m returns at most m elements with the highest scores; ties are resolved by lexicographic identifier order. Time is measured in seconds.

Symbol	Meaning
s, i, j	Session and item identifiers.

Symbol	Meaning
$e, t(e), a(e)$	An event, its time, and the time it becomes available to the system.
$q, s(q), t_q$	A recommendation request, its session, and its time.
$t_{0,s}, H, g$	First eligible request, outcome horizon, and inactivity gap, equal to 30 minutes for Coveo sessions.
$\mathcal{H}_q, \mathcal{F}_q$	Available history and all information available to the algorithm at request time.
$r_G, r_N, r(q), \mathcal{I}_q$	Times of the graph snapshot, publication of the learned retriever, the snapshot used, and the eligible catalog.
$\mathcal{Q}_s, Q_{\max}$	Session requests and their predefined limit.
π, π_0	Complete recommendation policy and comparison policy.
π_{base}	The first implementation, GCS+Rk.
$\mathcal{C}_q(\pi), M$	Candidates and maximum candidate count.
$L_q(\pi), K$	Final list and its length limit.
Y_q, y_{qi}	Set of future purchases after a request and binary label of candidate i.
$Y_s(\pi)$	Indicator of a purchase in a session when policy π is actually used. Distinct from the set Y_q.
$B(\pi), \Delta B$	Purchasing sessions per 1000 eligible sessions and their causal increase.
$\text{Hit}_s(\pi), \text{Hit@20}_J$	A first-list hit on recorded future purchases and its rate among sessions with such a target.
$f_q(i)$	Ranking score, not a purchase probability.
Z_s	Assigned experiment arm for impressions, 0 or 1.
$\xi, \widehat{\xi}$	Procedure configuration and selected configuration.
A_G, A_N, B, C, E, D	Blocks for topology and statistics, neural retriever training, ranker training, selection, reserve, and final evaluation; $A = A_G \cup A_N$.

The index r on an aggregate restricts events to those available at the snapshot. Adding a score to a monetary value requires a justified conversion coefficient. Comparison policy π_0 is fixed in advance; it need not coincide with π_{base}. Block A_N is reserved for the neural model and does not augment the graph or item statistics in the first run.

3 Coveo data and observation limits

3.1 Observed objects

The release contains tables of behavior, search events, and item features [1, 2]. The formulas below use the following event representation:

$$e = (s, i, t, a, \kappa, \eta), \qquad \kappa \in \{\text{detail}, \text{add}, \text{remove}, \text{purchase}, \text{search}, \text{page}\}. \tag{1}$$

Here $i = \varnothing$ is allowed for a page without an item and for search; η contains additional fields. Time t is obtained by dividing `server_timestamp_epoch_ms` by 1000. Ingestion times a are absent from the original dataset: in historical replay, the primary model assumes $a = t$, while delays are examined in a separate test.

Search is represented as

$$v = (s, t, u, R, C), \tag{2}$$

where u is `query_vector`, R is the list `product_skus_hash`, and C is `clicked_skus_hash`. The item content vector d_i comes from `description_vector`, and the image vector v_i from `image_vector`. Category c_i is a hashed hierarchy, and b_i^{price} is an ordinal price bucket.

3.2 What cannot be recovered from the dataset

Missing object	Consequence for the formulation
Actual order price, quantity, and margin	Monetary revenue and profit cannot be computed from a price bucket number.
Impression log for our recommendation widget	Agreement with a purchase does not establish the recommendation's influence.
Recommendation assignment probabilities	The primary historical experiment does not use causal reweighting of impressions.
Historical inventory and item activation dates	Catalog eligibility is defined by an observation rule, not by stock availability.
Time of each click within a search row	Field C cannot be used as a feature of the current or next request without a separate timestamped event.
Initial cart contents and quantities	Cart state denotes the last observed action, not an exact balance.
Historical content encoder versions	Use of the provided vectors is conditional on the released feature snapshot.
Confirmation of telemetry completeness	An empty set of recorded purchases does not prove that no purchase occurred. Evaluation over positively observed targets is defined in Section 14.1.

Assumption 3.1 (Availability of static features)**.** In historical evaluation, the provided vectors and categories are assumed available for items in the historical catalog. This is an assumption about side information, not evidence of its actual availability in the past. A mandatory comparison without the provided metadata tests sensitivity to this assumption (Section I.1). It does not recover the unknown historical version of the vectors.

Price buckets are encoded as categories or compared by order and equality; their differences and ratios have no monetary meaning. Hashed categories are compared by equality and their known hierarchy.

3.3 Snapshot and reproducible subsample

A data snapshot is defined by the tuple

$$\mathsf{D} = (\text{version}, \text{hashes}, \text{schema}, T_{\min}, T_{\max}, \text{sampling}, \text{cleaning}). \tag{3}$$

Within predefined time blocks, sessions are sorted by a stable pseudorandom key $h_{\text{seed}}(s)$. The initial algorithm is SHA-256 over the concatenation of the seed as an unsigned 64-bit big-endian integer, the ID byte length as an unsigned 64-bit big-endian integer, and the exact UTF-8 bytes of the ID, without case conversion or Unicode normalization. Sorting is lexicographic over the 32 digest bytes, with ties resolved by those same ID bytes; the seed is 17. Size n is set separately for each period before inspecting its outcomes. The first n identifiers are taken with all behavior and search rows, regardless of future purchases. This produces nested training samples, such as 200 and 500 thousand sessions.

The published counts include sessions, events, items, purchasing sessions, and search queries, as well as the fractions of missing vectors and unknown test items. Purchasing sessions are counted by identifier, not by `purchase` rows; catalog size is reported separately from sample size.

4 Time, cleaning, and available history

4.1 Event ordering

Behavior and search are merged by s and time; simultaneous events form a block. Row numbers define storage order, not causal order. Under historical $a = t$, a block contains all rows at that time; in the live service, its completeness is defined by the closed envelope in Section 16.8.

A single item view may generate both `detail` and `pageview`. The primary item sequence uses `product_action`; a page without an action does not create a second item transition. All source rows are retained for audit. Without an event identifier, exact row equality alone does not prove a technical duplicate. The primary variant uses the declared canonicalization `event_semantics=set_v1`: each combination (s, t, i, κ) maps to one canonical event, retaining the multiplicity of its source rows. For search, its payload additionally preserves the set of distinct `query_vector` values and the multiplicity of each, rather than the first or last value. Valid vectors are compared before normalization: by dimension and exact FP64 coordinates, with -0 and $+0$ treated as equal. Missing and invalid fields form separate markers with reason codes; source rows are retained. Fields R, C from (2) do not participate in this representation. A search block is unambiguous only if it contains one valid value and no markers. Two or more distinct valid vectors set an ambiguity flag; a marker also disables this context with the corresponding reason. Repeated copies of a vector do not create ambiguity. Row order does not select one of the queries. Below, $\mathcal{H}_q$, view counters, history $\mathcal{D}_q$, and the sequence encoder use these canonical events. This is a representation assumption, not a reconstruction of the true action count; source rows and their multiplicities are not removed.

A sensitivity check on C_{smoke} is mandatory for `event_semantics=multiset_v1`: multiplicity is included in counters, history, and averaging within time groups. Binary purchase targets remain sets, and edge support still counts distinct sessions. The report includes the fraction of affected sessions and changes in features and coverage. The choice of semantics is fixed before C_{select} and is shared by the models being compared. Transport retries in the live service are deduplicated by stable event ID before this operation; matching fields are not a substitute.

Invariant 4.1 (Simultaneity)**.** No directed transitions are constructed between events in the same time block, and no targets are assigned as "after the request." The request runs after the entire available block has been processed; its future targets have strictly greater times.

4.2 Information at decision time

Let a_q be the computation start time. Then

$$\mathcal{H}_q = \{e : s(e) = s(q),\ t(e) \le t_q,\ a(e) \le a_q\}. \tag{4}$$

$\mathcal{F}_q$ includes $\mathcal{H}_q$, available reference-data snapshots, published model parameters, and service state. Serving and independent application of the ranker on C, E, D require

$$\mathcal{C}_q,\ f_q,\ L_q \quad \text{to be functions only of } \mathcal{F}_q. \tag{5}$$

Future session length, final conversion, the next search, and future popularity are prohibited. Snapshot $r(q) < t_q$ is selected by its publication time. In B, the ranker uses historically eligible features and candidates, but training predictions are not an independent evaluation. The neural model trains on continuations of examples in A_N and is evaluated on subsequent independent periods.

The initial and neural variants use only u, without result list R, which depends on the unknown search engine policy. Reordering the returned R is a separate task; C remains a subsequent response. Its results are not combined with recommendation results.

4.3 Observed cart state

Define $b_{qi} \in \{\text{add}, \text{remove}, \text{unknown}\}$. Its initial value is unknown. The latest unambiguous observed `add` sets add, and `remove` sets remove. If the latest time group contains both actions, the value becomes unknown again. A repeated removal does not create a negative item quantity.

The model uses indicators of the three states and time since the latest unambiguous event. Actual cart contents are not reconstructed.

5 Requests, eligible items, and future targets

5.1 Observation start and schedule

A session is eligible if it contains a `detail` with a valid item identifier and no purchase has been observed by the end of that time block. The first such block defines $t_{0,s}$. The rule does not require a future purchase, multiple views, a content vector, or a nonempty response from the selected algorithm. Sessions that start with a purchase before the first eligible view are outside the target population; their fraction is published.

The sole request $q_s^{(1)}$ runs at $t_{0,s}$: $\mathcal{Q}_s = \{q_s^{(1)}\}$, $Q_{\max} = 1$ for all methods. Its prefix includes the entire block of simultaneous events. A future purchase does not select the request time. Subsequent lists are not evaluated in this experiment. The latest search block is checked for ambiguity as specified in Section 4.1.

Remark 5.1. The historical schedule $\mathcal{Q}_s$ follows the recorded path. Recommendations may change future actions and requests; log replay does not model this change.

5.2 Recorded target window

Let $t_{\text{last},s}^{\text{obs}}$ be the time of the last recorded event in the session's merged behavior and search stream. The target window is

$$\tau_s = \min(t_{0,s} + H, t_{\text{last},s}^{\text{obs}}). \tag{6}$$

The last record and a wait of $g = 1800$ seconds do not confirm session closure or telemetry completeness. The calendar buffer $H + g$ in Section 13.3 protects period boundaries without proving the absence of missing purchases. The first version uses horizon $H = 86400$ seconds.

5.3 Catalog and labels

In the primary historical variant, the catalog is fixed using A_G:

$$\begin{aligned} \mathcal{I}_G &= \{i : \exists e \text{ from eligible sessions in } A_G,\ i(e) = i\}, \\ \mathcal{I}_q &= \mathcal{I}_G \setminus \mathcal{P}_q. \end{aligned} \tag{7}$$

Here $\mathcal{P}_q$ denotes items already purchased in the visible prefix. The first request is admitted before any observed purchase, so $\mathcal{P}_q = \varnothing$. The exclusion remains a required safeguard when reusing the service. Recommendations after a purchase, repeat purchases of consumables, and item quantities require a separate task with a newly specified schedule, catalog, and labels; the binary target of the current experiment does not model quantity. Metadata do not confirm the historical availability of future items. A prefix item unknown to the snapshot receives the corresponding flag; both primary and fallback recommendations stay within $\mathcal{I}_q$.

In a real store, set (7) is replaced by the catalog available at request time, subject to stock, region, and publication restrictions. The quality of such filtering is not measured on Coveo. Items with an observed `add` are allowed in the primary variant; recommendations restricted to items new to the cart are evaluated separately with the same denominator.

For $s = s(q)$, the recorded target set and candidate labels are

$$Y_q = Y_s^{\text{obs}} = \{i : \exists e \text{ in the export},\ s(e) = s,\ \kappa(e) = \text{purchase},\ t_{0,s} < t(e) \le \tau_s,\ i(e) = i\}, \quad (8)$$

$$y_{qi} = \mathbf{1}\{i \in Y_q\}. \quad (9)$$

The target contains every distinct purchased SKU in the window, not only the first or last. Repeated purchase records contribute one set element; multiple items in the same time block are retained. Items in Y_q that are absent from $\mathcal{I}_q$ are not removed from evaluation. This separates catalog coverage error from ranking error. An empty Y_q means no target purchase was recorded, not a confirmed non-purchase. Even when $Y_q \neq \varnothing$, completeness of the purchase list is unknown. A zero candidate label means only that the item is absent from Y_q; its use is governed by Section 11.

6 Primary business criterion

6.1 Purchasing sessions and causal lift

Policy π includes the model, recommendation rules, fallback behavior, and request-count limits. Let $Y_s(\pi)$ denote the indicator of at least one purchase after policy assignment and before the end of its outcome window. A policy change may alter the session path and its end. The outcome is defined by the same window rule for every possible path.

Definition 6.1 (Policy value)**.** For a predefined population of eligible sessions,

$$B(\pi) = 1000\,\mathbb{E}[Y_s(\pi)], \qquad \Delta B(\pi, \pi_0) = 1000\,\mathbb{E}[Y_s(\pi) - Y_s(\pi_0)]. \quad (10)$$

The unit of ΔB is additional sessions with a purchase per 1000 eligible sessions.

Below, "eligible" means sessions admitted under the rule in Section 5. Let $\mathcal{S}_{\text{all}}$ be all recorded sessions from the same source and calendar window, and $\mathcal{S}_{\text{elig}}$ those that pass this rule. Report separately $r_{\text{elig}} = |\mathcal{S}_{\text{elig}}|/|\mathcal{S}_{\text{all}}|$ and exclusion counts by reason; outcome maturity is assessed in the next, separate step. With an empty denominator, the fraction is unavailable. Formula (10) is conditional on $s \in \mathcal{S}_{\text{elig}}$. It does not measure conversion across all visits. Multiplying by r_{elig} alone does not yield the overall effect: this requires unchanged eligibility, no effect outside the eligible population, and a common source population. A KPI for all visits requires a separate experiment with assignment before selection and its own denominator. When assignment occurs only after admission, the admission rate cannot be broken down by an arm that has not yet been assigned.

In A/B testing with a stable key, the primary cohort is further restricted to entry sessions $\mathcal{S}_{\text{entry}} \subseteq \mathcal{S}_{\text{elig}}$ under F.1. Then (10) and the business threshold explicitly use $B_{\text{entry}}(\pi) = 1000\mathbb{E}[Y_s(\pi) \mid$

$s \in \mathcal{S}_{\text{entry}}$], and the fraction of this cohort among eligible sessions is also reported. Confirmation of ΔB_{entry} does not extend to all $\mathcal{S}_{\text{elig}}$; a product needing such generalization requires a separate estimand and protocol. The notation ΔB in acceptance criteria always specifies the chosen population.

Any purchase in an eligible session counts as one success regardless of the number of items or a recommendation click. Selection by such a click would depend on behavior after arm assignment.

For example, increasing the fraction of purchasing sessions from 0.030 to 0.032 corresponds to $\Delta B = 2$, an absolute increase of 0.2 percentage points, and a relative increase of about 6.7%. This is an arithmetic example, not an effect estimate for Coveo or the system being developed.

The deployment configuration is selected by solving

$$\max_{\pi \in \Pi} \Delta B(\pi, \pi_0) \quad \text{subject to} \quad \begin{cases} \mathbb{P}(\ell_q > L_{\max}) \le \rho_L, \\ \mathbb{P}(\text{error}_q = 1) \le \rho_{\text{err}}, \\ \mathbb{E}[C_s^{\text{serve}}] \le c_{\max}, \\ L_q \text{ satisfies item constraints.} \end{cases} \tag{11}$$

Here ℓ_q is response latency, and C_s^{serve} is the cost of serving a session in the chosen currency unit. The product owner sets the constraints before comparing systems. When monetary rates are unavailable, a measurable computational budget is fixed instead of cost; no monetary savings are claimed.

6.2 Why a purchase label does not solve the causal problem

A historical log contains one outcome for each session, not both $Y_s(\pi)$ and $Y_s(\pi_0)$. Predicting a future purchase can select a model for testing but does not identify the difference (10).

Proposition 6.1 (Effects are not identifiable from matches)**.** For $0 < p_0 = \mathbb{E}[Y_s(\pi_0)] < 1$, the same distribution of observed purchases and the same recommendation matches are compatible with zero, positive, or negative effects of a new widget.

Proof. Fix the outcome distribution under the original policy. Three models with $Y_s(\pi) = Y_s(\pi_0)$, $Y_s(\pi) = 1$, and $Y_s(\pi) = 0$ produce effects 0, $1000(1 - p_0) > 0$, and $-1000p_0 < 0$, respectively, but agree on the original log because the new policy was never assigned there. In general, binary outcomes bound the effect to the interval $[-1000p_0,\ 1000(1 - p_0)]$. At $p_0 = 0$ or $p_0 = 1$, not every sign is possible, but the effect magnitude remains unidentified. The absence of one class in a finite log does not prove the corresponding boundary value of p_0. □

6.3 Monetary extension when the required data become available

If a real system provides paid order lines, returns, cost of goods, and additional expenses, one can define

$$G_s(\pi) = \sum_{o \in \mathcal{O}_s(\pi)} \left(R_o - C_o^{\text{goods}} - C_o^{\text{delivery}} - C_o^{\text{return}} \right) - C_s^{\text{serve}}, \tag{12}$$

where $\mathcal{O}_s$ denotes orders attributed to the session under a predefined rule. All terms must use the same currency unit and consistent treatment of taxes and returns. Then $1000\mathbb{E}[G_s(\pi) - G_s(\pi_0)]$ is the additional contribution margin per 1000 sessions. The return horizon is set separately. In Coveo, this functional has status `not_observed`.

A price bucket cannot substitute for R_o. Converting ΔB into money requires assumptions about order counts, baskets, and margins, which may change after deployment. The primary business criterion is ΔB.

6.4 Levels of evidence

Level	Supported conclusion	Required data
Historical suitability	The system finds future purchases in a fixed log and stays within budget.	Coveo and a reproducible protocol.
Operational suitability	Requests are served within the required latency using correct features.	Load tests and a shadow deployment.
Business superiority	ΔB exceeds a predefined practical threshold.	Randomized assignment and mature outcomes.
Monetary benefit	Additional margin is positive.	An experiment, financial data, and completed accounting for returns.

7 Item interaction graph

The graph uses history $\mathcal{H}_q$ and eligible catalog $\mathcal{I}_q$ from Sections 4 and 5. The time unit $t_{\text{unit}} = 1\,\text{s}$ makes logarithm arguments dimensionless. Topology and statistics are built only from A_G by time r_G. For recorded events, history eligibility is defined in Section 14.1; confirmed telemetry completeness is not assumed for this protocol. The baseline policy requires $r_G < t_q$; there is no separate training of a graph encoder. At the second stage, the neural retriever is trained on A_N and published by r_N, so its requests from B, C, E, D additionally require $r_G < r_N < t_q$.

7.1 Vertices, relations, and one session's contribution

A graph vertex is an item identifier. Three relations are defined:

$$\mathcal{R}_G = \{\text{detail} \to \text{detail},\quad \text{detail} \to \text{add},\quad \text{detail} \to \text{purchase}\}. \tag{13}$$

All events, including purchases, must precede the snapshot.

For a historical session s, item events are combined into nonempty groups sharing a timestamp; groups are ordered by time. Let $i_{s\ell}, \kappa_{s\ell}, t_{s\ell}$ denote the item, action, and time of event ℓ, and $k_s(\ell)$ its time-group index. No directed edges are created within a group: the order is unknown. For relation $r = (\kappa_1, \kappa_2)$, the set of eligible pairs is

$$\begin{aligned}\mathcal{P}^r_{sij}(r_0) = \{(\ell, m) : {} & i_{s\ell} = i,\ i_{sm} = j,\ i \neq j,\ \kappa_{s\ell} = \kappa_1,\ \kappa_{sm} = \kappa_2, \\ & 0 < t_{sm} - t_{s\ell} \leq \Delta_G,\quad 0 < k_s(m) - k_s(\ell) \leq B_G,\quad t_{sm} < r_0\}.\end{aligned} \tag{14}$$

Here r_0 is snapshot time, $\Delta_G > 0$ is the time window, and $B_G \in \mathbb{N}$ is the maximum distance between time groups of item events. Events without an item do not create such groups. Pairs do not

depend on row order within a group; repeated item occurrences are handled by session features, without self-loops $i \to i$.

A session's contribution is bounded by one and defined by a maximum:

$$w^r_{sij} = \begin{cases} \max\limits_{(\ell,m)\in\mathcal{P}^r_{sij}(r_0)} \exp\left(-\dfrac{t_{sm} - t_{s\ell}}{\tau_G}\right), & \mathcal{P}^r_{sij}(r_0) \neq \varnothing, \\ 0, & \text{otherwise}, \end{cases} \tag{15}$$

where $\tau_G > 0$ has time units. Then

$$W^r_{ij} = \sum_s w^r_{sij}, \qquad n^r_{ij} = \sum_s \mathbf{1}\{w^r_{sij} > 0\}, \qquad 0 \le W^r_{ij} \le n^r_{ij}. \tag{16}$$

The sums use snapshot sessions truncated at the snapshot boundary under the data preparation protocol.

7.2 Sparsification and graph-row regularization

Among edges with support $n^r_{ij} \ge n_G \ge 1$, retain at most d_G neighbors with the greatest W^r_{ij} for each pair (r, i). Ties are resolved by a fixed identifier order. The retained neighbor set is denoted $\mathcal{N}^r_i$.

$$P^r_{ij} = \begin{cases} \dfrac{W^r_{ij}}{\sum_{k\in\mathcal{N}^r_i} W^r_{ik}}, & j \in \mathcal{N}^r_i, \\ 0, & \text{otherwise}. \end{cases} \tag{17}$$

An empty row is zero; a nonempty row sums to one. P^r_{ij} expresses normalized association strength, not the probability of the next action: one session may produce several neighbors.

To attenuate rows with low total weight, use

$$d^r_i = \sum_{j\in\mathcal{N}^r_i} W^r_{ij}, \qquad \rho^r_i = \frac{d^r_i}{d^r_i + \lambda_G}, \qquad \lambda_G > 0. \tag{18}$$

λ_G has the same arbitrary units as d^r_i. Coefficient ρ^r_i attenuates low-weight rows; it does not estimate edge confidence. For a retained edge,

$$\rho^r_i P^r_{ij} = \frac{W^r_{ij}}{d^r_i + \lambda_G}, \qquad 0 \le \rho^r_i P^r_{ij} \le P^r_{ij} \le 1. \tag{19}$$

At fixed W^r_{ij}, increasing other neighbors' weights attenuates the edge. The threshold n_G provides additional protection against accidental associations.

Let $\mathcal{D}_q$ be the latest at most L_H available `detail` events. When truncating history, equal timestamps are resolved by item identifier; this is a technical selection rule, not a reconstruction of actual action order. Normalized history weights are defined as

$$\alpha_{q\ell} = \frac{\exp(-(t_q - t_{s(q)\ell})/\tau_H)}{\sum_{m\in\mathcal{D}_q} \exp(-(t_q - t_{s(q)m})/\tau_H)}, \qquad \ell \in \mathcal{D}_q, \quad \tau_H > 0. \tag{20}$$

For empty history, sums involving $\alpha_{q\ell}$ are zero. Cart actions are represented by separate features.

The graph score of candidate j for relation r is

$$g^r_{qj} = \sum_{\ell\in\mathcal{D}_q} \alpha_{q\ell} \rho^r_{i_{s(q)\ell}} P^r_{i_{s(q)\ell},j}. \tag{21}$$

Each relation's score is a separate feature: $0 \le g^r_{qj} \le 1$, with $g^r_{qj} = 0$ for empty history. A control variant may use the mixture $g_{qj} = \sum_r \omega_r g^r_{qj}$, where $\omega_r \ge 0$, $\sum_r \omega_r = 1$. This mixture is not required in the first run: each relation generates its own candidates and features.

Additional attenuation of individual edges is described in Appendix E.1. The first run uses only (21).

Invariant 7.1 (Historical information boundary)**.** Changing events with time $t \ge r(q)$ does not change W^r, P^r, ρ^r of this snapshot. Changing events with time $t > t_q$ does not change $\alpha_{q\ell}$ or g^r_{qj}.

7.3 Construction and request cost

Let N_{A_G} be the number of historical item events, and $b^{\max}_G$ the maximum size of one time group in the snapshot. After sorting and grouping, pair enumeration requires on the order of $N_{A_G} B_G b^{\max}_G$ operations: for each event, up to B_G subsequent groups of size at most $b^{\max}_G$ are checked. Observed $b^{\max}_G$ is included in the cost report; arbitrary splitting of a simultaneous group is prohibited. The number of retained edges is at most $|\mathcal{R}_G|\,|\mathcal{I}_G| d_G$. Computing graph candidates for one request accesses at most $|\mathcal{R}_G| L_H d_G$ edges. Intermediate aggregation memory is measured separately from the final graph. No dense $|\mathcal{I}_G|^2$ matrix is created; B_G, L_H, d_G bound the computational cost.

8 Content features and search context

8.1 Separate representation spaces

For item i, denote the provided description and image vectors by $v^{\text{text}}_i \in \mathbb{R}^{d_t}$ and $v^{\text{image}}_i \in \mathbb{R}^{d_v}$. In catalog notation, $v^{\text{text}}_i = d_i$, $v^{\text{image}}_i = v_i$. The coordinates of the two spaces have different meanings even when $d_t = d_v$. Coveo documentation states that the query vector is compatible with the item description vector; compatibility with the image vector is not claimed [1].

For each modality $m \in \{\text{text}, \text{image}\}$, define a mask M^m_i. It equals one if the vector exists, has the expected dimension, finite coordinates, and a norm greater than a fixed $\varepsilon_v > 0$. Define

$$\bar{v}^m_i = \begin{cases} v^m_i / \left\| v^m_i \right\|, & M^m_i = 1, \\ 0, & M^m_i = 0. \end{cases} \tag{22}$$

A missing vector is represented by zero and a mask; a zero cosine between valid vectors remains an observed value.

The history representation is built separately in each space:

$$u^m_q = \sum_{\ell \in \mathcal{D}_q} \alpha_{q\ell} M^m_{i_{s(q)\ell}} \bar{v}^m_{i_{s(q)\ell}}, \tag{23}$$

$$M^m_q = \mathbf{1}\{\left\| u^m_q \right\| > \varepsilon_v\}, \qquad \bar{u}^m_q = \begin{cases} u^m_q / \left\| u^m_q \right\|, & M^m_q = 1, \\ 0, & M^m_q = 0. \end{cases}$$

The norm of the sum is checked because directions may cancel. The content score and its mask are

$$c^m_{qi} = (\bar{u}^m_q)^\top \bar{v}^m_i, \qquad M^m_{qi} = M^m_q M^m_i. \tag{24}$$

For $M_{qi}^m = 1$, the score lies in $[-1, 1]$. Retrieval may use a weighted mixture of available modalities only:

$$c_{qi} = \begin{cases} \dfrac{\sum_m \gamma_m M_{qi}^m c_{qi}^m}{\sum_m \gamma_m M_{qi}^m}, & \sum_m \gamma_m M_{qi}^m > 0, \\ 0, & \text{otherwise}, \end{cases} \qquad \gamma_m \geq 0. \tag{25}$$

The ranker receives separate scores and masks. Variants are compared under identical candidate quotas.

8.2 Latest available search query

Let $b(q)$ be the latest search block available by t_q, and $t_{b(q)}$ its time. If there is none, or if $t_q - t_{b(q)} > \Delta_S$, search context is considered absent. An unusable latest block under Section 4.1 disables search; there is no return to an earlier block. Threshold Δ_S is specified in seconds. A valid query vector is normalized by rule (22); the result is denoted $\bar{z}_q \in \mathbb{R}^{d_t}$, and the availability mask is M_q^S.

$$s_{qi}^{\text{search}} = \bar{z}_q^\top \bar{v}_i^{\text{text}}, \qquad M_{qi}^S = M_q^S M_i^{\text{text}}. \tag{26}$$

s_{qi}^{search} measures query–description similarity; comparison with a graph score requires training or normalization.

Additional search-context features are

$$d_q^S = \begin{cases} t_q - t_{b(q)}, & M_q^S = 1, \\ 0, & M_q^S = 0, \end{cases} \qquad M_q^S, \quad a_q^S = \log\left(1 + \frac{d_q^S}{t_{\text{unit}}}\right). \tag{27}$$

Quantity d_q^S is measured in seconds; a_q^S is dimensionless. When search is absent, both age and mask are zero; the missing $t_{b(q)}$ is not evaluated in the zero branch. Field `product_skus_hash` is excluded from features and retrieval: candidates are selected from $\mathcal{I}_q$ by similarity to `query_vect or`, without copying the historical search results.

Invariant 8.1 (Feedback after a search query)**.** Field `clicked_skus_hash` is not a feature of the current search query: it contains subsequent clicks. Without individual click times, the field is also excluded from state reconstruction for a later request. A separate item-view event with a known time may be included under the normal availability rule.

Search-click prediction is allowed as a separate experiment with its own labels. Absence of a click does not imply a negative preference and does not change the purchase objective in Section 6.

8.3 Categories and price bucket

For categories of the candidate and a viewed item with path lengths $\ell_i^{\text{cat}}, \ell_j^{\text{cat}}$, similarity is defined as

$$c_{ij}^{\text{cat}} = \frac{\text{lcp}(i, j)}{\max(1, \ell_i^{\text{cat}}, \ell_j^{\text{cat}})}, \tag{28}$$

where lcp is the number of matching initial categories. For a missing path, the value is zero and a separate mask is set. A hash serves as a categorical label.

For ordinal feature `price_bucket`, use equality, the sign and absolute difference of bucket indices, with missing-value masks.

9 Candidate set construction

9.1 Sources and quotas

All sources are restricted to $\mathcal{I}_q$ under Section 5: already purchased items are excluded; viewed items and items observed in the cart are retained and evaluated in separate slices. Let $\mathcal{C}_q^{(a)}$ denote the list from source a. The first run uses six source groups:

1. the most recent distinct items from available views;
2. neighbors under each of the three graph relations;
3. items most similar in the description space;
4. items most similar in the image space;
5. eligible catalog items similar to the latest search query;
6. historically popular items;

Initial quotas are 50 candidates for each of the three graph relations, 40 for descriptions, 30 for images, 50 for search, 20 recent items, and 10 popular items. Their sum is $M = 300$. A neural source is added only at the second stage (Section 22.5); a semantic-code source is introduced in a separate study (Section D.5). An unavailable source returns an empty list and a reason code. A content source is enabled only with a valid history representation; arbitrary sorting of entirely missing scores is not used.

Quotas $m_a \geq 0$ are fixed before final evaluation:

$$\mathcal{C}_q^{\text{pre}} = \bigcup_a \mathcal{C}_q^{(a)}, \qquad |\mathcal{C}_q^{(a)}| \leq m_a, \qquad \sum_a m_a \leq M. \tag{29}$$

Duplicates are merged, preserving all source features. The neural-source quota covers all its subqueries; a new source redistributes quotas within the existing M. After merging, remaining slots are filled by popularity from $\mathcal{I}_q \setminus \mathcal{C}_q^{\text{pre}}$. Let $\mathcal{C}_q^{\text{fill}}$ be the first at most $M - |\mathcal{C}_q^{\text{pre}}|$ such items. The final set is

$$\mathcal{C}_q^{\text{main}} = \mathcal{C}_q^{\text{pre}} \cup \mathcal{C}_q^{\text{fill}}, \qquad |\mathcal{C}_q^{\text{main}}| \leq M. \tag{30}$$

Filling has a separate feature and is not limited by the initial popularity quota. It stops at M or catalog exhaustion. Adding the future correct answer is prohibited.

This is the primary candidate set. Before ranking, $\mathcal{C}_q^{\text{fallback}}$ is prepared separately: up to M eligible items, starting with distinct recent viewed items in descending time order (ties resolved by identifier), then popularity without duplicates. Fallback is not merged into the primary set beyond the budget. The normative set for the branch actually selected is

$$\mathcal{C}_q^{\text{served}} = \begin{cases} \mathcal{C}_q^{\text{main}}, & \text{branch}_q = \text{main}, \\ \mathcal{C}_q^{\text{fallback}}, & \text{branch}_q = \text{fallback}, \\ \varnothing, & \text{branch}_q \in \{\text{empty}, \text{error}\}. \end{cases} \tag{31}$$

The branch is fixed when choosing a response, reserving time for mandatory logging and return. If the selected branch subsequently fails to return in time, retain its candidates, $L_q = \varnothing$, and a separate failure code: the loss belongs to delivery, not retrieval. The value `error` in the formula means a failure before selecting a ready branch. In all cases, $L_q \subseteq \mathcal{C}_q^{\text{served}} \subseteq \mathcal{I}_q$ and $|\mathcal{C}_q^{\text{served}}| \leq M$. Below, $\mathcal{C}_q$ without a superscript denotes $\mathcal{C}_q^{\text{served}}$; for normal ranker training on B, it is $\mathcal{C}_q^{\text{main}}$. Fallback has its own fixed order and is not trained using the primary ranker's gradients. An unfinished primary set is logged as unmeasured, not as an empty diagnostic list.

Historical popularity is defined by the number of distinct snapshot sessions with a view of the item:

$$V_i^{\text{detail}} = \sum_{s \in A_G} \mathbf{1}\{s \text{ contains an available view of } i\}, \tag{32}$$

$$\pi_i^{\text{pop}} = \frac{V_i^{\text{detail}} + \eta_P}{\sum_{j \in \mathcal{I}_G} (V_j^{\text{detail}} + \eta_P)}, \quad \eta_P > 0.$$

At request time, popularity is restricted to $\mathcal{I}_q$. An unknown item may change session state and its content representation but receives no historical edges. Admission of new items follows the availability rule in Section 5.

9.2 Neighbor search in vector space

The first run computes similarity against every eligible vector: the cost for a modality of dimension d_m is on the order of $|\mathcal{I}_q| d_m$. If measured latency is excessive, approximate nearest-neighbor search (ANN) is tested separately. Its settings are fixed before final evaluation; a separate request sample is used to measure

$$\text{Agreement@}m = \frac{1}{N_Q^+} \sum_{q:\, |\mathcal{C}_{q,\text{exact}}| > 0} \frac{|\mathcal{C}_{q,\text{approx}} \cap \mathcal{C}_{q,\text{exact}}|}{|\mathcal{C}_{q,\text{exact}}|}. \tag{33}$$

N_Q^+ is the number of nonempty exact responses; when $N_Q^+ = 0$, the metric is undefined. The empty-response fraction is reported separately. An identical eligibility filter allows approximation losses to be measured, but agreement with exact search does not replace purchase evaluation.

9.3 Candidate provenance features

For each source, retain its mask, original score, and reciprocal rank:

$$o_{qi}^{(a)} = \mathbf{1}\{i \in \mathcal{C}_q^{(a)}\}, \qquad u_{qi}^{(a)} = \begin{cases} 1/(1 + \text{rank}_a(q,i)), & o_{qi}^{(a)} = 1, \\ 0, & o_{qi}^{(a)} = 0. \end{cases} \tag{34}$$

Ranks start at one; missing ranks and scores are represented by zero with a mask. A measurable score is computed for all candidates in the union, even if the item did not enter the corresponding source's quota. The transformation $1/(1 + \text{rank})$ intentionally gives $1/2$ to the first position; this is a bounded feature, not a probability.

10 Request–item pair features

Vector $x_{qi} \in \mathbb{R}^p$ contains only quantities available at q. The category transformation is fixed with the model; an item identifier is not used as a real-valued number. The table defines features for the first run.

Group	Contents
Graph	g_{qi}^r, connection to the latest item group, edge support, masks for missing neighbors

Description and image	c^m_{qi}, similarity to the latest item group, vector masks and empty-history-representation masks
Search	s^{search}_{qi}, query age, availability masks for the query and item description
Item history	Number of available views, time since the latest view, latest add or remove action
Session history	Number of events and distinct items, duration of available history, search presence, number of recorded adds
Catalog	Category, category matches, price bucket, and its difference from buckets of the most recently viewed items
Snapshot	Popularity and number of sessions with a recorded purchase of the item
Retrieval	Source masks, original scores, and reciprocal ranks

The second stage adds the learned session-vector score, the maximum intent score, intent-score variance, weight entropy, search similarity in the learned space, and availability masks. The semantic-code study adds the log probability of the code, the number of items with that code, and a mask indicating a found code. A code probability is not a purchase probability. When the feature set changes, the ranker is retrained on B; neural features are not needed to implement the first stage.

Set $\mathcal{I}^{\text{last}}_q$ contains all distinct items in the latest `detail` time group, without selecting a single last item. Weight n^{last}_{qj} equals 1 in `set_v1` and the view multiplicity of j in that group in `multiset_v1`, after removal of transport retries. For pairwise feature $\phi(i,j)$ with availability mask M^{ϕ}_{ij}, its mean and mask are

$$
\begin{aligned}
Z^{\phi}_{qi} &= \sum_{j \in \mathcal{I}^{\text{last}}_q} n^{\text{last}}_{qj} M^{\phi}_{ij}, \qquad M^{\text{last},\phi}_{qi} = \mathbf{1}\{Z^{\phi}_{qi} > 0\}, \\
\overline{\phi}_{qi} &= \begin{cases} \dfrac{\sum_{j \in \mathcal{I}^{\text{last}}_q:\, M^{\phi}_{ij}=1} n^{\text{last}}_{qj} \phi(i,j)}{Z^{\phi}_{qi}}, & Z^{\phi}_{qi} > 0, \\ 0, & Z^{\phi}_{qi} = 0. \end{cases}
\end{aligned}
\tag{35}
$$

For descriptions and images, $\phi(i,j)$ is the cosine of valid vectors, and $M^{\phi}_{ij} = M^m_i M^m_j$. For price buckets, average equality, $\text{sign}(b^{\text{price}}_i - b^{\text{price}}_j)$, and absolute difference; the mask requires both values to be known. An unavailable $\phi(i,j)$ is not evaluated. The graph maximum $\max_{j \in \mathcal{I}^{\text{last}}_q} \rho^r_j P^r_{ji}$ and maximum category similarity do not depend on multiplicity; an empty eligible set gives zero and mask 0.

The item i view count and time since its last view are defined as

$$
\begin{aligned}
n^{\text{detail}}_{qi} &= \sum_{e \in \mathcal{H}_q} \mathbf{1}\{i_e = i,\ \kappa_e = \text{detail}\}, \\
\delta^{\text{detail}}_{qi} &= t_q - \max\{t_e : e \in \mathcal{H}_q,\ i_e = i,\ \kappa_e = \text{detail}\}.
\end{aligned}
\tag{36}
$$

With no event, age and mask are zero. For numerical stability, the count may be transformed as $\log(1+n)$ and age as $\log(1+\delta/t_{\text{unit}})$.

Cart state and its age are defined in Section 4.

A smoothed purchase rate is not used: without confirmed observation completeness, its denominator is not established. This feature is excluded from the schema, not filled with zero. Recorded purchase counters are retained.

Statistics use only A_G; target encoding from labels in the ranker-training period is excluded. A new category receives a separate "unknown category" code.

11 Learning to rank future purchases

11.1 Labels, groups, and weights

The ranker trains on the first requests of sessions $\mathcal{S}_B^+$ with a recorded target (Section 14.1). The model computes score $f(x_{qi})$ with label $y_{qi} = \mathbf{1}\{i \in Y_q\}$, $i \in \mathcal{C}_q$. Request q forms a comparison group. A zero label means no observed purchase in the window, not a negative preference.

Positive and remaining candidates form the sets

$$\mathcal{C}_q^+ = \{i \in \mathcal{C}_q : y_{qi} = 1\}, \qquad \mathcal{C}_q^- = \{i \in \mathcal{C}_q : y_{qi} = 0\}, \qquad \mathcal{J}_q = \mathcal{C}_q^+ \times \mathcal{C}_q^-. \tag{37}$$

One session contributes one group. The weights are

$$w_q^{\text{sess}} = 1, \qquad w_q^{\text{pair}} = \frac{w_q^{\text{sess}}}{\max(1, |\mathcal{J}_q|)}. \tag{38}$$

After multiplication by Δ_{qij}, the sum of pair weights and gradient norm depend on candidate composition and current ranks; equal actual contributions from different sessions are not claimed. A group without a positive item contributes no pairwise gradient, but a session with a recorded target remains in the evaluation denominator. The target item is not added to candidates. If there are no training pairs, report insufficient data, not successful training.

Assumption 11.1 (Surrogate comparison under implicit feedback)**.** Each observed purchased candidate is trained to outrank every candidate without a purchase in the same window. This is an assumption about a useful surrogate for the historical task, not a statement about exposure or preference: the unpurchased item may never have been shown. The historical policy, availability, and popularity affect labels; without an impression log, this training does not eliminate selection bias.

This assumption does not require replacing the task with positive–unlabeled learning: its identifying assumptions would also need justification. Policy selection is checked by an independent historical comparison and then an A/B protocol.

11.2 Pairwise logistic loss

For positive i and negative j, let $d_{qij} = f(x_{qi}) - f(x_{qj})$. The basic training objective is

$$\mathcal{L}_{\text{pair}}(f) = \sum_q w_q^{\text{pair}} \sum_{(i,j)\in\mathcal{J}_q} \log(1 + \exp(-\beta_R d_{qij}))\,, \qquad \beta_R > 0. \tag{39}$$

Parameter β_R sets the score scale. Compute the stable softplus$(-\beta_R d_{qij})$, not a direct exponential of a large positive argument. At $d_{qij} = 0$, the derivative with respect to the positive score is $-w_q^{\text{pair}}\beta_R/2$, so minimization increases the positive score and decreases the negative score. The logistic transformation of their difference describes pair order, not the probability of an item purchase.

11.3 Greater weight for errors near the top of the list

In LambdaMART, pair weight depends on how much swapping the pair would change top-position quality [12]. Let r_{qi} be the current position of i among all candidates, and K_R the ranking-training depth. Ranks start at one; order is defined by key $(-f(x_{qi}), \mathrm{id}(i))$, with identifiers compared lexicographically by UTF-8 bytes. For binary labels, use gain $G_R(0) = 0$, $G_R(1) = 1$. Define

$$D_R(k) = \frac{\mathbf{1}\{k \le K_R\}}{\log_2(1+k)}, \qquad Z_q = \sum_{k=1}^{\min(K_R, |\mathcal{C}_q^+|)} D_R(k). \tag{40}$$

For $Z_q > 0$, normalized list quality is $\sum_{i \in \mathcal{C}_q} y_{qi} D_R(r_{qi})/Z_q$. Its absolute change after swapping a positive and a negative item is

$$\Delta_{qij} = \frac{|D_R(r_{qi}) - D_R(r_{qj})|}{Z_q}, \qquad \omega_{qij} = w_q^{\text{pair}} \Delta_{qij}. \tag{41}$$

At $Z_q = 0$, there are no pairs and gradients are zero. A pair with both positions below K_R has zero weight. If exactly one position is above the cutoff, use the full absolute discount difference; there is no additional pair truncation or normalization by the score difference.

Weights ω_{qij} are recomputed from current positions and held fixed when taking derivatives. This pseudo-gradient rule does not assume a single smooth function with all these derivatives. At $\Delta_{qij} = 1$, it reduces to pairwise logistic training.

For one pair, denote $p_{qij} = 1/(1 + \exp(\beta_R d_{qij}))$. Contributions to the first derivative and diagonal second derivative are

$$\begin{aligned} g_{qi}^{R,(ij)} &= -\omega_{qij}\beta_R p_{qij}, & g_{qj}^{R,(ij)} &= +\omega_{qij}\beta_R p_{qij}, \\ h_{qi}^{R,(ij)} &= \omega_{qij}\beta_R^2 p_{qij}(1-p_{qij}), & h_{qj}^{R,(ij)} &= h_{qi}^{R,(ij)}. \end{aligned} \tag{42}$$

Derivatives are summed over each candidate's pairs. The trees use a diagonal approximation, omitting mixed second derivatives. The group's gradients sum to zero: the absolute score level is unspecified. For numerical implementation, set $u = \beta_R d_{qij}$, $v = \exp(-|u|)$. Then $p = v/(1+v)$ for $u \ge 0$ and $p = 1/(1+v)$ for $u < 0$, while $p(1-p)$ is computed as $v/(1+v)^2$. This preserves small curvature without subtracting nearby numbers; no additional positive floor is imposed on h.

11.4 Tree ensemble and regularization

The ranker is a sum of trees:

$$f_{N_T}(x) = f_0 + \nu \sum_{m=1}^{N_T} T_m(x), \qquad 0 < \nu \le 1. \tag{43}$$

A tree leaf outputs one number; ν limits a new tree's contribution. Constant f_0 does not affect order.

For leaf ℓ of the next tree, denote $G_\ell = \sum_{(q,i)\in\ell} g_{qi}^R$ and $H_\ell = \sum_{(q,i)\in\ell} h_{qi}^R$. With penalty $\lambda_2 w_\ell^2/2$ and $\lambda_2 > 0$, the locally optimal leaf value is

$$w_\ell^* = -\frac{G_\ell}{H_\ell + \lambda_2}. \tag{44}$$

The formula follows by minimizing the quadratic expression $G_\ell w + (H_\ell + \lambda_2) w^2/2$. The reduction in this approximation when splitting a leaf into L, R is

$$\text{Gain} = \frac{1}{2}\left[\frac{G_L^2}{H_L + \lambda_2} + \frac{G_R^2}{H_R + \lambda_2} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda_2}\right] - \gamma_T. \tag{45}$$

$\gamma_T \geq 0$ is the penalty for adding a leaf. A split is allowed when its gain is positive and constraints hold on depth, leaf count, observation count, and the sum of second derivatives in each child leaf. The first run fixes a budget of 200 iterations; actual $N_T \leq 200$ is stored in the run manifest, because the absence of an eligible split may stop new-tree construction earlier. Later budget selection uses C_{select} under a predefined plan; after selection, the model is frozen until final evaluation.

11.5 Executable contract for custom derivatives

Use the Python API `lightgbm.train`, version `lightgbm==4.6.0`, with callable `objective=session_lambdas_v1` [13]. The function receives raw scores and a `Dataset`, and returns the sums g_{qi}^R, h_{qi}^R from (42). At every iteration, full ranks, Z_q, and all pairs from (37) are recomputed. This is a custom LambdaMART objective; equivalence to the built-in `objective=lambdarank` is not claimed. The built-in objective's parameters `label_gain`, `lambdarank_norm`, `lambdarank_truncation_level`, and `sigmoid` are not set here and do not control the callback: gain $[0, 1]$, $K_R = 20$, and $\beta_R = 1$ are implemented explicitly within it.

Rows are grouped by request, requests are ordered by $(s, t_q, \text{id}(q))$, and rows within a group by item identifier; all orders are part of the data schema. Nonempty group sizes are passed as `Dataset.group`. An empty group creates no rows, but remains in $|\mathcal{Q}_s|$ for weighting. Weights w_q^{pair} are applied once inside the callback; `Dataset.weight=None`, with no additional row/query weights. Groups without pairs receive zero derivative arrays. Sums are computed in FP64, then the callback returns contiguous FP32 arrays: this is the type accepted by the selected version's C API. Nonfinite scores, gradients, or Hessians stop training with an error instead of being replaced by zeros.

The complete first-ranker configuration below is supplemented only by the fixed feature schema and data; unspecified parameters use the defaults of version 4.6.0 specifically. The serialized run manifest contains default values, the derivative-function hash, a version file with package hashes, and platform/compiler information. A new library version requires the contract to be checked again.

```
objective=session_lambdas_v1 (callable); metric="None"
boosting="gbdt"; data_sample_strategy="bagging"
device_type="cpu"; tree_learner="serial"; num_threads=1
deterministic=true; force_col_wise=true; force_row_wise=false
learning_rate=0.05; num_leaves=31; max_depth=8
lambda_l1=0; lambda_l2=1; min_gain_to_split=0
min_data_in_leaf=100; min_sum_hessian_in_leaf=0.001
bagging_fraction=1; bagging_freq=0; bagging_by_query=false
feature_fraction=1; feature_fraction_bynode=1; extra_trees=false
max_bin=255; min_data_in_bin=3; bin_construct_sample_cnt=200000
feature_pre_filter=false; enable_bundle=false
use_missing=true; zero_as_missing=false; use_quantized_grad=false
boost_from_average=false; linear_tree=false
path_smooth=0; max_delta_step=0
```

```
cat_l2=0; cat_smooth=1; max_cat_threshold=32
max_cat_to_onehot=4; min_data_per_group=100
seed=17; data_random_seed=17; feature_fraction_seed=17
bagging_seed=17; extra_seed=17; verbosity=1
train: num_boost_round=200; valid_sets=None; feval=None
       init_model=None; callbacks=None
Dataset: weight=None; init_score=None
predict: raw_score=true; num_iteration=N_T
```

The initial score is $f_0 = 0$. Category is the only native categorical column in the first run: its name is explicitly passed in `categorical_feature`; the dictionary is built from A_G in UTF-8 order, with code 0 for unknown values and $1, 2, \ldots$ for known values. The price bucket is an ordinal number with a separate mask. The set, order, and types of all remaining numerical features are stored in the schema; the item identifier is not a feature. There are no monotonic, interaction, or forced-split constraints.

LightGBM builds histogram-based splits, so enumeration of all real-valued thresholds is not assumed. Its `min_data_in_leaf` is checked approximately through Hessians: it is a row parameter, not a query parameter or a strict guarantee of 100 actual rows. Actual leaf sizes are published by routing the training rows; a strict minimum would require a separate algorithm and a new experiment. With L1, leaf smoothing `path_smooth`, and the `max_delta_step` bound disabled, LightGBM's gain has scale $2(\text{Gain} + \gamma_T)$ before threshold subtraction, so `min_gain_to_split`$= 2\gamma_T$; both are zero in the first run. `cat_l2=0` excludes the additional categorical leaf penalty absent from (44). The chosen positive `cat_smooth=1` acts differently: when grouping categories, it restricts bin eligibility and smooths the sorting key $G_{\text{bin}}/(H_{\text{bin}} + 1)$. Thus the allowed case $G_{\text{bin}} = H_{\text{bin}} = 0$ does not produce $0/0$ in the sorting key; callback values and the leaf formula are unchanged. This parameter must not be disabled together with the leaf penalty. The library's small internal numerical constants are accommodated by test tolerance rather than bitwise equality.

The acceptance test compares all g, h entries with an independent FP64 calculation: before casting, the callback tolerance is $10^{-12} + 10^{-10}|v_{\text{ref}}|$; after FP32 conversion, it is $10^{-7} + 10^{-6}|v_{\text{ref}}|$. Cases cover several groups, two positive labels, tied scores, $K_R < |\mathcal{C}_q|$, zero-contribution groups, differing candidate counts, and logits $\pm 10^4$. A separate single-step test on a numerical feature with an unambiguous split checks leaves and doubled gain within 10^{-6}; size constraints are explicitly relaxed only in its fixture. Permuting input rows after canonicalization must preserve derivatives. Applying weights a second time must be detected by the test, and built-in LambdaRank is not used as the reference.

11.6 Negative examples and sparse purchases

The primary version uses all candidates without negative sampling. A separate variant with reweighted unpurchased candidates is preregistered for sensitivity analysis. For each $i \in \mathcal{C}_q^+$, instead of uniform weight $1/|\mathcal{C}_q^-|$, it uses mixture $\frac{1}{2}u_q(j) + \frac{1}{4}v_q(j) + \frac{1}{4}h_{qi}(j)$: u_q is uniform over $\mathcal{C}_q^-$, v_q is proportional to $1 + N_j^{A_G}$ (the number of A_G sessions containing the item), and h_{qi} is uniform over the first $\min(20, |\mathcal{C}_q^-|)$ unpurchased items under fixed category and description similarity to i (category match first, then cosine, then identifier; unavailable similarity is zero). The full pair weight in this variant is $w_q^{\text{sess}}\Delta_{qij}$ multiplied by this mixture and divided by $|\mathcal{C}_q^+|$. This is a different surrogate objective, not an exposure correction. The variant is trained on B and compared on C_{select} within the registered budget. For both variants, publish Hit@20 and ranking quality for head/tail, defined solely by A_G popularity (head is the first $\lceil 0.1|\mathcal{I}_G| \rceil$ items

in descending $N_j^{A_G}$ order, with ties resolved by ID; tail is all remaining items). Compare policy quality, not numerical values of different loss functions. If data reduction is unavoidable, pair (i, j) is included with known positive probability ξ_{qij}. For a fixed current order, an unbiased estimator of the local gradient sum uses weight

$$\widetilde{\omega}_{qij} = \frac{\mathbf{1}\{(i,j) \text{ is sampled}\}}{\xi_{qij}} \omega_{qij}. \tag{46}$$

ξ_{qij} refers to the pair; ranks and Z_q use the full group. Recomputing them on the sample does not recover the original LambdaMART. Small ξ_{qij} increase variance; clipping inverse weights introduces bias and requires separate designation.

Missing purchases are not reconstructed with invented labels. Scores from auxiliary next-view and add-to-cart tasks may be used as features when trained without temporal leakage.

A separate future study may predict a purchase within a session from available history and candidates. A single scalar prediction for a request does not itself compare items or different lists: adding a common value to scores preserves their order. Such a prediction is useful for diagnostics or auxiliary training but does not estimate the causal utility of a recommendation list and is not part of the first run.

11.7 Limits on score interpretation

Calibration is disabled; scores are not purchase probabilities. Period E remains reserved; it does not supplement training or model selection. Probabilistic methods of list construction require a separate formulation and suitable data.

12 Final recommendation list construction

12.1 Primary recommendation method

The baseline policy and neural extension sort $\mathcal{C}_q$ by $f(x_{qi})$ and return the first $K_q = \min(K, |\mathcal{C}_q|)$ distinct items. Calibration with $a_{\text{cal}} > 0$ does not alter this order. Tied scores use the fixed identifier order. The first run uses $K = 20$; all comparisons share the same upper limit K and length rule, not the actual K_q. Short and empty responses are included without artificial padding.

The first run and initial neural variant use $\lambda_D = 0$: no item-similarity penalty is applied; the calibrator is disabled. Changes to list diversity and probabilistic constraints are discussed in Appendix E.2.

12.2 Fallback response

If the ranker is unavailable, a separate fallback policy is used: the most recent eligible items and historical popularity in the order of $\mathcal{C}_q^{\text{fallback}}$ from (31); the first K items are returned. If no eligible items exist, an empty list is returned. All eligibility constraints remain in force on failure; the reason code is included in the request log. The primary model's calibrator is not applied to fallback scores. Abstaining from display and changing K define separate policies; their causal utility is tested under Section F, with a no-widget control if needed. Recommendation explanations are defined in Section 19.

13 Temporal protocol for training and evaluation

The protocol is shared by the core system and neural extension: inputs contain available history, and targets are subsequent behavior. The neural-training period remains reserved in the first run.

13.1 Periods and their roles

Let A_G, A_N, B, C, E, D be disjoint calendar intervals in the stated order, with $A = A_G \cup A_N$:

$$\sup A_G < \inf A_N, \quad \sup A_N < \inf B, \quad \sup B < \inf C, \quad \sup C < \inf E, \quad \sup E < \inf D. \tag{47}$$

Gaps and boundary losses are determined by the target horizon and time buffer, not by the observed target value. Period C is split in advance into two disjoint subintervals:

$$C = C_{\text{smoke}} \cup C_{\text{select}}, \qquad C_{\text{smoke}} \cap C_{\text{select}} = \varnothing, \qquad \sup C_{\text{smoke}} \leq \inf C_{\text{select}}. \tag{48}$$

After splitting C, the six major periods form seven parts, whose boundaries are fixed in the run manifest. Calendar admission (49) is applied separately to each subinterval: debugging targets do not cross the selection boundary. Throughout this document, parameter selection "on C" means only C_{select}, while implementation validation uses C_{smoke}.

Period	Purpose	Permitted use
A_G	Topology and reference data	Graph, paths, catalog $\mathcal{I}_G$, popularities, recorded-purchase counters, and historical feature scales.
A_N	Neural retriever training	Prefixes, recorded future purchases, auxiliary next views; training the graph encoder, content projections, intents, and search. Unused by the core system.
B	Ranking training	Candidates and features from the already fixed retriever; fitting trees to subsequent purchases.
C	Debugging and selection independent of debugging	C_{smoke}: implementation checks and fixes; C_{select}: one-time selection from a frozen list of variants (dimensions, epochs, loss weights, quotas, tree count).
E	Reserve	Unused in the current formulation.
D	Final evaluation	One independent evaluation of the frozen configuration and declared comparisons.

The start and end of each period, including both C subintervals, are fixed in the run manifest before training. In the first run, A_N and E are not merged into other periods or used for reference data.

13.2 Sequence of the first run and final evaluation

1. **Implementation validation.** Run the graph, content features, search, and LambdaMART with a 200-iteration budget and no parameter tuning. On C_{smoke}, check candidates, features, ranking, and metrics; measure response time and memory, and fix errors. C_{select}, E, D remain closed. These are local measurements; service load acceptance is not required. The full variant without

metadata does not block the first C_{smoke}; its mandatory deadlines are specified below. After debugging, freeze the code, features, metrics, configuration list, random initial states, and tie-breaking rule.

2. **Final-variant selection.** Then compare configurations on C_{select} under the frozen plan and computational budget. Results do not generate new variants. Changing the implementation, features, metrics, or list after inspecting C_{select} turns it into development data; another confirmatory selection requires a new, unseen period. Without such a period, the result remains exploratory. An example plan is given in Appendix I; it is not part of the first run.
3. **Independent evaluation.** Before opening D, separately freeze the entire final evaluation plan, check that it matches the selected model, and open D once. E remains closed.

A neural study is allowed after the baseline's data, formulas, features, and costs have been checked; proven purchase lift is not required. Each variant's training respects the period roles in the table above; C_{select} selects results and saved states but supplies no gradients.

The background catalog for training, normalization, and reconstruction is restricted to $\mathcal{I}_G$. An unknown item's features may participate in projection training only when that item is already observed in an A_N prefix. Its future interactions do not expand the candidate catalog. The caveat in Assumption 3.1 applies to all provided vectors. After A_N, weights and the vector index are fixed. Changing either requires rebuilding B features and retraining the ranker.

For semantic identifiers, first freeze the encoder trained on A_N, then train the additional module on that same period. Results on A_N are not an independent evaluation.

When retrieval changes, trees are retrained on B. New events in subsequent periods change only the current session state, not reference data.

Changing the method based on D results makes subsequent evaluations on D exploratory.

13.3 Calendar admission and complete reading

Horizon H and buffer g are defined in Section 5. Every block $J = [a_J, b_J)$ uses the same rule:

$$t_{0,s} \in J, \qquad t_{0,s} + H + g < b_J. \tag{49}$$

The rule is identical for sessions with and without a recorded purchase; the next-view label also cannot cross the boundary. After admission, IDs are sampled as described in Section 3. All released behavior and search rows of a selected session are read in full; session parts do not move between periods. Omitting released rows is a preparation error. Reading the complete export does not confirm telemetry completeness.

In A_G, an additional requirement is $t^{\text{obs}}_{\text{last},s} + g < b_{A_G}$: a session with a late tail is excluded in full, regardless of event type. History includes selected sessions both with and without a purchase. The catalog and statistics are not expanded with events from subsequent periods. The time constraint does not prove actual session closure.

Let N_J^{eligible} be the number of sessions with an eligible first request in J before calendar exclusions and ID sampling, and N_J^{obs} the retained count. Publish both counts, exclusion reasons, and the fraction of retained sessions:

$$\text{Retention}(J) = \frac{N_J^{\text{obs}}}{N_J^{\text{eligible}}}. \tag{50}$$

With an empty denominator, the ratio is undefined.

Invariant 13.1 (Temporal availability with a fixed model)**.** For requests in B, C, D, features and candidates depend only on the available prefix, the retrieval package published before the request, and the stipulated static information. Changing events after the request while holding the package fixed does not change these quantities. C_{smoke} is for debugging, and C_{select} for selection; independent final evaluation of the selected model uses only D. When training on A_N and B, future events are allowed in the target variable but not in the same example's input.

Trained weights are not considered available before A_N requests. Normalizations, negative-example distributions, and contrastive pairs also have a declared source period.

13.4 Model versions and final freeze

The core system uses one fixed package. In the deferred graph-update mode (Appendix C), packages are published at times $r_1 < r_2 < \cdots$; a request uses the latest:

$$r(q) = \max\{r_j : r_j < t_q\}. \tag{51}$$

Graph cutoff, encoder training, and publication times are retained. Features in B reproduce the intended version sequence; a new encoder or index cannot be assigned to an old request.

Until an update is ready, the previous package is used and its age is logged. In a confirmatory experiment, updates are prohibited or fully specified by its protocol.

Before opening D, freeze the data, intervals, request rule, $H, Q_{\max}, K, M$, failure handling, all compared configurations, primary metric, and interval construction. The selection budget covers the number of configurations and random initializations, training time, and peak memory. Equal candidate counts do not imply equal computational cost.

13.5 One primary final comparison

When studying an improvement to the core system, before opening D, select one configuration π_1 on C_{select} and fix control π_0: the graph, item features, search, and LambdaMART ranker. The primary historical result is the paired difference $\widehat{\Delta}_{\text{Hit}}(\pi_1, \pi_0)$ on D and its interval. Each variant has its own ranker trained on B; eligible data, request schedule, and candidate budget are shared. Selection on C_{select} does not continue after inspecting D.

Other variants and slices are descriptive. For multiple confirmatory claims, preregister m_{hyp} null hypotheses, valid p-value calculations, and an overall level α. For example, Holm's procedure [18] orders $p_{(1)} \le \cdots \le p_{(m_{\text{hyp}})}$ and sequentially rejects hypotheses while

$$p_{(j)} \le \frac{\alpha}{m_{\text{hyp}} - j + 1}; \tag{52}$$

at the first violation, the remaining hypotheses are not rejected. The procedure permits dependence between valid p-values but does not correct errors in their calculation, including ignoring dependence among a user's sessions. If only the core system is implemented, evaluate its coverage and costs on D without a superiority claim.

14 Evaluation on the historical log

The historical log can establish which subsequent purchases would appear in the computed lists. It cannot reveal whether purchases would change after actually displaying those lists. Thus the

primary historical metric measures coverage of observed purchases, while the causal business metric is defined separately in Section 6.

14.1 Session population and primary metric

Protocol `observed_purchases_v1` evaluates the sole recommendation list $q = q_s^{(1)}$ with $K = 20$ against the recorded target $Y_q = Y_s^{\text{obs}}$ from (8). Let $\mathcal{S}_J^{\text{obs}}$ be sessions in period J that pass calendar admission and sampling under Section 13.3. Their count is $N_J^{\text{obs}} = |\mathcal{S}_J^{\text{obs}}|$; boundaries and subsample sizes are fixed before analyzing future purchases, and shortfalls are not filled with purchasing sessions. For purchase-target training and evaluation, define

$$\mathcal{S}_J^+ = \{s \in \mathcal{S}_J^{\text{obs}} : Y_s^{\text{obs}} \neq \varnothing\}, \qquad N_J^+ = |\mathcal{S}_J^+|. \tag{53}$$

Selection into $\mathcal{S}_J^+$ is not passed to candidate retrieval, features, or the decision to recommend. Historical reference data use all retained sessions in A_G, not only purchasing sessions. Resource use and failures are measured also on sessions without a recorded target: the outcome is unknown at request time.

For a fixed policy π:

$$\text{Hit}_s(\pi) = \mathbf{1}\{L_{q_s^{(1)}}(\pi) \cap Y_s^{\text{obs}} \neq \varnothing\}, \tag{54}$$

$$\text{Hit@20}_J(\pi) = \frac{1}{N_J^+} \sum_{s \in \mathcal{S}_J^+} \text{Hit}_s(\pi), \qquad N_J^+ > 0. \tag{55}$$

This is the fraction of sessions with a recorded future purchase in which the first list contains at least one target item; the metric lies in $[0, 1]$. Purchase counts and repeated records do not increase a session's contribution. Changing list length or request count requires a new protocol.

The membership of $\mathcal{S}_J^+$ is identical across methods. For failures, late responses, or empty results, the specified fallback applies. If an eligible list is not delivered on time, $L_q = \varnothing$; the session remains in the denominator. A fallback hit counts, and fallback frequency is published separately. Out-of-catalog targets are not removed. At $N_J^+ = 0$, the metric is unavailable, not zero.

Telemetry completeness remains unknown even for positive sessions. Sessions with $Y_s^{\text{obs}} = \varnothing$ are not treated as confirmed non-purchasing sessions. Evaluation is conditional on a future purchase being recorded: it does not measure purchase frequency across all sessions, unobserved purchases, or the causal effect of recommendations. Neither the point estimate nor its interval eliminates event-recording or historical-policy bias.

Controls for the first evaluation are A_G popularity and recently viewed items filled with popularity; order and filters are specified in Section 12. Publish paired Hit@20 differences against them. Metadata sensitivity is checked under I.1. Selection on C_{select} maximizes (55); the budget and tie rule are fixed before C_{select}, and the selected primary pair before D. Intervals and sufficiency conditions are specified in Section 15; report contents are in I.2. Tuning on D is prohibited.

14.2 Exact upper bound for candidate retrieval

Below, for period J, every session sum is over $\mathcal{S}_J^+$, $N = N_J^+$, $q = q_s^{(1)}$. Define

$$U_s(\pi) = \mathbf{1}\{\mathcal{C}_q^{\text{served}}(\pi) \cap Y_q \neq \varnothing\}, \tag{56}$$

$$\text{Coverage}_{\text{cand},J}(\pi) = \frac{1}{N} \sum_s U_s(\pi). \tag{57}$$

Set $\mathcal{C}_q^{\text{served}}(\pi)$ is defined in (31). Branch switching includes fallback; a failure of an already selected response retains its candidates. Coverage of a completed $\mathcal{C}_q^{\text{main}}$ is reported separately with the fraction of unmeasured sets; it does not replace the operational policy's bound. The session population is not restricted to successful responses.

Proposition 14.1 (Ranking limit under fixed retrieval)**.** If $L_q(\pi) \subseteq \mathcal{C}_q^{\text{served}}(\pi)$, then

$$\text{Hit@20}_J(\pi) \le \text{Coverage}_{\text{cand},J}(\pi). \tag{58}$$

With fixed branches and candidates, $K \ge 1$, unrestricted choice of a subset of size at most K, and no delivery failures, the bound is attainable by a ranker that knows future purchases. A fixed fallback order or additional list constraints may make it unattainable; the inequality itself still holds.

Proof. List inclusion implies $\text{Hit}_s \le U_s$. At $U_s = 1$, an ideal ranker includes a target item in the list; at $U_s = 0$, a hit is impossible. □

14.3 Mandatory decomposition of coverage losses

The presence of an eligible target is

$$V_s = \mathbf{1}\{\mathcal{I}_q \cap Y_q \ne \varnothing\}. \tag{59}$$

For sessions in $\mathcal{S}_J^+$, $1 \ge V_s \ge U_s(\pi) \ge \text{Hit}_s(\pi)$, hence

$$1 - \text{Hit}_s(\pi) = (1 - V_s) + (V_s - U_s(\pi)) + (U_s(\pi) - \text{Hit}_s(\pi)). \tag{60}$$

The three disjoint cases are: no eligible targets; retrieval missed eligible targets; retrieved targets were not delivered in the list. The mean terms give an exact decomposition of $1 - \text{Hit@20}_J$. Starting on C_{smoke}, publish N_J^{obs}, N_J^+, $\sum_s V_s$, $\sum_s U_s$, $\sum_s \text{Hit}_s$, and the three losses.

Conditional candidate coverage given an eligible target is

$$\text{Coverage}_{\text{cand,avail}} = \frac{\sum_s U_s}{\sum_s V_s}, \qquad \sum_s V_s > 0. \tag{61}$$

A zero denominator means the metric is unavailable. Report $\sum_s V_s / N$ alongside it to separate catalog coverage from retrieval quality.

The purchase table distinguishes absence from $\mathcal{I}_G$ from ineligibility at request time. Its rows are not summed as session losses: one session may contain purchases of different types. Source failures, missing features, and deadline overruns form separate slices, not a fourth loss term. A match $i \in L_q \cap Y_q$ always means $\text{Hit}_s = 1$.

14.4 Source contribution and overlap

The first report uses already computed source results. Let $\mathcal{R}_q^{(a)}$ denote the available portion of source a's ordered eligible-item list before quota m_a; the depth and truncation of this list are recorded. If a source returns only its post-quota response, pre-quota coverage is marked unmeasured. Future targets do not participate in search; diagnostic lists do not replace actual $\mathcal{C}_q$. For a nonempty union, define

$$\begin{aligned} O_q^{ab} &= \frac{|\mathcal{R}_q^{(a)} \cap \mathcal{R}_q^{(b)}|}{|\mathcal{R}_q^{(a)} \cup \mathcal{R}_q^{(b)}|}, \\ \mathcal{R}_q^{(a),\text{uniq}} &= \mathcal{R}_q^{(a)} \setminus \bigcup_{b \ne a} \mathcal{R}_q^{(b)}. \end{aligned} \tag{62}$$

For an empty union, overlap is undefined. For each source, publish the empty-response fraction, list sizes before and after quota, and future-target coverage before quota, after quota, and in the final union. Unique items and overlaps are shown separately. Coverage of any family of measured lists $\mathcal{D} = (\mathcal{D}_q)_q$ uses the same session units:

$$\mathsf{H}(\mathcal{D}) = \frac{1}{N} \sum_s \mathbf{1}\{\mathcal{D}_{q_s^{(1)}} \cap Y_s^{\text{obs}} \neq \varnothing\}. \tag{63}$$

Incomplete measurements are not replaced by empty lists; full pre-quota coverage is then undefined. Overlap is averaged over defined values with equal session weights; the missing fraction is reported separately. The sum of source coverages is not the coverage of their union. Additional deeper retrieval is optional; its cost is separated from the working configuration's latency.

To measure source contribution with candidates held fixed, let $\mathcal{O}_q(i)$ denote their source set, including fallback filling. Remove only items obtained exclusively from source a:

$$\begin{aligned} \mathcal{C}_q^{-a} &= \{i \in \mathcal{C}_q : \mathcal{O}_q(i) \setminus \{a\} \neq \varnothing\}, \\ \Delta_a^{\text{fixed}} &= \frac{1}{N} \sum_s (U_s - U_s^{-a}) \geq 0. \end{aligned} \tag{64}$$

Here U_s^{-a} is computed by (56) using $\mathcal{C}_q^{-a}$; freed slots are not filled. A unique item for a request does not necessarily make a unique session contribution: another source may have found another target item in the same session. This measures candidate-coverage loss from removing a source from completed results, not the performance of a reconfigured system. The reconfigured system is tested in a separate predefined experiment: redistribute quotas, perform fallback filling, retrain the ranker on B, and compare policies under Appendix I. That difference may have either sign. Retraining for every source is not part of the mandatory first run.

14.5 Choosing the next change

A research hypothesis is formulated on C_{smoke} after checking the implementation and loss table; the list of variants to compare is fixed before opening C_{select}. Large $\sum_s (1 - V_s)$ calls for analysis of the catalog and target eligibility; large $\sum_s (V_s - U_s)$, of sources and quotas; large $\sum_s (U_s - \text{Hit}_s)$, of ranking and timely list return. Technical causes are checked first. Low candidate coverage alone does not prove weak retrieval: target availability and (61) must be considered.

The candidate bound uses knowledge of the future. The gap to the delivered list does not guarantee an attainable ranker improvement and does not estimate lost money or conversion. To compare ranking alone, hold $\mathcal{C}_q$ fixed: a smaller gap caused by worse retrieval is not an improvement. A new hypothesis, budget, and success criterion are fixed before testing it; after inspecting D, a new final evaluation period is needed.

14.6 Auxiliary metrics and slices

For a request with $|Y_q| > 0$, diagnostic recall is

$$R_q@K = \frac{|L_q \cap Y_q|}{|Y_q|}. \tag{65}$$

The denominator of the mean over requests with purchases is published separately. To diagnose ordering, use

$$\mathrm{DCG}_q@K = \sum_{j=1}^{|L_q|} \frac{\mathbf{1}\{L_{q,j} \in Y_q\}}{\log_2(j+1)}, \tag{66}$$

$$\mathrm{NDCG}_q@K = \frac{\mathrm{DCG}_q@K}{\sum_{j=1}^{\min(K,|Y_q|)} 1/\log_2(j+1)}, \quad |Y_q| > 0. \tag{67}$$

DCG weights are not measured probabilities of viewing positions. Mean Recall and NDCG use the same $\mathcal{S}_J^+$.

Report separately coverage of purchased items inside the observed cart and outside it. Cart state is determined only from the prefix; it may be incomplete. Group coverages are not added: one session may contain purchases from both groups.

Mandatory slices are specified before inspecting final results: available history length, presence of earlier search, cart presence, historical target-item popularity, content-vector availability, and whether the snapshot knows the item. Popularity-group boundaries use A_G, not frequencies from period D.

Purchases of unknown or ineligible items remain in targets and denominators, with their fraction reported separately. First observation in the log does not prove an item is new: its catalog introduction date is unknown.

15 Uncertainty in historical comparisons

First lists are evaluated on $\mathcal{S}_D^+$: sessions with a recorded future purchase under Section 14.1. Below, $N = N_D^+ = |\mathcal{S}_D^+|$, and sums over s use only this population. Intervals are conditional on a purchase entering the log; they do not remove bias from incomplete telemetry.

15.1 Paired session comparison

Policies π_1 and π_0 are evaluated on the same sessions. For $N > 0$, the difference in first-list hit rates is

$$d_s = \mathrm{Hit}_s(\pi_1) - \mathrm{Hit}_s(\pi_0) \in \{-1, 0, 1\}, \qquad \widehat{\Delta}_{\mathrm{Hit}} = \frac{1}{N}\sum_s d_s. \tag{68}$$

During resampling, outcomes from both policies are carried together, preserving their disagreement information for each session.

For a single fixed policy, estimate coverage itself:

$$x_s = \mathrm{Hit}_s(\pi), \qquad \widehat{H} = \mathrm{Hit@20}_D(\pi) = \frac{1}{N}\sum_s x_s. \tag{69}$$

Its interval uses the same calendar method, replacing d_s with x_s in (74)–(75). The half-width threshold $h_{\text{single}} > 0$ is set separately from the paired-comparison threshold h_{pair}; both are expressed as proportions. For $N = 0$, estimates are unavailable and the result is `insufficient`.

For $N > 1$ and approximately independent sessions, the secondary standard error, conditional on that assumption, is

$$\widehat{\mathrm{se}}(\widehat{\Delta}_{\mathrm{Hit}}) = \sqrt{\frac{1}{N(N-1)} \sum_{s} (d_s - \bar{d})^2}, \qquad \bar{d} = N^{-1} \sum_{s} d_s. \tag{70}$$

The primary Coveo interval uses a temporal bootstrap of consecutive blocks under Section 15.3, subject to Section 15.4. Independent session resampling [19] is only a sensitivity check and does not replace an unusable primary interval.

For replicate b, draw N session indices independently and uniformly from $\{1, \ldots, N\}$ and compute

$$\widehat{\Delta}^{*(b)}_{\mathrm{Hit}} = \frac{1}{N} \sum_{j=1}^{N} d_{I_j^{(b)}}, \qquad b = 1, \ldots, B_{\mathrm{boot}}. \tag{71}$$

Paired first-list outcomes are carried together. Replicate count $B_{\mathrm{boot}} = 2000$ and the generator's initial state are fixed in advance. Quantiles are defined in Section 15.3.

15.2 Planning precision for period D

Before opening D, set the paired-difference interval half-width $h_{\mathrm{pair}} > 0$ in proportions and the planned variance $v_{\mathrm{plan}} \approx \mathrm{Var}(d_s)$. Justify it using an authorized pilot on C_{smoke} after training, without D or future leakage. The threshold, variance, assumptions, temporal-dependence multiplier, and pilot provenance enter the immutable pre-D evaluation plan; their absence does not block `preflight` or the first training run. Under independent sessions,

$$N_D^{+,\mathrm{iid}} = \left\lceil \frac{z_{1-\alpha/2}^2 \, v_{\mathrm{plan}}}{h_{\mathrm{pair}}^2} \right\rceil, \tag{72}$$

where z_u is the level-u standard normal quantile. This follows from the half-width approximation $z_{1-\alpha/2}\sqrt{v_{\mathrm{plan}}/N_D^+}$; power for a causal-effect experiment is calculated separately.

For a single-policy estimate, use the same scheme with substitutions $d_s \mapsto \mathrm{Hit}_s(\pi)$, $h_{\mathrm{pair}} \mapsto h_{\mathrm{single}}$, and $v_{\mathrm{plan}} \approx \mathrm{Var}(\mathrm{Hit}_s(\pi))$. The primary estimate type is fixed in `inference.primary_estimate_type`. A policy pair and paired-difference threshold are mandatory only for `paired`; `single_policy` requires one policy, h_{single}, and its planned outcome variance.

Variance depends on policy disagreements, not only purchase count. Zero variance in a small pilot does not justify $v_{\mathrm{plan}} = 0$. With insufficient data, use a safety margin or the rough bound $\mathrm{Var}(d_s) \le 1$; the calculation's provenance and assumptions are included in the protocol.

For temporal dependence, set a multiplier $\kappa_{\mathrm{dep}} \ge 1$ and plan

$$N_D^{+,\mathrm{plan}} = \left\lceil \kappa_{\mathrm{dep}} N_D^{+,\mathrm{iid}} \right\rceil. \tag{73}$$

On the pilot, divide the variance under time-block resampling by the variance under independent sessions, bounding the result below by one. With a zero denominator, the ratio is undefined. Block duration and the safety margin are fixed in advance. The multiplier does not guarantee precision under a regime change or few blocks; it does not correct the final interval for independent sessions. The dependence conditions in Section 15.3 remain in force for the primary interval.

Period boundaries account for the forecast number of sessions with a recorded target, calendar cycles, and exclusion of the last $H + g$ from the start-time window. Session selection does not

depend on target presence. After opening D, boundaries and enrollment rules are not changed to obtain a convenient result. Publish the actual interval, its half-width, and whether h_{pair} or h_{single} is met. Additional enrollment requires a new protocol. For a fixed public log, `insufficient` is an acceptable outcome: publish the available estimate and the reason for inadequate precision without shifting periods or requiring unavailable additional data. The chosen threshold h_{pair} or h_{single}, parameters $v_{\text{plan}}, \kappa_{\text{dep}}$, and their provenance and assumptions become mandatory run-manifest fields at `pre_D`; actual period boundaries are fixed before training. An unfilled field is not replaced by an invented measurement.

15.3 Temporal dependence and interpretation limits

Shared promotions, assortment, and demand create temporal dependence among sessions. Before opening D, fix a dense calendar grid with step $\delta_{\text{grid}} > 0$ and an origin anchored to the D boundary. For $D = [a_D, b_D)$, its J cells cover only the eligible start-time window $[a_D, b_D - H - g)$, excluding the final calendar buffer $H + g$. This buffer does not confirm observation completeness. An integer number of cells is required: boundaries and step are aligned before opening D; unaligned boundaries invalidate the run manifest. Each session is assigned in full to a cell by $t_{0,s}$. All cells within the window are retained, including those without sessions with a recorded target; no artificial tail is added. Let $\mathcal{S}_j^+$ contain sessions from $\mathcal{S}_D^+$ in cell j. Record

$$n_j = |\mathcal{S}_j^+|, \qquad D_j = \sum_{s \in \mathcal{S}_j^+} d_s. \tag{74}$$

The moving-block bootstrap for stationary dependent observations [20] selects length $\ell_{\text{boot}} = m\delta_{\text{grid}}$, $m \in \{1, \dots, J\}$, using an authorized pilot on C_{smoke} and domain constraints without using D; block length is fixed before opening D. For each replicate, independently sample $\lceil J/m \rceil$ starting points uniformly from $\{1, \dots, J - m + 1\}$; take m consecutive cells from each starting point, concatenate the resulting sequences, and retain the first J indices $J_1^*, \dots, J_J^*$. Thus adjacency is preserved within each selected block, and the two policies are always carried together. Use the ratio of sums

$$\widehat{\Delta}_{\text{Hit}}^* = \frac{\sum_{\ell=1}^{J} D_{J_\ell^*}}{\sum_{\ell=1}^{J} n_{J_\ell^*}}. \tag{75}$$

Both numerator and denominator are recomputed in every replicate. An unweighted mean of daily metrics with differing session counts has a different meaning. The primary interval uses the empirical quantiles $\alpha/2$, $1 - \alpha/2$ from B_{boot} replicates of (75). For every bootstrap interval in this specification, the level-p quantile is the element at index $\lceil pB_{\text{boot}} \rceil$ in the sorted sample. A zero denominator makes a replicate undefined: such cases are logged, and interval computation receives `insufficient`, rather than silently discarding them or replacing them with zero.

Assumption 15.1 (Validity of the temporal bootstrap)**.** Each estimated functional has its own vector of block sums. For a paired difference, it is (D_j, n_j); for a single coverage estimate, $(\sum_{s \in \mathcal{S}_j^+} \text{Hit}_s(\pi), n_j)$. The corresponding joint process is sufficiently close to stationary over the eligible start-time window, has finite second moments, positive mean denominators, and dependence that weakens with lag. Block length covers the substantial dependence horizon; distant blocks are not linked by a dominant common shock. Asymptotic justification requires block length to grow while remaining small relative to observation length. A joint central limit theorem for component means and consistency of the chosen bootstrap for the same limit are required; qualitative stationarity alone does not prove these properties. The estimated functional must be regular in the limiting means and have nonzero limiting variation.

Coveo has no stable cross-session ID, so calendar blocks do not prove that repeat-user dependence has been removed. If another log provides such an ID and user dependence dominates, user-cluster analysis is chosen before opening the test; shared calendar shocks require additional treatment of temporal dependence.

Before D, the run manifest fixes the grid, block lengths, two additional lengths for sensitivity, generator, and replicate count. The necessary technical threshold is $J_{\text{eff}} = \lfloor J/m \rfloor \geq 20$ for the primary calculation; this is the number of available full blocks, not an estimate of independent observations. For the fixed partition $\{1, \ldots, m\}, \{m+1, \ldots, 2m\}, \ldots$ into J_{eff} full blocks, the number of blocks with $\sum n_j > 0$ must be at least 20; overlapping bootstrap blocks are not counted here as new observations. Empty intervals create no information. For each functional and slice, count only blocks containing its observations. Failure of the threshold or Assumption 15.1 gives `insufficient` for the corresponding interval. Validity of the paired-difference process does not establish validity of a single policy's coverage process, or vice versa.

Publish the time series D_j/n_j for $n_j > 0$, session counts n_j, boundaries of promotions and assortment changes, and the primary and two predefined sensitivity intervals. Empty cells are marked as having no metric. A trend or substantial dependence of the conclusion on block length limits interpretation; choosing the most convenient interval is prohibited. Data from D are not used to tune length, remove problematic days, or replace the primary method. Too few blocks cannot be remedied simply by increasing bootstrap replicates. For interval bounds, report Monte Carlo error using a method chosen in advance; replicate count does not determine the statistical sufficiency of the original stream.

Both methods evaluate already trained models on this stream. Uncertainty from retraining and seasonal changes requires several temporal evaluations with repeated training; overlapping evaluations are not independent.

15.4 Information sufficiency and degenerate intervals

The number of nonempty blocks does not protect against rare policy disagreements. For the coverage being tested, set $c_s = d_s$ for a pair comparison and $c_s = \text{Hit}_s(\pi)$ for a single-policy evaluation. Let $\mathcal{B}_b$ contain sessions from $\mathcal{S}_D^+$ belonging to a full nonoverlapping block of m cells under Section 15.3. For each category a ($-1, 0, 1$ for a pair; $0, 1$ for one policy), compute

$$N_a = \sum_s \mathbf{1}\{c_s = a\}, \qquad J_a = \sum_{b=1}^{J_{\text{eff}}} \mathbf{1}\{\exists s \in \mathcal{B}_b : c_s = a\}, \qquad |\{a : N_a \geq 100,\ J_a \geq 20\}| \geq 2. \tag{76}$$

These are initial protective thresholds: a single policy requires at least 100 hits and 100 misses, with each outcome present in at least 20 blocks. A pair requires two sufficiently represented categories; therefore, at least 100 disagreements and 20 blocks containing them. Rare categories are not merged after inspecting D to pass the threshold.

Failure of (76), zero empirical variation, an undefined replicate, or coincident primary bootstrap interval bounds gives `insufficient`. The point estimate and all counts are published, but zero width does not confirm precision or permit $S_{\text{offline}} =$ passed. Policy identity proven for every eligible input is reported separately as an algebraic result with proof and artifact hashes, not as measured precision.

For a usable interval $[l, u]$, additionally check $(u - l)/2 \leq h_{\text{pair}}$ or h_{single}. Observation-count, block-count, and half-width thresholds are fixed before D. They exclude clearly uninformative estimates but do not guarantee coverage, power, or absence of long-range dependence. An interval

for rare outcomes with different assumptions requires a separate advance plan; it is not substituted for the primary interval after inspecting the result.

16 Feature storage and updates

16.1 Training–serving consistency

Feature j is defined by a contract shared by training and serving:

$$\mathsf{F}_j = (\text{entity}, \phi_j, \text{sources}, \text{available_at}, \text{version}, \text{missing_rule}). \tag{77}$$

The feature version belongs to the pinned package. The package is chosen from compatible versions published strictly before t_q, even if $a_q > t_q$; within this set, the latest version in the publication order recorded in the run manifest is selected. A version published between t_q and a_q is prohibited. If no suitable package exists, `version_unavailable` is recorded; the new package is not used even for fallback.

For a replayed request, verify

$$\max_j \frac{\left| x_{qj}^{\text{replay}} - x_{qj}^{\text{serve}} \right|}{\varepsilon_j + \left| x_{qj}^{\text{replay}} \right|} \le \delta_{\text{feat}}. \tag{78}$$

Categories, masks, candidates, snapshots, and tie-breaking order must match exactly; the relative tolerance applies to floating-point numbers. Parameters, numeric precision, and the index are identical; training transformations, dropout, and encoder noise are disabled. Tolerances for FP16 and BF16 are set separately from those for double precision. The check includes the original ordered lists from each source, availability masks, quotas, deduplication, and the selected branch; agreement of the final feature vector alone is insufficient.

16.2 Numerical stability

Equivalent mathematical expressions are implemented using numerically stable primitives. For finite x, use

$$\text{softplus}(x) = \max(x, 0) + \log 1p(e^{-|x|}), \tag{79}$$

$$\text{sigmoid}(x) = \begin{cases} 1/(1+e^{-x}), & x \ge 0, \\ e^{x}/(1+e^{x}), & x < 0. \end{cases} \tag{80}$$

The pairwise logistic loss is computed as $\text{softplus}(-\beta_R d)$; binary cross-entropy is computed from logits, without evaluating $\log(\text{sigmoid}(x))$ or $\log(1 - \text{sigmoid}(x))$. Sums of exponentials use

$$\text{LSE}(z) = z_* + \log \sum_j e^{z_j - z_*}, \qquad z_* = \max_j z_j, \tag{81}$$

or `logaddexp` for blockwise accumulation. This rule applies to attention, softmax, intent weights, contrastive loss, SID, and catalog normalization. Masks are applied before taking the maximum. An empty admissible domain is handled by a separate branch: the source is unavailable, and $-\infty - (-\infty)$ is not evaluated. The value $-\infty$ is allowed only as an explicitly defined marker for an excluded logit; it is not an item score. Entropy and KL use stored log probabilities as specified

in 22.3: the logarithm of a value rounded to zero is not computed; derivatives are checked as well. Exact binomial bounds are computed using beta-distribution quantiles with analytical boundary cases for zero successes and all successes; tails use log-CDF, without summing factorials.

The first run uses FP64 for features, scores, metrics, and statistical intervals. Exception: derivatives of the specified function computed in FP64 are passed to LightGBM through contiguous FP32 arrays under the contract in 11.5; their tolerances are checked separately. Identifiers and counters are not stored in FP16/BF16; counters use 64-bit integers with overflow checks. In neural variants, FP16/BF16 are allowed for matrix operations only with at least FP32 accumulation; norms, softmax/LSE, losses, and derivatives are accumulated in at least FP32, and final metrics and statistics in FP64. The run manifest records the numeric types at each stage, libraries, rounding modes, and tolerances. Initial tolerances for the FP64 check in (78): $\varepsilon_j = 10^{-8}$ and $\delta_{\text{feat}} = 10^{-6}$. Any other numeric type requires a separate check against FP64 before model selection.

The norm of a finite vector is computed with scaling: when $v_* = \max_j |v_j| > 0$, first compute $w = v/v_*$ and $r = \sqrt{\sum_j w_j^2}$, and the unit direction is w/r. The check $\|v\| > \varepsilon$ is evaluated as $v_* > \varepsilon/r$, avoiding the potentially overflowing product $v_* r$; when $v_* = 0$, the zero branch is selected immediately. Finite individual coordinates do not justify directly summing their squares.

An external modality and a computational error are treated differently. An absent external vector receives mask 0 and reason `vector_absent`; an incorrect dimension or nonnumeric/nonfinite coordinates receive mask 0 and `vector_invalid`; a norm no greater than ε receives mask 0 and `vector_zero`. All three cases use a zero vector of the expected dimension. This transformation is identical in preparation, training, and serving; other usable modalities are retained. An error in a required ID, timestamp, or envelope structure produces `invalid_input`, not an absent modality. NaN/Inf values that unexpectedly arise after this preparation in features, losses, derivatives, or scores stop training; during serving, they invalidate the computation and activate the declared fallback. Replacing an internal error with a zero mask is prohibited. Tests cover logits $\pm 100, \pm 10^4$, empty masks, zero norms, and the largest permitted groups. An FP64 reference, finite permitted results, and unchanged masks are required; FP64 absolute/relative tolerances are $10^{-10}/10^{-8}$, and FP32 tolerances are $10^{-6}/10^{-5}$. Reordering nearly equal scores is not hidden by a numerical tolerance: the common tie-breaking order applies.

16.3 Session state updates

State $\mathsf{U}_s(t)$ contains a bounded recent history, action counters, the last event time, observed cart state, and search context. For a newly available block $E_{s,t}$,

$$\mathsf{U}_s(t) = \mathcal{T}\big(\mathsf{U}_s(t^-), E_{s,t}; \nu_{\text{features}}\big). \tag{82}$$

$\mathcal{T}$ is deterministic and depends only on the available history. Redelivery is handled under the contract in Section 16.4; matching Coveo rows do not replace a transport event identifier.

For a decaying aggregate of a single action type, a convenient recurrence is

$$x_{si}(t) = e^{-(t-t')/\tau_x} x_{si}(t') + \sum_{e \in E_{s,t}} \mathbf{1}\{i(e) = i,\ \kappa(e) = \kappa_x\}. \tag{83}$$

Under sequential processing, this is a sum of decaying past events. An exact sliding window requires an event queue.

Storage retains the last L_{hist} general blocks for replay, a separate queue of the last L_H `detail` events for the graph, the latest admissible search context, and the necessary aggregates over the

entire visible prefix. The general tail does not replace the view queue: frequent searches must not evict the L_H events that are still needed. Memory for all queues, aggregates, and delivery markers is accounted for separately. Removing a block does not reset aggregates over the entire visible session. Operational state is removed after inactivity once the required outcome records have been written; state and outcome retention periods are specified separately.

16.4 State storage contract

Process memory is sufficient for replay; serving requires a key-value store with atomic record updates. No product is prescribed; retention periods and delivery limits are recorded in the test description.

1. **Record key and contents.** The key (ν_{source}, s) includes the source identifier namespace. The record contains U_s, revision number v_s, version ν_{features}, the last arrival time, a completeness flag, and markers for accepted events. Purchases and the request log are stored independently of the operational record.
2. **Concurrent access.** Verifying that the read value v_s is unchanged and writing the new state, $v_s + 1$, and event markers are performed as a single atomic operation. If the revision differs, the record is read again and the transition is recomputed within the overall response deadline. A request uses one confirmed state version and one package V, without mixing fields. The snapshot includes only events available to the request (Section 4); if the record contains later events, the snapshot is reconstructed from the log. The transformation version matches the package; changing it requires a declared state transformation or reconstruction.
3. **Redelivery.** The block receipt, ID check, and counter/completeness updates are atomic. Receipts and event markers are retained for at least Δ_{retry}; a verified duplicate does not change the logical state, including completeness. After this period, IDs are checked against the log or the response is `retry_unverifiable`, without treating the delivery as a new event. The order of transport checks is specified in 16.8.
4. **Retention and lateness.** The lifetime after the last arrival, T_{state}, the lateness limit δ_{late}, and outcome retention are specified separately. The tail is bounded by block count and time. After a record is removed, a late event requires reconstruction from the log or an incompleteness marker.
5. **Failure and recovery.** An unavailable, incompatible, or incomplete record activates the prescribed fallback with a reason code; unread history is not treated as empty. If an admissible fallback or a required write is not ready by the deadline, the result is a technical failure. Snapshot reconstruction, version checking, and retrying an atomic write are included in the state budget and are not performed after it expires. Initial limits: one atomic-update retry, at most 1000 replayed events, and at most 1 MiB of log data per request. Exceeding any limit produces `state_replay_limit`; background reconstruction does not delay the response.
6. **Cached computation results.** The reuse key depends on the visible prefix, request time, and transformation package:

   $$(\nu_{\mathcal{H}_q}, t_q, \mathsf{V}).$$

   Identifier $\nu_{\mathcal{H}_q}$ identifies the session, visible events, and search. History and intent weights also depend on time and the model; a session identifier alone is insufficient for reuse.

16.5 Requirements for request replay

The run manifest specifies one of two modes. `ideal_deterministic` uses $a = t$, frozen data, and normally operating sources; this mode does not reproduce observed operational failures. In this

mode, evaluations on $C_{\text{smoke}}, C_{\text{select}}, E, D$ do not require a service implementation or the `post_load` stage. The report states the mode and its assumptions; local time and memory measurements do not establish operational guarantees. `execution_trace` additionally stores an evaluation trace: t_q, a_q, the package, a reference to the immutable visible prefix/state snapshot, the selected branch and delivery status, source availability masks and unavailability reasons, truncation decisions, the seed, and the fault-injection schedule. The reference includes a hash, but the hash itself does not replace retained inputs.

The trace records the actual number of steps/nodes when search stops early; if this does not uniquely reproduce the list, that source's list is stored. Trace completeness and input availability are checked before computing $\mathcal{C}_q^{\text{served}}$ and U_s. A healthy rerun does not replace a lost trace: the corresponding candidate set is marked unmeasured. Trace access and retention follow Section 16.8; expiration of input retention ends the ability to replay exactly, but does not invalidate retained aggregate estimates.

16.6 Arrival time and late events

Input time is an integer number of UTC milliseconds since the Unix epoch within the signed 64-bit range. Before production operation, the run manifest specifies the allowed calendar range $[T^{\text{in}}_{\min}, T^{\text{in}}_{\max})$; it must cover the intake period and permitted lateness. An incorrect type, unit, or range produces `invalid_input`; the value is neither rounded nor corrected based on an assumed unit. A verified duplicate of an accepted block is handled before rechecking time bounds, and its original timestamps are not changed. For a new block, before modifying the history, verify

$$-\delta_{\text{late}} \le t(e) - a_{\text{block}} \le \delta_{\text{future}}. \tag{84}$$

a_{block} is the server time of the atomic decision. In the initial profile, $\delta_{\text{future}} = 0$: there is no waiting buffer. Violating the upper bound causes quarantine with reason `timestamp_future`, and violating the lower bound gives `timestamp_late`. The block does not change history, $t_{\text{last},s}$, or closed time groups. A new trusted block marks incompleteness as specified in 16.8; fallback remains active until recovery. An error from an untrusted sender does not change another party's session. Positive clock-ahead tolerance is prohibited in this profile.

An accepted late event changes only responses after its arrival. State is reconstructed from a checkpoint preceding the tail and tail events in timestamp order. The checkpoint contains all aggregates and queues; without it, the complete log is required within budget; otherwise, state is incomplete and fallback applies. The independent purchase log continues recording outcomes; past responses are unchanged. Delays, duplicates, and out-of-order delivery are tested separately from the main historical mode $a = t$.

16.7 Partial data availability

A missing feature, a disabled source, and a read failure have different codes. An item modality that is absent uses its standard mask if this mode was included in training. A storage failure is not encoded as an absent modality: a source is disabled only under a preregistered degradation rule; an unregistered combination of unavailable sources activates fallback. A partial source list is allowed only with an explicit completeness flag and the prescribed quota; an arbitrary portion of a list at deadline expiry is not considered a complete result.

The eligibility catalog is mandatory: an item is served only after its eligibility is confirmed in the request's version. An older copy is allowed only within the validity period pinned in the run manifest. Unknown eligibility excludes an item from both the primary and fallback branches; inability

to verify even the fallback produces a technical failure. Unread history is not interpreted as a new session. For each degradation mask, report frequency, response length, quality, and the number of technical failures.

16.8 Trust boundaries, input limits, and trace retention

The threat model includes forged/bulk events, Sybil sessions, SKU popularity inflation, expensive history reconstruction, and artifact substitution. Source authentication confirms origin, not the truthfulness of behavior. Required schema and size checks precede computation; unusable external vectors are handled as specified in 16.2.

Sealed transport block. The initial production-service profile is `sealed_block_v1`. Each session namespace has one registered aggregator responsible for combining behavior and search before sending a block. The ownership registry is fixed before launch; authorization is checked against the authenticated sender, not the submitted namespace name. One immutable envelope contains the source namespace, session ID, event time, a unique `block_id`, event count, stable event IDs, the events themselves, the payload's SHA-256, and `closed=true`. All events have the same s, t; the event count equals the lengths of the payload and ID list, and `event_ids[k]=event s[k].event_id` for every k. IDs within the envelope are unique. The service computes the hash from the exact bytes of the immutable payload. The receipt binds the namespace, session, time, block ID, event count, and this hash; a matching ID alone is insufficient. A different serialization is not an identical transport retry. The service accepts only a complete envelope and does not combine fragments from different requests. An incomplete envelope adds no events; its rejection for an established trusted session is accounted for under the completeness rules below. Receiving the complete envelope and queueing are included in the overall deadline from the first byte under Section 17.4. A separate 10 ms limit does not apply to the demonstration service. Aggregator assembly is reflected in $a(e) - t(e)$ and the separately measured event-to-response latency. The service records a_{block} at the atomic decision on a verified block, including quarantine decisions. For accepted events, $a(e) = a_{\text{block}}$; rejected events are not included in available history. Sender time does not replace these timestamps. When ingestion and a request are combined, the cutoff a_q is recorded after block acceptance; the response deadline is still measured from the first byte. Recovering retries does not change the original $a(e)$.

Limits: 1 MiB per envelope, 256 UTF-8 bytes per SKU, 1 to 100 raw events per envelope, 100 logical events per block, 1000 logical events per session, and 1000 search-list entries. Logical counters are determined after deduplicating event IDs and applying the selected canonicalization. Before accepting a complete block, the sum of the previous counter and its new logical events is checked; crossing the limit rejects the entire new block, without partial truncation. Unused search lists are removed after input validation.

Already closed $(\nu_{\text{source}}, s, t)$ groups are neither extended nor reopened. The same `block_id` with different contents, or new IDs for a closed timestamp, produces `closed_block_conflict`, not admissible lateness. The new unknown portion is quarantined; accepted history is preserved. Ordinary late delivery is allowed only for a block not yet accepted and follows (84). The historical-data adapter groups the entire known time block before creating an envelope; the production service does not assume knowledge of future packets.

The rate limit is a token bucket of 10 raw events/s with burst 100; the cost of the full attempt, including retries, is checked atomically. Cost is determined by the actual raw event count, not the header; an empty, rejected, or unparseable attempt costs at least 1. Connection and sender limits

apply before the session is established. The following distinct transitions then apply:

1. A verified complete duplicate of an accepted block changes neither counters, completeness, nor responses. If tokens are insufficient, it may be rejected as `duplicate_rate_rejected`; this is not a history gap. The rule also applies after δ_{late}, while the receipt remains verifiable.
2. An envelope containing only known events does not create a new block. A partial duplicate with new IDs is not handled event by event: for an already closed timestamp it is a conflict; for a new block, it violates the mapping between an event ID and its original fields. New unverified events are not considered accepted; existing records are unchanged.
3. A rejected new block from a trusted aggregator leaves a gap and incompleteness marker; subsequent requests use fallback until recovery. An unverifiable retry receives `retry_unve rifiable`, without counting the events again; reconstruction or log verification determines completeness.

Receipt verification is bounded by one indexed read and the envelope size. If storage is unavailable, the state fallback contract applies; authentication failure does not change another party's session. Quarantine of new events is replayed offline with a gap marker: graph transitions across an unknown segment are prohibited. Input limits do not use the future session length, change assignment, remove an admitted session from evaluation/ITT, or disable the independent purchase log. They do not replace the recommendation-request limit $Q_{\max}$.

Tenant limits on events/s, bytes/s, active sessions, and the daily contribution of a single SKU must have numeric values before launch. Reaching a limit triggers throttling or quarantine with a reason code. For the graph, at most one session contribution to the support of each pair is retained; for popularity, at most one session contribution per SKU; additional SKU/source limits are registered as part of the data transformation. Spikes in edge support and popularity are checked against previous windows; automatically incorporating suspicious events into a new package is prohibited until quarantine review. Sybil load, a single-SKU attack, an oversized block, and mass history reconstruction are mandatory tests. These measures limit impact but do not guarantee identification of all fake users. Unknown operational limits mean the production profile is not ready, not that intake is unlimited.

The minimum required response record contains a pseudonymous session key, versions, the selected branch, served item identifiers, the outcome, and technical status. Full candidate lists, scores, and detailed reasons are stored only in a bounded diagnostic log; raw queries, vectors, and history are excluded from the ordinary log. The pseudonymous key is produced by a keyed transformation with namespace separation; hashing an identifier is not considered anonymization. For A/B linkage, the assignment key remains stable throughout the experiment horizon, while diagnostic-log keys rotate independently.

Initial retention policy: state is deleted after 24 hours of inactivity; $\Delta_{\text{retry}} = 24$ hours, $\delta_{\text{late}} = 30$ minutes; the replay log is retained for 48 hours, and detailed traces for 7 days. Minimal assignment and outcome records are retained until the prespecified analysis closing date plus 30 days, then deleted or replaced with aggregates without session keys. Redelivery markers live independently of state retention and for at least the retry period. Retention periods and storage owners are included in the run manifest; these values are a project profile, not a claim about legally required retention. Access is role-based, transmission and storage are encrypted, and diagnostic-trace reads are audited. Deletion covers state, the replay log, diagnostic copies, and backups under the declared expiry period; recovery does not restore deleted records without reapplying the deletion registry. Public reports and user-facing explanations do not expose session identifiers, raw queries, or action sequences.

17 Computational budget and serving

Scope of the demonstration version. For a portfolio, one local process with a verified package, state, repeatable responses, fallback, explanations, and a log is sufficient. Time boundaries, item eligibility, input limits, and the prohibition on late publication remain in force. Production load (Section 17.5), RTO/RPO and recovery of multiple instances (Section 17.6), production signatures, and monitoring are outside its required acceptance criteria and require a separate task. The run manifest's production-profile `serving`/`operations` fields, `post_load`, and S_{serve} belong to this deferred operational acceptance, not the demonstration.

17.1 Preparation, training, and request processing

Version preparation, training, and serving are measured separately. Preparation for the first run creates the graph, content vectors, and lookup tables; extensions add paths, learned vectors, approximate nearest-neighbor (ANN) indexes, and the SID tree. Training transformations and code noise are used only during training. A request uses prepared paths and item vectors, computing the session state, candidates, features, and ranking. Item-feature search in the first run is exact; shortest paths are not recomputed. Autoregressive SID decoding belongs to the serving budget of a separately enabled research variant.

17.2 Version contents

A consistent package is published:

$$\mathsf{V} = (\nu_D, \nu_{\mathcal{I}}, \nu_G, \nu_{\text{vec}}, \nu_{\text{features}}, \nu_f, \nu_{\text{policy}}, \nu_N, \nu_{\text{SID}}). \tag{85}$$

The graph, vectors, and ranker are replaced atomically. Vector-space compatibility is checked by provenance, not dimension alone.

Version ν_N combines the indirect-link topology, encoder weights, the learned item-vector table, and its search index. Version ν_{SID} contains the encoder, codebooks, decoder, code-to-item mapping, and valid-prefix tree. A disabled module receives version `not_used`; in the first run, this applies to ν_N and ν_{SID}. Replacing a codebook without updating its corresponding tree and decoder makes the package incompatible.

The package manifest contains the SHA-256 of every artifact, exact library versions and a lockfile, the build commit, data cutoff and publication time, builder identifier, and transformation dependencies. In the production service, the manifest is signed by a trusted publisher (initial profile: Ed25519); the trust key comes from a separate trusted store, not the package being verified. Before activation, the signature, all hashes, schema compatibility, and version authorization are checked. Checking a hash without authenticating the publisher is insufficient. Artifacts are read by a declarative loader without executing serialized code; paths outside the package, unsupported types, and sizes exceeding declared bounds are rejected. A local research package may use the `local_checksum_only` profile, which explicitly does not verify the authenticity of a production package. A package that fails verification is not activated.

A request follows this sequence:

1. validate input structure and pin the published package;
2. obtain the eligible catalog, verify the state's transformation version, and update the state atomically;

3. for an enabled neural variant, compute the session representation and search intent weights once;
4. retrieve candidates and their provenance;
5. compute features and scores;
6. apply final-list constraints;
7. record the response, latency, and execution status.

Fallback is included in π at its actually observed usage rate.

17.3 Memory and operation counts

Let n_I be the number of items, $|\mathcal{R}_G|$ the number of graph relations, d_G the retained neighbor count, d_T, d_V the dimensions of content vectors, p the number of features, T_f the number of trees, and D_f their maximum depth. The main costs are of order

$$\text{Mem}_G = O(|\mathcal{R}_G| n_I d_G), \tag{86}$$

$$\text{Mem}_{\text{content}} = O(n_I(d_T + d_V)), \tag{87}$$

$$\text{Work}_{\text{graph},q} = O(|\mathcal{R}_G| L_H d_G), \tag{88}$$

$$\text{Work}_{\text{rank},q} = O(Mp + MT_f D_f). \tag{89}$$

Exact content search over the catalog requires $O(n_I(d_T + d_V))$ scalar operations per request. If needed, an approximate index is compared with exact search before opening D. A complete graph over n_I^2 pairs is not materialized.

Costs of optional extensions. Item vectors of dimension d require $O(n_I d)$ memory and are recomputed before the index is published. Session representations are computed once for candidate sources and ranking features.

Indirect-edge preparation is bounded by path length, the number of expansions, and the number of retained neighbors. Deterministic early termination is part of the model; its frequency across vertices is reported.

Exact normalization over the catalog requires $O(n_I d)$ operations per request; an ANN index does not accelerate it. Training also stores intermediate values for derivatives. Blockwise log-sum-exp accumulation preserves the exact loss. Negative-item sampling defines a different procedure (Appendix A) and does not eliminate item-representation recomputation or global regularizers. Total training time and peak RAM and accelerator memory are measured. Exhausting the budget is a computational failure; the number of epochs is not changed without declaration.

ANN acceptance contract. Each approximate source has its own quality measured as

$$\text{IndexRecall}\,@m = \text{Agreement}\,@m \tag{90}$$

from (33) on a fixed probe sample. This measures agreement of neighbor-search results, not purchases. Exact and approximate search use identical vectors, filters, quota m, and tie-breaking order. Approximate results are rescored using exact dot products, deduplicated, and checked for eligibility; the original fractions of ineligible and duplicate results are reported, and both are zero in the final list. Deterministic search expansion after filtering has a specified limit; exhausting it does not permit removing the filter or increasing M.

Additionally, compute set precision with numerator $|C_q^{\text{approx}} \cap C_q^{\text{exact}}|$ and denominator $|C_q^{\text{approx}}|$, exact top-1 agreement, and score regret

$$\text{Regret}_q = \max(0, r_q(i_q^{\text{exact}}) - r_q(i_q^{\text{approx}})).$$

Here $r_q(i)$ is the dot product of the original query and item representations for this source, before temperature division and other scale factors; their norms do not exceed 1. For content search this is cosine similarity; for neural search, the numerator of its score formula. The run manifest records `regret_score=unscaled_bounded_dot_v1` and references to both representation formulas. For nonempty responses, regret lies in $[0, 2]$ and is invariant to positive scaling of the ranking score. When the exact response is nonempty and the approximate response is empty, precision, recall, and top-1 agreement are zero, while regret is $+\infty$; such empty responses are therefore prohibited on control requests. An empty exact response is counted separately and requires an empty ANN response under the same masks and filter. Stratification is fixed by available-modality masks and the popularity of the exact top-1: the popular group contains the first $\lceil 0.1|\mathcal{I}_G| \rceil$ items ordered by decreasing session count in A_G, with ties broken by ID; the rest form the second group. The boundary is fixed before ANN validation.

Initial engineering thresholds: mean IndexRecall and precision at least 0.98, top-1 agreement at least 0.99, and mean regret at most 0.01. In every predeclared nonempty segment, recall and precision must be at least 0.95, top-1 agreement at least 0.95, and regret at most 0.02. At least 10 000 nonempty exact responses overall and 1000 per segment are required; a smaller sample means `insufficient`, not a passed segment. Across ten repetitions of identical inputs, lists after exact rescoring must match exactly; the initial random-generator state, index, thread count, and tie-breaking order are included in the run manifest. These are empirical thresholds on a fixed control sample from C_{smoke}, not population guarantees. Index parameters are selected before C_{select}.

ANN must also satisfy the common P95/P99, failure, and fallback-response profile and the total memory limit in Section 17.5, including the index. On C_{select}, the change in Hit@20 relative to exact search is reported on the common $\mathcal{S}^{+}_{C_{\text{select}}}$. The allowable decrease δ_{ANN} is specified in Hit@20 units before comparison; without a specified tolerance, ANN is excluded from selection. This selection filter does not replace the final interval on D. If ANN is disabled, its status is `not_used`.

Training-example table. The candidate table size, up to $N_{\text{train}} Q_{\max} M$ rows, is estimated before launch. If requests or negative candidates are sampled, inclusion probabilities and the loss are specified; the evaluation catalog is not reduced to save computation.

17.4 Latency and fallback branches

Let ℓ_{qj} be the duration of a sequential stage, and ℓ_q^{wait} the waiting time for service. Then

$$\ell_q = \ell_q^{\text{wait}} + \sum_{j=1}^{J} \ell_{qj}. \tag{91}$$

For thresholds b_j and b_0, without assuming independence,

$$\mathbb{P}\left(\ell_q > b_0 + \sum_j b_j\right) \le \mathbb{P}(\ell_q^{\text{wait}} > b_0) + \sum_j \mathbb{P}(\ell_{qj} > b_j). \tag{92}$$

Stage percentiles do not add up to a percentile of total latency. Request rate, concurrency, catalog, and the fraction of warm states are fixed.

The fallback list is prepared at the start of the shared candidate procedure. If the primary branch exhausts its budget before the overall response deadline, an already prepared eligible branch is used, reserving time for logging and return; if none exists, an empty list is returned with a reason code. Reaching the overall deadline does not start new computation. Evaluation does not remove such a request. Fallback forms a separate branch $\mathcal{C}_q^{\text{fallback}}$ and does not extend $\mathcal{C}_q^{\text{main}}$. When a ready branch is selected with time reserved for logging and return, `branch` and $\mathcal{C}_q^{\text{served}}$ are recorded; mixing the two sets to improve the candidate bound is prohibited. Always, $L_q \subseteq \mathcal{C}_q^{\text{served}} \subseteq \mathcal{I}_q$ and $|\mathcal{C}_q^{\text{served}}| \leq M$. If no ready branch is selected, the active set is empty. If the deadline or mandatory write is not satisfied after selection, $L_q = \varnothing$ and `delivery_status=error`, but the recorded candidate set is retained: at the session level, delivery loss remains in $U_s(\pi) - \text{Hit}_s(\pi)$.

For the demonstration, the initial overall timeout $b_{\text{deadline}} = 1000$ ms is fixed before launch and covers the entire response, including the mandatory write. This is a waiting limit, not a performance guarantee under load. Individual stages have no hard deadlines; the reserve for logging and returning fallback is an execution parameter, not a separate acceptance threshold. The production profile has separate requirements in Section 17.5. A monotonic clock enforces the overall deadline with time reserved for returning fallback. A noninterruptible operator is bounded in advance or isolated; asynchronous cancellation does not guarantee completion. Each operation uses the remaining overall time after accounting for the still-required write and return; no new independent stage deadline is created. Retries share the same budget. After a branch switch, a late result from a cancelled operation does not change the response or overwrite state; a confirmed state write remains valid even if the response fails. Unfinished operations are cleaned up in an isolated pool with a fixed limit; new calls are rejected when it is full. Separate codes are `invalid_input`, `overload_rejected`, `backend_timeout`, `cancelled`, `state_replay_limit`, and `logging_failure`; a single aggregate `error` does not replace these reasons.

SID runs only when sufficient time remains in the overall candidate-retrieval budget. The step limit and search width are fixed. When time is exhausted, declared quotas and fallback rules apply, without a hidden increase in M. SID skip frequency and the quality change due to the time limit are reported separately. If fallback is not yet ready or the mandatory write is incomplete, a technical failure code is returned; the request remains in the evaluation denominators.

The demonstration log links the request, package, candidates, public response, and outcome. Two-phase commit and RPO=0 are not required; a write failure or unknown delivery is not declared a successful completion. Acceptance checks normal primary serving, replay without recomputation, an actual computation timeout with fallback, and failure with incomplete state. The HTTP response is checked against the log. Total latency, load, and memory are published as measurements; P95/P99 and fallback rate have no mandatory thresholds for the demonstration. Normal CPU-core scheduling is retained.

17.5 Reproducible load profile

Deferred production profile: applies only under a separate task and does not block the demonstration API. Its requirements have not yet been verified by testing.

Parameter	Initial value
Compute node	8 virtual CPU cores, 32 GiB RAM, no GPU during serving
Offered load	100 requests per second
Load generator	Response-independent schedule; at most 32 outstanding generator requests
Warm-up	5 minutes before measurement
Measurement	30 minutes, three separate repetitions
Service latency target	P95 ≤ 50 ms, P99 ≤ 100 ms
Total peak RAM of service processes	At most 28 GiB
Response deadline	120 ms from arrival at the service
Allowed technical failures	At most 0.1% of all scheduled requests
Allowed fallback-response rate	At most 1%

The report specifies the CPU model, processes and threads, library versions, numeric type, catalog, index parameters, and memory. Equal core counts do not guarantee equal performance; GPU execution is tested separately.

Overall memory limit. The RAM limit applies to all variants, including exact search, indexes, local state, queues, and concurrently loaded packages during updates/rollback. The initial metric is the maximum summed PSS of service processes: shared pages are accounted for proportionally. If PSS is unavailable, a conservative sum of peak RSS across all test processes is allowed; shared pages counted multiple times are not subtracted using an unverified estimate. The selected measurement tool, sampling interval, and process set are fixed before testing and are identical across compared variants. Exceeding the tabulated limit means `failed`, regardless of ANN.

Distributions of prefix length, search presence, missing values, and repeat requests come from C_{smoke} inputs or a preapproved production profile. Outcomes from D are not used to select load. Session states reproduce the declared transitions; repeating the same warm request does not replace a request stream. Cold-cache startup and recovery after failure are measured separately.

The generator schedules requests every 10 ms independently of responses; inputs and initial state are identical for all variants. A burst profile is declared separately. If the limit of 32 prevents sustaining 100 requests per second, undelivered requests and the actual rate are recorded; a slowed-down test does not establish the claimed load. Server concurrency and queue limits are specified separately from the generator limit. For the initial node, the server allows at most 16 active requests and 16 waiting requests; input reception and queueing together are limited to 10 ms. When full, the queue does not grow: the service returns `overload_rejected` without an expensive state read. Limits are global to the node, not independently repeated in each process. Rejection and its resulting empty response are included in operational denominators; an assigned session remains in ITT. A recommendation request may be retried only with the same idempotency key and within the original deadline; a server response does not trigger an unlimited automatic retry loop. Event delivery follows Section 16.8 and does not restart an old response deadline.

Service latency is measured from the first request byte at its ingress to readiness of the complete response, including waiting, feature retrieval, and the mandatory write. External network and client processing belong to a separate end-to-end latency. Scheduled send time, actual send time, service arrival, and completion receive separate timestamps; generator delay is not hidden inside an incomplete measurement.

Requests are selected by scheduled send time and observed until completion or deadline, includ-

ing those unfinished at the end of the window. For acceptance percentiles, failure, nondelivery, or deadline expiry receives latency $+\infty$; actual times to error are retained. A fallback response uses its measured latency and branch marker. Percentiles over successful primary responses alone are supplementary diagnostics. The empirical quantile at level $p \in (0, 1]$ for $n > 0$ values is defined as $x_{(\lceil pn \rceil)}$ after sorting, including $+\infty$. For an empty window, rates and quantiles are undefined: `insufficient`, not zero. The status log is reconciled with an independent ingress counter and the generator schedule; missing records count as failures and are not removed from the denominator. This rule also applies to operational windows W.

Each of the three repetitions must meet latency, failure, and fallback-response targets; pooling repetitions must not hide a failed run. Throughput, maximum queue length, actual concurrency, and CPU/RAM usage are reported separately. Load acceptance precedes the A/B test; during the experiment, latencies and failures continue to be monitored as guardrail metrics.

17.6 Recovery and rollback

Before publication, the last verified package version and its state schema are retained. The initial rollback RTO requirement is 60 s from the operator/guardrail-controller decision to readiness of the verified version on all instances; RPO for confirmed assignment and outcome writes is zero. These are targets tested through fault injection, not reported measurements. Each request is pinned to one version; rollback does not mix artifacts or change previously stored responses. Incompatible state is reconstructed or migrated under a verified rule; eligible fallback applies until it is ready. Returning to an old version with an expired catalog is prohibited. Loading two versions is included in peak memory; if this is impossible, the switch is staged with the same RTO and availability checks. Testing covers process loss, unavailable state, mandatory-write failure, package corruption, and rollback to a different feature version. Measured RTO/RPO and failed-request codes are reported separately from steady-state load results.

18 Monitoring quality and data changes

18.1 Immediately observable metrics

For a request window W, define

$$\widehat{e}_W = \frac{1}{|W|} \sum_{q \in W} \mathbf{1}\{\text{error}_q = 1\}, \tag{93}$$

$$\widehat{f}_W = \frac{1}{|W|} \sum_{q \in W} \mathbf{1}\{\text{fallback}_q = 1\}, \tag{94}$$

$$\widehat{u}_W = \frac{1}{|W|} \sum_{q \in W} \mathbf{1}\{\text{the prefix contains an unknown item}\}, \tag{95}$$

$$\widehat{m}_{W,j} = \frac{1}{|W|} \sum_{q \in W} \mathbf{1}\{x_{qj} \text{ is missing}\}. \tag{96}$$

Response length, the fraction of empty responses, latency percentiles, the candidate-count distribution, snapshot age $t_q - r(q)$, and the incompatible-version rate are measured separately. For the neural component, monitoring covers rates of disabled modalities, vector norms, the distribution of intent weights, and ANN disagreement with exact search. The semantic module additionally tracks the number of used codes, frequencies of code collisions between items, truncation of

large groups, and the quality gap between soft training and final discrete serving. Lower latency achieved through more empty responses is not considered an improvement without accounting for the other constraints.

18.2 Distribution changes

Bin boundaries for a numeric feature are determined on training data; missingness forms a separate category. Let p_b and q_b be smoothed proportions in the reference and current windows, $b = 1, \dots, B_h$:

$$p_b = \frac{n_b + \varepsilon_h}{n + B_h \varepsilon_h}, \qquad q_b = \frac{m_b + \varepsilon_h}{m + B_h \varepsilon_h}, \qquad h_b = \frac{p_b + q_b}{2}. \tag{97}$$

The Jensen–Shannon divergence is defined as

$$\mathrm{JS}(p, q) = \frac{1}{2} \sum_b p_b \log \frac{p_b}{h_b} + \frac{1}{2} \sum_b q_b \log \frac{q_b}{h_b}. \tag{98}$$

It is zero for identical distributions and does not exceed $\log 2$. It indicates an input change; it does not prove that recommendation quality has deteriorated. Thresholds are set using sequential control windows, without adjustment after inspecting the problematic period.

Registry and decision rule. The registry includes input features and missingness, unknown items, source masks, source quotas and list sizes, the fraction of unique candidates, scores and final-list length, the selected branch, and failure reasons. Categories and numeric boundaries are frozen from early data; smoothing in (97) uses $\varepsilon_h = 1/2$. The reference window contains at least 10 000 requests; current windows are nonoverlapping UTC hours with at least 1000 requests and 200 sessions. Lower volume returns `insufficient`; the window is not declared stable. A separate counter of insufficient telemetry raises a notification after three consecutive such windows.

Thresholds are fixed jointly for the entire registry. Before operation, on at least 100 control hourly windows preceding C_{select} and not overlapping the reference window, compute $A_w = \max_j \mathrm{JS}_{w,j} / 0.05$. Window w uses only a previously available reference. The threshold $h_{\text{drift}} = \max(1, Q_{0.99}(A_w))$ is fixed together with the reference, feature set, and nearest-rank empirical-quantile rule. Control windows from C_{smoke} are allowed; the registry and calibration rule are specified before opening them. Insufficient control windows mean unverified monitoring, not permission to tune thresholds on D. On calibration windows, record separately the exceedance rate $A_w > h_{\text{drift}}$ and the alert frequency of the complete rule below; these are in-sample quantities, not false-alarm estimates. Such an estimate requires separate windows and a justified assumption that no real shift occurred. Time of day, weekday, and window dependence are considered in diagnostics; future alert frequency is not guaranteed.

An alert is raised when $A_w > h_{\text{drift}}$ in two consecutive sufficient windows; it is cleared after three windows with $A_w < 0.8 h_{\text{drift}}$. Repeat notifications for the same cause are sent at most once per hour, but new failures are recorded immediately. The maximum over the full registry accounts for multiplicity in empirical calibration; individual JS values are not interpreted as p-values and are not subjected to FDR correction as significance tests. Changing the registry, boundaries, or reference requires recalibration on early windows. The notification recipient and investigation owner are recorded in the run manifest.

A distribution-change alert triggers checks of completeness, versions, sources, and output distributions, without automatic retraining. Separate 5-minute operational windows with at least 1000

requests are compared with the initial technical-failure limit of 0.1% and fallback limit of 1%. Exceedance in two consecutive windows activates a previously verified fallback/rollback and pauses enrollment of new experimental units; already assigned units remain in ITT. Initial guardrail requirements, regardless of window volume: the same protection is triggered by 20 consecutive technical failures of valid admitted requests or three consecutive failed internal health checks. Checks run every 5 s with separate diagnostic IDs and a 120 ms deadline; they do not modify user state or enter business-metric denominators. Request sequences use arrival order; an unfinished request receives its outcome when its deadline expires. Success resets the corresponding counter. Errors from untrusted or invalid inputs do not trigger a global rollback of a healthy version. An unauthentic package is blocked before activation; a single attempt to load it does not stop the active service. A detected eligibility violation in actual serving blocks it immediately. During A/B testing, the rules of Section F apply: a conclusion from an ordinary fixed-horizon analysis after a guardrail stop is not considered confirmatory.

Coveo dates are shifted by an integer number of weeks. Weekday order is preserved. No link to actual holidays is established without the original calendar dates.

18.3 Metrics with delayed outcomes

Historical evaluation updates Hit@20 for recorded targets under the common calendar admission rule. Unknown coverage remains unknown. Production-service conversion is computed only with verified complete outcome logging for mature cohorts defined by assignment time. Counting early positive outcomes while negative outcomes remain incomplete is prohibited.

In operation, current conversion can be observed as $\widehat{p}_W = \sum_{s \in W} Y_s / |W|$. Its change relative to the previous week mixes the model effect, traffic composition, and external circumstances. The causal effect is estimated against a concurrently running control group. Segments for confirmatory analysis of actual exposures are defined before variant assignment. For example, search use after assignment cannot support a claim about a causal effect "among those who used search."

18.4 Model updates

Data changes trigger diagnostics: log completeness, version compatibility, target availability, and changes in candidate sources. A new model is trained only on history that passes calendar admission and the complete-read check, undergoes the same checks, and is published as a new version. Crossing the JS threshold alone does not authorize automatic model replacement.

During a confirmatory A/B test, behavioral lookup tables and the model are frozen by default. Training a shared model on responses from both variants creates cross-group interference; this requires a separate experimental design. Operational fixes that do not alter the prescribed policy are documented.

19 Explaining results and limits of interpretation

For each recommended item, candidate-source provenance, source scores, and the snapshot version are retained in a diagnostic trace under the access and retention rules of Section 16.8. Related events are replaced by minimal reason codes; their raw sequence is not included in the public explanation. A graph explanation identifies the relation, the prefix item with the largest contribution,

and support n_{ij}^r. A content explanation identifies the modality used and measured similarity. A search explanation refers to the available query and compatible vector space.

An explanation describes the computation, not the cause of a purchase. It is limited to Coveo fields: an anonymized vector does not reveal an item's name, purpose, properties, or compatibility with a particular phone model.

The first variant uses templates based on the computation trace. A language model does not extend the available facts; ranking based on one requires a catalog with text and separate validation of quality, cost, and feature availability.

20 Result completeness and acceptance criteria

Historical evaluation is accepted under the `observed_purchases_v1` protocol in Section 14.1. Success does not establish purchase growth and does not, by itself, authorize an A/B test.

20.1 Result states

The state is represented by the tuple

$$S = (S_{\text{data}}, S_{\text{math}}, S_{\text{offline}}, S_{\text{serve}}, S_{\text{business}}). \tag{99}$$

Each component takes a value in $\{\text{not_run}, \text{passed}, \text{failed}, \text{insufficient}\}$. For business impact, not_observed is also used if no randomized test has been conducted. Failure and insufficient data are distinct states.

Component	Required conditions for `passed`
S_{data}	The run manifest is complete for the current stage; hashes and schema match; the canonicalization rule is fixed; session overlap across periods is empty; selected sessions have been fully read from the data release, calendar admission and recorded targets are computed correctly. Unknown telemetry completeness is preserved, including for sessions with purchases; exclusions are listed.
S_{math}	All enabled-component tests required at the current stage under K pass within the specified tolerances, including derivatives, mask completeness, time boundaries, and the hit-loss identity.
S_{offline}	The frozen protocol has been executed on D, with $N = N_D^+ > 0$ and an independent complete report on Hit@20_D, including the sensitivity comparison without metadata. Replay mode and trace completeness satisfy Section 16.5. Every reported estimate meets the assumptions, information and nondegeneracy thresholds in 15.4, the minimum block count, and required half-width: h_{pair} for a pair, h_{single} for one policy, both expressed as proportions. The sign of the difference is reported separately: `passed` means the estimate is usable, not that the model is superior.

Component	Required conditions for `passed`
S_{serve}	Each of the three load runs satisfies all thresholds in 17.5, including the total limit and peak memory accounting under 17.5; feature-parity, failure, recovery, integrity, and input-limit checks pass. Partial source failure is allowed only within the registered fallback policy and its limits.
S_{business}	The complete authorized A/B analysis is performed without assignment or logging violations, the lower bound exceeds the prespecified business threshold, and guardrail constraints are satisfied.

A detected contract violation or measured breach of a technical threshold yields `failed`; a correct procedure with insufficient sessions containing a recorded target, blocks, or precision yields `insufficient`. For the business component, with a usable interval $[\ell_B, u_B]$ and all other conditions satisfied, the status is `passed` when $\ell_B > \delta_{\text{accept}}$, `failed` when $u_B \leq \delta_{\text{accept}}$, and `insufficient` when $\ell_B \leq \delta_{\text{accept}} < u_B$; the threshold is defined in F.3. If both a violation and insufficient data are known, `failed` takes precedence; all reasons are retained. `not_run` means the stage has not yet been executed.

Run stages are defined in 13.2. While D remains closed, $S_{\text{offline}} = \text{not_run}$, regardless of debugging success. For local `ideal_deterministic` operation, state S_{offline} does not depend on S_{serve}; Until service acceptance checks are run, $S_{\text{serve}} = \text{not_run}$. The report includes the loss decomposition in 14.3 and source contributions in 14.4; technical failures form a separate breakdown, not an additional term in the hit-loss decomposition.

A pilot A/B test additionally requires verified complete experiment outcome logging, denoted $O_{\text{AB}} = 1$:

$$\mathsf{Ready}_{\text{pilot}} = \mathbf{1}\{O_{\text{AB}} = 1\}\,\mathbf{1}\{S_{\text{data}} = S_{\text{math}} = S_{\text{offline}} = S_{\text{serve}} = \text{passed}\}. \tag{100}$$

For this formula, S_{serve} must be obtained specifically under the operational profile: tenant and item limits are filled in, the package signature is verified, retention and access rules are active, and monitoring is calibrated on enough windows. Success under the local `local_checksum_only` profile or `insufficient` monitoring does not authorize a pilot. The acceptance profile is stated explicitly in the report. Demonstrated business superiority additionally requires a positive decision on ΔB under Section F.3 and all guardrail constraints. Power is a property of the prespecified plan, not of the result computed after the experiment. A complete specification does not establish that the system has passed testing.

20.2 Single-request result

$$\mathsf{R}_q = (q, s, t_q, \mathsf{V}, L_q, \mathcal{C}_q, \text{branch}, \text{delivery_status}, \text{scores}, \text{sources}, \text{status}, \text{latency}). \tag{101}$$

This is a logical diagnostic structure, not a requirement to retain all candidates and scores for every production-service request indefinitely. The minimum required record and bounded diagnostic log are separated under 16.8. Recorded targets are joined only during evaluation; production-experiment outcomes are joined after maturation. Candidates are replayed using the pinned mode and trace from 16.5. Without a complete trace, actual degradation and $\mathcal{C}_q^{\text{served}}$ are not considered measured; a healthy recomputation does not automatically reconstruct them. The historical report retains the session set, recorded targets, the unknown status of telemetry completeness, selection parameters, intervals, failures, and cost. The run identifier combines data, configuration, code,

and dependency versions. Changing the initial random-generator state does not create an independent visitor sample.

20.3 Complete computational formulation

For configuration ξ, historical artifacts A_G, retrieval parameters θ_ξ, and ranker f_ξ are constructed sequentially. Denote their fitting procedures by $\mathcal{T}_G, \mathcal{T}_N, \mathcal{T}_R$:

$$\mathsf{A}_G = \mathcal{T}_G(D_{A_G}; \xi), \tag{102}$$

$$\theta_\xi = \begin{cases} \varnothing, & \text{neural extension disabled,} \\ \mathcal{T}_N(D_{A_N}; \mathsf{A}_G, \xi), & \text{extension enabled,} \end{cases} \tag{103}$$

$$\mathcal{C}_q^\xi = \mathcal{G}_\xi(\mathcal{H}_q, \mathsf{A}_G, \theta_\xi, \mathcal{I}_q), \tag{104}$$

$$x_{qi}^\xi = \Phi_\xi(\mathcal{H}_q, i, \mathsf{A}_G, \theta_\xi), \tag{105}$$

$$f_\xi = \mathcal{T}_R(\{(x_{qi}^\xi, y_{qi}) : s \in \mathcal{S}_B^+, q = q_s^{(1)},\ i \in \mathcal{C}_q^\xi\}), \tag{106}$$

$$\widehat{\xi} \in \operatorname*{arg\,max}_{\xi \in \Xi_{\text{feasible}}} \text{Hit@20}_{C_{\text{select}}}(\pi_\xi). \tag{107}$$

The history for $\mathcal{T}_G$, first requests from sessions with a recorded target for $\mathcal{T}_R$, and their targets are defined in Section 14.1. In the first run, $\Xi_{\text{feasible}} = \{\xi_0\}$: one prespecified configuration is executed, without tuning on C_{select} results. Period A_N remains reserved; its data do not augment the baseline system's graph or lookup tables. Configurations for subsequent comparisons and the selection rule are fixed before opening C_{select}. Ξ_{feasible} includes only variants that pass the declared constraints. When comparing the original model with one new variant, an exact Hit@20 tie retains the original model. Time and memory are reported separately; load runs are unnecessary for tie-breaking. A Hit@20 tie tolerance is not tuned to the results: integer hit totals are compared on common sessions. If no variants are eligible, selection receives status `insufficient` or `failed` according to the established reason, and no winner is assigned. The final list is sorted by $f_{\widehat{\xi}}$ unless a separate diversification procedure was selected in advance. Period D is excluded from every fitting or selection procedure. For the SID variant, θ_ξ includes an additional encoder, decoder, and code-to-item mapping trained before B.

The business objective (11) is tested with actual exposures under Appendix F.

Part II

Neural extension

21 Rationale and sequence for the neural extension

This part follows the baseline run on p. 1. It adds one learned retrieval component; multiple intents and item vectors are components of it, not separate mandatory stages.

21.1 Relationship to work from 2024–2026

The table lists the adopted principles. The formulas below define an original adaptation for Coveo. Source status was checked as of September 19, 2026.

Work	Principle and role in the project
SPARE, ACM TORS, 2025 [3]	Shortest paths and compact graph propagation. Relation types, their costs, and the purchase target are adapted.
MiaSRec, SIGIR 2024 [4]	Multiple session intents. Search-dependent intent weights are added in this model.
MACL, Pattern Recognition, 2026 [5]	Contrastive modality alignment; Coveo provides precomputed vectors.
DIGER, SIGIR 2026 [7]; GRID, 2025 [6]	Learned semantic codes and comparison of coding methods. Used only in a separate study.

The results of these studies concern their own data, products, and objectives; they do not specify an expected purchase uplift on Coveo.

21.2 Configuration and training chronology

The neural policy π_{main} extends π_{base} with learned graph and content vectors, session intents, search weights, and a new candidate source. The research policy π_{SID} additionally uses semantic codes. Both preserve the overall limit M.

Training order and calendar admission of targets are specified in 22.1. The graph is published at r_G before A_N, and learned retrieval at r_N before B. Fitting on A_N is not independent evaluation on that period. Before ranker training on B, the encoder and item vectors are frozen; tree gradients do not flow into them.

21.3 Implementation order and controlled extensions

The first training run uses the configuration in H.2. One and two intents are compared according to the plan in I. If the budget for the exact full-catalog loss is exceeded, the sampled mode in A is allowed; its quality is checked separately. It receives no additional candidate quota.

To control for history processing, Appendix B retains the item graph and input information while replacing the session encoder. A variant with additional cart actions or a sequence of search

queries is tested separately: it also changes the input set. The value of the graph itself is tested by disabling the graph branch.

Diagnostics in 14.5 motivate the next experiment. All comparisons specify the control, changed component, number of runs, final hit rate, uncertainty of the difference, and cost. A smaller gap to the candidate bound is not, by itself, a criterion for improvement.

21.4 Hypotheses following diagnostics

The following variants are outside the required recipe; their benefit is not assumed.

Search history. When losses occur in sessions with multiple queries, examine the number of available queries, changes in their vectors, and consistency with views. All features use available history; a change in interest is not treated as an error in the search signal. Original text length cannot be reconstructed from the supplied vector.

Hard negatives. Investigate changes to the sampling distribution under the conditions in Appendix A; compare final policies, not numerical values of different approximate losses.

Category smoothing. When losses occur on rare items, test a mixture of the graph row and a normalized category distribution. For weighted W_{ij}, a Bayesian interpretation requires a separate observation model. A full prior must not be expanded into a dense catalog matrix: a bounded support or a separate candidate source is needed.

A neural ranker is considered after diagnosing losses with retrieval fixed (Section 11). Predicting purchase probability and calibrating it require a different observation set (Section 11.7). Catalog expansion requires a separate new-item admission rule from Appendix C; metadata alone do not establish an availability date.

22 Learned model of graph, content, and session intents

Data and training rules are specified in 22.1, quotas and settings in H.2. The sampled denominator in A does not change the representation architecture.

22.1 What is learned and when

Catalog $\mathcal{I}_G$, graphs P^r, paths, and statistics are built from A_G and fixed. Parameters θ_N are trained on A_N using first requests under calendar admission (49). Ranker training, configuration selection, and final evaluation follow the common order:

$$A_G \prec A_N \prec B \prec C \prec E \prec D. \tag{108}$$

Symbol $\prec$ denotes strict temporal precedence with a calendar gap of $H + g$. Period E is reserved and unused. No session belongs to two periods; telemetry completeness remains unknown.

Neighbors are determined only from A_G: a graph built from all of A_N would reveal future links to its early prefixes. The epoch count, dimension, and loss coefficients for the first training run are fixed. Subsequent comparisons on C_{select} follow a prespecified plan; every variant is trained only on A_N without further training on C or E.

22.2 Shortest paths in the fixed graph

To search for indirect links, combine known relations:

$$\bar{P}_{ij} = \sum_{r \in \mathcal{R}_G} \zeta_r P^r_{ij}, \qquad \zeta_r \geq 0, \quad \sum_r \zeta_r = 1. \tag{109}$$

Coefficients ζ_r are part of the configuration; row sums may be less than one because of empty relations. For $\bar{P}_{ij} > 0$, define the dimensionless edge length

$$\ell^G_{ij} = c_G - \log \bar{P}_{ij}, \qquad c_G > 0. \tag{110}$$

A strong link reduces length, while c_G penalizes each hop. This geometry does not define purchase-sequence probabilities.

For a directed path $p = (i = i_0, i_1, \ldots, i_h = j)$, its length is the sum of edge lengths. Bound the number of hops by H_P:

$$D^G_{ij} = \min_{\substack{p: i \rightsquigarrow j \\ 1 \leq |p| \leq H_P}} \sum_{(u,v) \in p} \ell^G_{uv}. \tag{111}$$

If no path exists, the minimum is $+\infty$; $i = j$ is excluded. All lengths are positive, so a shortest path has no cycle. Distance is directed: generally $D^G_{ij} \neq D^G_{ji}$. For each vertex, retain at most d_P items with the smallest $D^G_{ij} \leq R_P$, $R_P > 0$. The resulting set $\mathcal{N}^P_i$ and distances are frozen. Equal distances are resolved by identifier order.

Bounded search tracks the state "vertex, number of hops." Ordinary search with a single label per vertex may miss a valid path under the H_P constraint. The approximate variant expands at most B_P states per source vertex, in increasing length order with fixed tie-breaking. When the budget is exhausted, the valid neighbors found are retained with a truncation flag. Their distances $\widetilde{D}^G_{ij}$ are lengths of discovered paths, not necessarily exact minima. Below, D^G_{ij} denotes the configuration-specified variant: exact or approximate. It is shared by the model version and computed in advance.

Illustrative example: $\bar{P}_{ab} = 0.8$, $\bar{P}_{bc} = 0.7$, $\bar{P}_{ac} = 0.1$, $c_G = 0.1$. Edge $a \to c$ has length approximately 2.403, while path $a \to b \to c$ has length 0.780; when $H_P \geq 2$, the link to c may be retained through b.

22.3 Learned links and separate item representations

For $i \in \mathcal{I}_G$, the learned identifier vector $b^N_i \in \mathbb{R}^d$ is initialized as specified in Section 22.10. Introduce normalization

$$\mathsf{n}_\varepsilon(v) = \frac{v}{\max(\varepsilon_N, \|v\|)}, \qquad \varepsilon_N > 0. \tag{112}$$

It is defined at zero as well; the result's norm does not exceed one. The usable-direction mask is $\mathbf{1}\{\|v\| > \varepsilon_N\}$.

The historical distribution over retained paths and its learned adjustment are defined as follows:

$$\begin{aligned} \nu^P_{ij} &= \frac{\exp(-D^G_{ij}/\tau_P)}{\sum_{v \in \mathcal{N}^P_i} \exp(-D^G_{iv}/\tau_P)}, \\ A^N_{ij} &= \frac{\nu^P_{ij} \exp(\eta_N \langle \mathsf{n}_\varepsilon(b^N_i), U_N \mathsf{n}_\varepsilon(b^N_j) \rangle)}{\sum_{v \in \mathcal{N}^P_i} \nu^P_{iv} \exp(\eta_N \langle \mathsf{n}_\varepsilon(b^N_i), U_N \mathsf{n}_\varepsilon(b^N_v) \rangle)}. \end{aligned} \tag{113}$$

Here $\tau_P > 0$ and $\eta_N \geq 0$ are configuration parameters, and $U_N \in \mathbb{R}^{d \times d}$ is learned on A_N. The formulas apply only to nonempty $\mathcal{N}^P_i$; otherwise, all A^N_{ij} are zero. Weights are learned over fixed links.

Log probabilities. On a nonempty admissible support, first compute $\widetilde{z}_j = z_j - \max_v z_v$, then $\mathrm{logsoftmax}(z)_j = \widetilde{z}_j - \log \sum_v e^{\widetilde{z}_v}$. All differences $z_j - \mathrm{LSE}(z)$ below are computed this way, not through the logarithm of rounded softmax values. For (113), retain

$$\begin{aligned} \ell^P_{ij} &= -D^G_{ij}/\tau_P - \mathrm{LSE}_{v\in\mathcal{N}^P_i}(-D^G_{iv}/\tau_P), \\ z^A_{ij} &= \ell^P_{ij} + \eta_N \langle \mathsf{n}_\varepsilon(b^N_i), U_N \mathsf{n}_\varepsilon(b^N_j)\rangle, \\ \ell^A_{ij} &= z^A_{ij} - \mathrm{LSE}_{v\in\mathcal{N}^P_i}(z^A_{iv}), \qquad A^N_{ij} = \exp(\ell^A_{ij}). \end{aligned} \tag{114}$$

Rounding an exponential to zero is allowed; its logarithm is not reconstructed from that zero. Entropy and KL below use the stored log probabilities and are differentiated through the same computation. Masked positions are excluded before operations; empty support produces a separate zero branch with zero gradient. Multiplying zero by infinity and replacing NaN with zero are prohibited.

The four branch representations are

$$\begin{aligned} e^{\mathrm{id}}_i &= \mathsf{n}_\varepsilon(b^N_i), & e^G_i &= \mathsf{n}_\varepsilon\left(\sum_{j\in\mathcal{N}^P_i} A^N_{ij} B_N b^N_j\right), \\ e^T_i &= \mathsf{n}_\varepsilon(T_N \bar{v}^{\mathrm{text}}_i), & e^V_i &= \mathsf{n}_\varepsilon(V_N \bar{v}^{\mathrm{image}}_i). \end{aligned} \tag{115}$$

Matrix dimensions are $B_N \in \mathbb{R}^{d\times d}$, $T_N \in \mathbb{R}^{d\times d_t}$, and $V_N \in \mathbb{R}^{d\times d_v}$. The identifier mask is $\mathbf{1}\{i \in \mathcal{I}_G\}$; the graph mask is $\mathbf{1}\{\mathcal{N}^P_i \neq \varnothing\}$; text and image use M^{text}_i and M^{image}_i. Denote these four masks by m^b_i, $b \in \{\mathrm{id}, G, T, V\}$. An unavailable branch produces a zero vector and a zero mask.

Let χ_i contain modality masks and $\log(1+V^{\mathrm{detail}}_i)$ from A_G. When at least one branch is available, define

$$\gamma^N_{ib} = \frac{m^b_i \exp(c_b + v_b^\top \chi_i)}{\sum_a m^a_i \exp(c_a + v_a^\top \chi_i)}, \qquad e_i = \mathsf{n}_\varepsilon\left(\sum_b \gamma^N_{ib} e^b_i\right). \tag{116}$$

Coefficients c_b, v_b are learned. If all masks are zero, $e_i = 0$; define $m^N_i = \mathbf{1}\{\|e_i\| > \varepsilon_N\}$. Zero directions are excluded only from neural retrieval.

Coveo vectors are fixed; the projections and listed parameters are learned. An unknown identifier in the prefix may use the supplied vectors under the static-metadata assumption. Its identifier and graph branches are absent, and historical frequency is zero; the candidate catalog is not expanded.

The learned search representation uses the same text projection:

$$z^N_q = \mathsf{n}_\varepsilon(T_N \bar{z}_q), \qquad m^{SN}_q = M^S_q \mathbf{1}\{\|T_N \bar{z}_q\| > \varepsilon_N\}. \tag{117}$$

Query and image are compared only in the learned shared space, not by taking the dot product of their original vectors.

22.4 Multiple intents and selection by search context

From $\mathcal{D}_q$, retain events with at least one available input branch, that is, $\sum_b m^b_i > 0$, obtaining $\mathcal{D}^N_q$. This condition depends on the data, not the learned norm of e_i. Let n^N_q be the number of distinct items in this set, $J_I \geq 1$ the upper bound on the number of intents, and $J_q = \min(J_I, n^N_q)$. Each

$k = 1, \ldots, J_I$ has a learned vector $c_k^N \in \mathbb{R}^d$. For event ℓ, define

$$
\begin{aligned}
r_{q\ell}^N &= \tanh\left(F_N e_{i_{s(q)\ell}} + a_N^t \log\left(1 + \frac{t_q - t_{s(q)\ell}}{t_{\text{unit}}}\right) + a_N^f \log(1 + n_{q,i_{s(q)\ell}}^{\text{detail}}) \right), \qquad (118)\\
z_{qk\ell}^{H,N} &= -\frac{t_q - t_{s(q)\ell}}{\tau_H} + (c_k^N)^\top r_{q\ell}^N, \qquad k \le J_q,\\
\beta_{qk\ell}^N &= \exp\left(z_{qk\ell}^{H,N} - \mathrm{LSE}_{v \in \mathcal{D}_q^N}(z_{qkv}^{H,N}) \right),\\
h_{qk} &= \mathsf{n}_\varepsilon \left(\sum_{\ell \in \mathcal{D}_q^N} \beta_{qk\ell}^N e_{i_{s(q)\ell}} \right).
\end{aligned}
$$

Matrix F_N has dimension $d \times d$, and a_N^t, a_N^f are vectors of dimension d. Function tanh is applied coordinatewise. This normalizes the temporal weights in (20) with an intent adjustment, computed without intermediate rounding of $\alpha_{q\ell}$: mask $\mathcal{D}_q^N$ is applied before stable LSE. Frequencies and ages come from the prefix; equal timestamps have equal ages, and permutations within a group do not change the sums.

Dynamic intent selection through entmax from the full MiaSRec is not reproduced. Upper bound J_I limits cost; for a single item, $J_q = 1$. Directions are latent vectors, not established categories of customer goals.

Let $\mathcal{K}_q = \{1, \ldots, J_q\}$ be the direction indices. If this set is empty but $M_q^S = 1$, define $\mathcal{K}_q = \{1\}$ and $h_{q1} = z_q^N$. If the input search vector is also absent, the set remains empty. A zero direction retains the training request with zero scores: shrinking norms must not exclude examples from the loss. For nonempty $\mathcal{K}_q$, intent weights are

$$
\begin{aligned}
\delta_{qk}^N &= a_k^I + (v_k^I)^\top h_{qk} + \beta_S m_q^{SN} e^{-d_q^S/\tau_{SN}} \langle z_q^N, h_{qk} \rangle, \qquad (119)\\
\omega_{qk} &= \frac{\exp(\delta_{qk}^N)}{\sum_{v \in \mathcal{K}_q} \exp(\delta_{qv}^N)}, \qquad u_q^I = \sum_{k \in \mathcal{K}_q} \omega_{qk} h_{qk}.
\end{aligned}
$$

Parameters a_k^I, v_k^I are learned; for entropy, retain $\ell_{qk}^I = \delta_{qk}^N - \mathrm{LSE}_{v \in \mathcal{K}_q}(\delta_{qv}^N)$ and $\omega_{qk} = \exp(\ell_{qk}^I)$. $\beta_S = \log(1 + e^{b_S}) \ge 0$ is parameterized by learned b_S, and $\tau_{SN} > 0$ is measured in seconds. When $m_q^{SN} = 0$, the entire search term is zero; d_q^S is fully defined in (27). Weight ω_{qk} specifies a direction's relative importance, not a purchase probability. With only one direction, its weight is always one, so add a direct search contribution with a bounded coefficient:

$$
\begin{aligned}
\ell_q^g &= \begin{cases} b_g + (v_g)^\top u_q^I + (w_g)^\top z_q^N + a_g^t a_q^S, & m_q^{SN} = 1, \\ 0, & m_q^{SN} = 0, \end{cases} \qquad (120)\\
g_q^{SN} &= m_q^{SN} \bar{g}_N \,\mathrm{sigmoid}(\ell_q^g), \qquad 0 \le \bar{g}_N \le 1,\\
u_q^N &= (1 - g_q^{SN}) u_q^I + g_q^{SN} z_q^N.
\end{aligned}
$$

Here $\mathrm{sigmoid}(x) = (1 + e^{-x})^{-1}$; $b_g, a_g^t \in \mathbb{R}$ and $v_g, w_g \in \mathbb{R}^d$ belong to θ_N. In the absence of a valid search direction, $m_q^{SN} = 0$, giving $g_q^{SN} = 0$, including when $M_q^S = 1$ but the projection is zero. Degenerate search does not reduce the scale of a usable history vector. Limit $\bar{g}_N$ is specified by the configuration. For empty $\mathcal{K}_q$, define $u_q^I = 0$. Training-example admission uses the original mask M_q^S, not learned m_q^{SN}; reducing the projection norm does not exclude the example from the loss. The direction threshold creates a piecewise differentiable computation: the mask is fixed during gradient computation, and the derivative is not claimed to be continuous at the threshold. With search alone, $h_{q1} = z_q^N$ and the formula preserves $u_q^N = z_q^N$. Without search, $u_q^N = u_q^I$. The

convex combination preserves $\left\|u_q^N\right\| \leq 1$ and can change the direction even when $\omega_{q1} = 1$. When $\bar{g}_N = 0$, search affects intent weights and the baseline source; for search-only history, the fallback rule $h_{q1} = z_q^N$ still applies.

Illustrative example: for orthogonal directions with equal initial weights, a search vector aligned with the first, with $\beta_S = \log 3$ and zero age, yields weights $3/4$ and $1/4$.

22.5 Candidate retrieval and ranking features

For $\tau_N > 0$, the neural score is

$$\begin{aligned} s_{qi}^N &= \frac{1 - g_q^{SN}}{\tau_N} \sum_{k \in \mathcal{K}_q} \omega_{qk} \langle h_{qk}, e_i \rangle + \frac{g_q^{SN}}{\tau_N} \langle z_q^N, e_i \rangle \\ &= \frac{\langle u_q^N, e_i \rangle}{\tau_N}. \end{aligned} \tag{121}$$

Temperature τ_N changes the loss scale while preserving item order. The norm constraints imply $|s_{qi}^N| \leq 1/\tau_N$.

The primary new source returns the first m_N^{mix} items by s_{qi}^N among $i \in \mathcal{I}_q$ with $m_i^N = 1$. The item-vector matrix is fixed after training. For practically zero u_q^N, the source returns an empty list with a mask, avoiding arbitrary selection among equal zero scores.

An additional source selects up to m_{Nk} items by $\langle h_{qk}, e_i \rangle$ for each active intent. This preserves items associated with a direction of small ω_{qk}. Lists are treated as separate sources but use one matrix e_i: the mixture u_q^N and all h_{qk} need only different queries to the same shared index. A separate index per intent is unnecessary. The total neural quota m_N and baseline quotas satisfy

$$m_N^{\text{mix}} + \sum_{k=1}^{J_I} m_{Nk} \leq m_N, \qquad m_N + \sum_{a \in \mathcal{A}_{\text{base}}} m_a \leq M. \tag{122}$$

Inactive directions and directions with norm no greater than ε_N have empty lists during candidate retrieval. The overall budget is not increased; deduplication, popularity backfill, and eligibility follow Section 9.

For every candidate in the union, the ranker additionally receives s_{qi}^N, $\max_{k \in \mathcal{K}_q} \langle h_{qk}, e_i \rangle$, similarity $\langle z_q^N, e_i \rangle$ with mask $m_q^{SN} m_i^N$, coefficient g_q^{SN}, the number of directions with norm above ε_N, and the following two statistics:

$$\begin{aligned} H_q^I &= -\sum_{k \in \mathcal{K}_q} \exp(\ell_{qk}^I) \ell_{qk}^I, \\ V_{qi}^I &= \sum_{k \in \mathcal{K}_q} \omega_{qk} \left(\frac{\langle h_{qk}, e_i \rangle}{\tau_N} - \frac{\langle u_q^I, e_i \rangle}{\tau_N} \right)^2. \end{aligned} \tag{123}$$

Entropy describes how distributed the weights are; variance describes differences in item scores. Its center u_q^I excludes the direct search contribution, which is not an intent in the mixture. Without usable vectors, values are zero with masks. Provenance and rank features for the new source are retained separately. The final $f(x_{qi})$ is retrained on B for every architecture being compared. The neural score is not called a conversion probability.

22.6 Primary training objective: recorded future purchases

Let $\mathcal{Q}_N$ contain the first requests from selected sessions $\mathcal{S}^{\text{obs}}_{A_N}$. The primary loss uses only sessions $\mathcal{S}^{+}_{A_N}$ with a recorded future purchase; the others are available to the auxiliary tasks below. For a request, define $Y_q = Y^{\text{obs}}_{s(q)}$, $Y^G_q = Y_q \cap \mathcal{I}_q$, and $a^{\text{buy}}_q = \mathbf{1}\{|Y^G_q| > 0\}$. When $a^{\text{buy}}_q = 1$, the target distribution is $t^{\text{buy}}_{qi} = \mathbf{1}\{i \in Y^G_q\}/|Y^G_q|$. Distinct items in recorded purchases receive equal shares regardless of unknown quantity and price. The fixed training-input mask is $m^N_q = \mathbf{1}\{|\mathcal{D}^N_q| > 0 \text{ or } M^S_q = 1\}\mathbf{1}\{|\mathcal{I}_q| > 0\}$. Set $\mathcal{D}^N_q$ and the mask depend on input data, not on learned norms. With empty $\mathcal{K}_q$ and no search, $u^N_q = 0$. For any nonempty $\mathcal{I}_q$, the normalized model is defined as

$$p_N(i \mid q) = \frac{\exp(s^N_{qi})}{\sum_{j \in \mathcal{I}_q} \exp(s^N_{qj})}. \tag{124}$$

The exact denominator includes the entire nonempty request catalog, including zero item representations. When $u^N_q = 0$, the distribution is uniform and the example is retained. Requests without available input are excluded before training; their fraction is recorded.

Each session contributes one request, so $w^N_q = 1$; additionally, set $a^{\text{buy}}_q = 0$ when $m^N_q = 0$. The primary loss is

$$\mathcal{L}^N_{\text{buy}} = -\frac{\sum_{q \in \mathcal{Q}_N} w^N_q a^{\text{buy}}_q \sum_{i \in \mathcal{I}_q} t^{\text{buy}}_{qi} \log p_N(i \mid q)}{\sum_{q \in \mathcal{Q}_N} w^N_q a^{\text{buy}}_q}. \tag{125}$$

A request term with $a^{\text{buy}}_q = 0$ is zero without evaluating undefined t^{buy}_{qi}. If the denominator is zero, purchase training is impossible; the variant receives an insufficient-data status. The loss learns to select an item from a recorded purchase, not to estimate purchase probability or causal effect. Sessions without a recorded target are available to auxiliary tasks but do not enter Hit@20 or become confirmed negative outcomes. Purchases outside $\mathcal{I}_G$ do not expand the catalog; their fraction is recorded, and they remain possible misses during evaluation.

For one request, the score derivative has a simple form:

$$\frac{\partial\left[-\sum_i t_{qi} \log p_N(i \mid q)\right]}{\partial s^N_{qj}} = p_N(j \mid q) - t_{qj}. \tag{126}$$

Parameter derivatives are computed by the chain rule. For stability, the maximum score is subtracted; the denominator may be computed in blocks. Item sampling and approximate-loss bias are specified in Appendix A.

22.7 Auxiliary next-view prediction

Let t^P_q be the time of the first recorded purchase after t_q in the same session ($+\infty$ if no record exists). The target is the next recorded view, with no claim that actions between records are complete. The admissible time set is

$$\mathcal{T}^+_q = \{t_e : e \in s(q),\ \kappa_e = \text{detail},\ t_q < t_e \le \tau_s,\ t_e < t^P_q\}. \tag{127}$$

If $\mathcal{T}^+_q \neq \varnothing$, set $t^+_q = \min \mathcal{T}^+_q$; items in the entire group at that time form D^+_q. The last view and a view exactly at the horizon are allowed; a view simultaneous with a purchase is not. For empty $\mathcal{T}^+_q$, set $D^+_q = \varnothing$ without computing t^+_q. After intersection with $\mathcal{I}_q$, introduce mask a^{next}_q for a

nonempty target with $m_q^N = 1$, and the uniform distribution t_{qi}^{next} over that target. Loss $\mathcal{L}_{\text{next}}^N$ is defined by formula (125), replacing $(a^{\text{buy}}, t^{\text{buy}})$ with $(a^{\text{next}}, t^{\text{next}})$. With an empty denominator, this auxiliary loss is zero with a flag indicating no training examples. Weight $\lambda_{\text{next}} \geq 0$ is initially fixed, then selected on C_{select} by Hit@20, including the zero-weight variant. Next-view accuracy does not replace this criterion.

22.8 Contrastive alignment of two representations

Two representations of one available prefix are constructed by independently disabling available branches id, G, T, V with specified probabilities. Each item retains at least one originally available branch; if all are disabled, the first of them is restored in a fixed order. Disabling changes the masks in (116), after which the session computation is repeated. Events, their times, and the search vector are preserved. Original texts and images are not required.

Denote the results of n_ε normalization by $\tilde{u}_q^{(1)}, \tilde{u}_q^{(2)}$. Identical masks do not create a pair; a zero learned vector is retained in the loss with zero scores. Each batch contains at most one request per session. Signature $\mathsf{sig}(q)$ contains only encoder inputs: items, relative times, frequencies, and supplied vectors. In request batch $\mathcal{B}_N$, define the comparison set

$$\mathcal{V}_q = \{v \in \mathcal{B}_N : s(v) \neq s(q), \quad \mathsf{sig}(v) \neq \mathsf{sig}(q), \quad Y_v^G \cap Y_q^G = \varnothing\}. \tag{128}$$

This set includes only requests with differing random-transformation masks; duplicate signatures among comparisons are retained once in a fixed order. Labels from A_N are used only in the loss, excluding matching purchases from contrasted examples. Different purchases do not establish different interests.

For $\mathcal{V}_q \neq \varnothing$ and $\tau_C > 0$, define

$$\ell_q^{1\to 2} = -\log \frac{\exp(\langle \tilde{u}_q^{(1)}, \tilde{u}_q^{(2)} \rangle / \tau_C)}{\exp(\langle \tilde{u}_q^{(1)}, \tilde{u}_q^{(2)} \rangle / \tau_C) + \sum_{v \in \mathcal{V}_q} \exp(\langle \tilde{u}_q^{(1)}, \tilde{u}_v^{(2)} \rangle / \tau_C)}. \tag{129}$$

Loss $\mathcal{L}_C^N$ is the weighted mean of $(\ell_q^{1\to 2} + \ell_q^{2\to 1})/2$ over eligible requests with weights w_q^N normalized by their sum. Without eligible pairs, the loss is zero, with a note in the report. With random batches, the expected loss is minimized; batch composition and size are specified by the configuration.

Reproducible selection of contrastive comparisons. Rule `contrastive_sampler=label_filtered_v1` is part of the training run manifest; it is not a learned parameter. Batch order, random masks, and loss normalization are specified in Section 22.10. After forming a batch, only the filters in (128) are applied; masks and counts of excluded comparisons by reason are retained.

In a preregistered comparison, `contrastive_sampler=label_free_v1` removes only the condition $Y_v^G \cap Y_q^G = \varnothing$, preserving all other rules and the budget. This measures the effect of using targets to select comparisons. A request without a recorded purchase can serve as a comparison even without this filter; absence of a record does not confirm different interests. These rules apply only to training period A_N; candidates and inputs in B, C, D do not use future purchases when constructing representations.

The representation is assumed to tolerate partial modality loss. This may fail when a distinguishing feature is lost, so a variant without the contrastive component is required. Sessions in the denominator are not confirmed negative exposures; preserving meaning when vectors are disabled and increasing conversion are not guaranteed.

22.9 Regularization and final training objective

To limit deviations of graph weights from their historical reference, introduce

$$\mathcal{L}_G^N = \frac{1}{|\mathcal{I}_G^P|} \sum_{i \in \mathcal{I}_G^P} \sum_{j \in \mathcal{N}_i^P} \exp(\ell_{ij}^A)(\ell_{ij}^A - \ell_{ij}^P), \qquad \mathcal{I}_G^P = \{i : \mathcal{N}_i^P \neq \varnothing\}. \tag{130}$$

For empty $\mathcal{I}_G^P$, set $\mathcal{L}_G^N = 0$. On shared positive support, the penalty is nonnegative and vanishes when rows match. It limits changes to historical links without establishing causality.

The final objective is

$$\min_{\theta_N} \quad \mathcal{L}_{\text{buy}}^N + \lambda_{\text{next}} \mathcal{L}_{\text{next}}^N + \lambda_C \mathcal{L}_C^N + \lambda_G^N \mathcal{L}_G^N + \lambda_\theta \left\| \theta_N \right\|^2, \qquad \lambda_{\text{next}}, \lambda_C, \lambda_G^N, \lambda_\theta \geq 0. \tag{131}$$

The primary loss has weight one. Other weights do not represent the monetary value of events: they are initially specified in Appendix H, then selected on C_{select} under the comparison plan. Zero weight disables a task. The squared norm of θ_N is the sum of squared scalar parameters; dimensions are fixed before training.

22.10 Reproducible training recipe for extensions

Profile `neural_training_v2` applies to enabled neural, sequential, and SID modules. Reference environment: NumPy 2.1.3, PyTorch 2.6.0, CPU, FP32 for parameters, losses, and optimizer state; one computation thread and one inter-op thread, with deterministic algorithms. AMP, graph compilation, gradient accumulation, learning-rate scheduling, and early stopping are disabled. Exact builds, hardware, and dependency hashes are pinned in the version file under Section H.4; bitwise agreement across different environments is not assumed.

Initialization and randomness. Use `Generator(PCG64(seed))`. For module $m = 0, 1, 2$ (main neural model, sequential model, SID), separate streams have seed $17 + 100m + j$: $j = 0$ for initialization, $j = 1$ for example order, $j = 2$ for branch masks, $j = 3$ for SID noise, $j = 4$ for negative items, $j = 5$ for search dropout, and $j = 6$ for random control SID codes. A disabled consumer makes no calls; states are not reset between epochs. Call order is specified in the corresponding sections. The parameter schema contains names, shapes, roles, and shared references: tensors are traversed by UTF-8 name, and coordinates in row-major order; the shared text/search matrix is initialized once. Random values are generated in FP64 by a `uniform` call for the whole tensor, then cast once to FP32.

A linear-transformation matrix of shape $d_{\text{out}} \times d_{\text{in}}$ receives independent Uniform$[-a, a]$ values, $a = \sqrt{6/(d_{\text{in}} + d_{\text{out}})}$. Identifier vectors b_i^N, intent vectors c_k^N, and codebook vectors $c_{\ell b}^{\text{SID}}$ receive Uniform$[-\sqrt{3/d_{\text{emb}}}, \sqrt{3/d_{\text{emb}}}]$, where d_{emb} is their dimension; rows are ordered by item ID or numeric index. Sequential-model positions p_b receive Uniform$[-0.02, 0.02]$. All other vectors and scalars are zero, except LN scale factors γ, which are one, and b_g from (202). Matrix exception: $U_N = 0$, so initially $A_{ij}^N = \nu_{ij}^P$. The same rules apply to the sequential model's own parameters; blocks absent from its architecture are not created. These rules also specify $b_S = 0$, decoder biases, and time/frequency coefficients; intents do not use identical zero initialization.

Batches and loss scaling. Plans for all epochs are constructed before training. For each epoch, sessions sorted by UTF-8 are shuffled using `permutation`; their first requests are split into batches

of size 64. Every eligible request appears exactly once per epoch; the short final batch is retained. The batch plan and batch count T_{ep} are stored before training. For contrastive masks, iterate over requests in batch order, views $1, 2$, unique input-prefix items by UTF-8 ID, and branches id, G, T, V. Each branch receives one `random()` call, including absent branches; an available branch is disabled when the value is below the specified probability. Restoration after complete dropout follows Section 22.8.

Let ℓ_q^r be the request term in the full loss for task $r \in \{\text{buy}, \text{next}\}$, already including w_q^N, the mask, and the global denominator in (125). On batch $\mathcal{B}$, minimize

$$\mathcal{L}_{\mathcal{B}} = T_{\text{ep}} \sum_{q \in \mathcal{B}} (\ell_q^{\text{buy}} + \lambda_{\text{next}} \ell_q^{\text{next}}) + \lambda_C \mathcal{L}_{C,\mathcal{B}}^N + \lambda_G^N \mathcal{L}_G^N + \lambda_\theta \left\| \theta_N \right\|^2 . \tag{132}$$

Global denominators are not recomputed per batch. At fixed parameters, the mean of the first terms over all batches equals the full purchase and auxiliary loss, including short batches. Contrastive loss is normalized within each batch according to its definition; the graph penalty is computed over the entire fixed graph at every step. This exact cost is included in the training budget.

Parameter updates. Use `torch.optim.Adam` with one group containing all unfrozen parameters, zero initial moments, and settings

```
lr=0.001; betas=(0.9,0.999); eps=1e-8; weight_decay=0
amsgrad=false; foreach=false; fused=false; maximize=false
capturable=false; differentiable=false
```

Each step computes the full gradient of $\mathcal{L}_{\mathcal{B}}$, including the explicit L2 penalty; gradients are dense, with zero for absent contributions. Its global Euclidean norm is computed in FP64. If the norm exceeds 5, the entire gradient is multiplied by $5/\left\|g\right\|$ before updating Adam moments; for zero norm, no division is performed. L2 is not added again through the optimizer. Nonfinite losses, gradients, or parameters abort training. Exactly one Adam step is performed after each batch; its moments and step counter continue across epochs. The main and sequential models are trained for three epochs. A checkpoint stores parameters, moments, step number, batch plan, and all generator states.

22.11 Checking intent distinctness

Several directions may describe one interest. Their coincidence is not ruled out by the loss. Let $\mathcal{K}_q^+ = \{k \in \mathcal{K}_q : \left\|h_{qk}\right\| > \varepsilon_N\}$, $n_q^I = |\mathcal{K}_q^+|$, and $\widehat{h}_{qk} = h_{qk}/\left\|h_{qk}\right\|$ for these directions. For $n_q^I \geq 2$ and prespecified $0 < \delta_I < 1$, define

$$\text{Sim}_q^I = \frac{2}{n_q^I(n_q^I - 1)} \sum_{\substack{k<l \\ k,l \in \mathcal{K}_q^+}} \langle \widehat{h}_{qk}, \widehat{h}_{ql} \rangle, \tag{133}$$

$$\text{Dup}_q^I = \frac{2}{n_q^I(n_q^I - 1)} \sum_{\substack{k<l \\ k,l \in \mathcal{K}_q^+}} \mathbf{1}\{\langle \widehat{h}_{qk}, \widehat{h}_{ql} \rangle \geq 1 - \delta_I\}.$$

The first metric is mean similarity, the second the fraction of nearly coincident pairs. They are undefined when $n_q^I < 2$; report the fraction of such requests, norms, and the frequency of zero directions so that zeroing cannot hide degeneration.

Let $\mathcal{C}^I_{qk}$ be the candidates of a single intent. For pairs with a nonempty union, measure

$$O^I_{qkl} = \frac{|\mathcal{C}^I_{qk} \cap \mathcal{C}^I_{ql}|}{|\mathcal{C}^I_{qk} \cup \mathcal{C}^I_{ql}|}. \tag{134}$$

For a request, average O^I_{qkl} over eligible pairs $k < l$; if no such pairs exist, the result is undefined. Metrics are averaged over first requests on which they are defined. Missing-value rates use all first requests; missing values are not replaced by zeros. Separately compare $\text{Coverage}_{\text{cand}}$ and final Hit@20 for $J_I = 1$ and $J_I = 2$ with the same total neural-source quota. Each variant's ranker is retrained. A high H^I_q is possible even for identical directions.

Breakdowns use history length and search presence. The penalty $\sum_{k \neq l} \langle c^N_k, c^N_l \rangle^2$ is not part of the initial objective: it is also minimized when all $c^N_k = 0$, without ensuring distinct intents. Added complexity is justified by measured direction overlap without additional coverage, not by the number of viewed items alone.

22.12 Search-signal robustness

Limit $\bar{g}_N$ bounds the direct contribution of z^N_q. It does not bound the entire search influence: parameter β_S in intent weights remains learned and has no upper bound. Distributions of g^{SN}_q, $\max_k \omega_{qk}$, H^I_q, and purchase hit rate by query age are retained. The maximum is taken over $\mathcal{K}_q$; for an empty set, it is undefined and accompanied by a missingness mask. To compare the query with history, use the vector without search mixing

$$v^{\text{hist}}_q = \mathsf{n}_\varepsilon \left(\sum_{\ell \in \mathcal{D}^N_q} \alpha_{q\ell} e_{i_{s(q)\ell}} \right). \tag{135}$$

The cosine between z^N_q and v^{hist}_q is computed only when both norms exceed ε_N. Disagreement may indicate a change of interest. This diagnostic uses historical data; search after A/B assignment does not define a subgroup for a confirmatory causal comparison.

If diagnostics show sensitivity to search, a separate variant with random dropout of this signal during training is allowed. For every prefix and step, independently draw $D^S_q \sim \text{Bernoulli}(1-p_S)$, $0 \leq p_S < 1$, and use

$$\widetilde{M}^S_q = D^S_q M^S_q \tag{136}$$

in all search branches of the new encoder: projection and its mask, intent weights, direct contribution, and the search-only fallback representation. A disabled vector is replaced with zero. The same D^S_q applies to both contrastive views of this prefix; baseline sources and target labels are unchanged. For $p_S > 0$, stream $j = 5$ is called once through `random()` for every request in batch order, including requests without search; $D^S_q = \mathbf{1}\{U \geq p_S\}$. No calls occur when $p_S = 0$ or during serving.

Admission m^N_q and the training-weight denominator are computed before random dropout. If no input remains afterward, $u^N_q = 0$ and the example is retained with a uniform probability distribution over items. The expectation of the selected loss over this randomness is minimized. The initial configuration has $p_S = 0$; a nonzero value is selected on C_{select} only in a separate comparison. During serving, the signal is not disabled. This robustness check does not prescribe suppressing search when interest changes and does not guarantee diversity or increased hit rate.

22.13 Cost and failure behavior

Let $N_G = |\mathcal{I}_G|$, and $d_0 \le |\mathcal{R}_G| d_G$ be the maximum out-degree of the combined baseline graph. Stored paths require $O(N_G d_P)$ memory. Bounded search expands at most $N_G B_P$ states and examines at most $N_G B_P d_0$ outgoing edges; queue cost depends on the implementation. Before truncation, exact search can be substantially more expensive in the worst case. The fraction of vertices with truncation is reported alongside candidate quality.

For one full recomputation of item representations, projections require on the order of $N_G[d(d_t + d_v) + d^2]$ operations, and graph aggregation after pretransforming vectors requires on the order of $N_G d_P d$. Once the model is fixed, the result is stored in an $N_G \times d$ matrix and an index; the graph neural network is not recomputed during serving. Session computation requires on the order of $L_H d^2 + J_I L_H d + d d_t$ operations if the current search query must be projected again. An exact catalog scan requires $O(N_G d)$; approximate search is validated under the same exact-search agreement protocol as the baseline content sources.

With a full denominator, neural-training cost for N_Q^N requests per epoch includes $O(N_Q^N N_G d)$; blockwise computation reduces memory, not the number of products. Additional costs are graph-representation and projection recomputation at every parameter update and contrastive pairs. Cached vectors cannot replace representations from the current encoder. Time, memory, and epoch limits are fixed before training; exceeding them is a computational failure.

New-source failures are handled only under the common contracts in 16.7 and 17.4. Unregistered degradation activates $\mathcal{C}_q^{\text{fallback}}$ with its fixed order, without the primary model's ranker.

Invariant 22.1 (Fixed knowledge and available prefix)**.** With θ_N and A_G artifacts fixed, changing events after t_q does not change h_{qk}, ω_{qk}, g_q^{SN}, u_q^I, u_q^N, or s_{qi}^N. Permuting rows within one time group does not change the result. Disabling all new sources restores the baseline policy when its original quotas and ranker are restored.

Part III

Appendices: computational variants and additional studies

A Approximate training with item sampling

This is an optional computational mode of the neural extension; the baseline system does not use it. The reference is the full denominator in (124). All positive items are included exactly; the rest use weighted random sampling [11]. Bias requires validation [9]; MIDX is not reproduced. Agreement in gradients and quality with the exact mode is not assumed. NCE, which solves a different task of distinguishing observations from noise, is not used.

A.1 Positive items and sampling distribution

Consider one eligible request q from A_N and one task $a \in \{\text{buy}, \text{next}\}$ with a nonempty target. Denote its item set by T_q^a: $T_q^{\text{buy}} = Y_q \cap \mathcal{I}_q$, $T_q^{\text{next}} = D_q^+ \cap \mathcal{I}_q$. Eligibility masks and request weights are preserved from Sections 22.6–22.7. On T_q^a, the original uniform distribution t_{qi}^a is defined. Complement $\mathcal{N}_q^a = \mathcal{I}_q \setminus T_q^a$ contains negative items for the training task; absence from the target does not establish that an item was viewed and rejected.

Let $N_G = |\mathcal{I}_G| > 0$, and let π_j^{pop} be the popularity distribution from (32), built only from A_G. For fixed $\delta \in (0, 1]$, define

$$Q_G(j) = (1 - \delta)\pi_j^{\text{pop}} + \frac{\delta}{N_G}, \qquad j \in \mathcal{I}_G. \tag{137}$$

The uniform component gives every catalog item positive probability. If $\mathcal{N}_q^a \neq \varnothing$, the conditional distribution for this request is

$$Q_q^a(j) = \frac{Q_G(j)}{\sum_{v \in \mathcal{N}_q^a} Q_G(v)}, \qquad j \in \mathcal{N}_q^a. \tag{138}$$

Allowed mass $b_q^a = \sum_{j \in \mathcal{N}_q^a} Q_G(j)$ is computed at the start of each enabled task with a nonempty complement by direct compensated Kahan summation in FP64, in UTF-8 ID order; subtracting excluded-item mass from one is not used. In the mathematical model, sampling from Q_G and rejecting forbidden items gives Q_q^a and requires on average m/b_q^a attempts for m items; small mass reduces the benefit and is recorded. Distribution Q_G is independent of learned weights and fixed from A_G. Labels exclude targets only during training on A_N.

In ideal sampling, independently draw $m \geq 1$ items $J_1, \ldots, J_m$ with replacement from Q_q^a. Denote the number of times item j is drawn by $C_{qj}^a = \sum_{b=1}^m \mathbf{1}\{J_b = j\}$. Duplicates retain their multiplicity; otherwise, the sum's expectation changes. The sample is refreshed each time a request is presented; stream $j = 4$ from profile 22.10 is used. In batch order, process the purchase task and then the next-view task for each request. Cumulative sums of Q_G in UTF-8 ID order are built once in FP64, divided by the final sum, and stored as F with final element 1.

Before sampled-mode training, verify that the computed $Q_G(j)$ are finite and positive and that $0 = F_0 < F_1 < \cdots < F_{N_G} = 1$. With items indexed in this order, the grid $U \in \{0, 1, \ldots, 2^{53} - 1\}/2^{53}$ from an FP64 `random()` call corresponds to masses

$$\rho_j = 2^{-53}\left(\lceil 2^{53} F_j \rceil - \lceil 2^{53} F_{j-1} \rceil\right).$$

Every item must satisfy $\rho_j > 0$ and $|\rho_j - Q_G(j)| \le 10^{-6} Q_G(j)$; this is a project tolerance, and the comparison uses $Q_G(j)$ computed in FP64. Violation of these conditions, or nonfinite or nonpositive b_q^a for a nonempty complement, causes failure `negative_sampling_precision_unsupported` before consuming the sampling stream or updating parameters. No item is removed from support, and probabilities are not silently set to zero. This check does not establish exact sampling from real-valued Q_G: the expectation identities and limiting statements below concern ideal sampling; the implementation is its numerical approximation with the stated tolerance for base probabilities.

Each attempt makes one `random()` call and `searchsorted(F,U,side="right")`; forbidden items are rejected until m admissible results are obtained. Probabilities are computed in FP64. Disabled tasks, empty targets, empty complements, and exact mode do not consume this stream. The sample is not reduced when the training budget is exhausted: the run fails.

A.2 Denominator estimation and derivatives

For brevity, set $s_i = s_{qi}^N$. The exact denominator and its random estimate are

$$Z_q = \sum_{i \in T_q^a} e^{s_i} + \sum_{j \in \mathcal{N}_q^a} e^{s_j}, \tag{139}$$

$$\widehat{Z}_q^a = \sum_{i \in T_q^a} e^{s_i} + \frac{1}{m} \sum_{b=1}^{m} \frac{e^{s_{J_b}}}{Q_q^a(J_b)}.$$

Each positive target appears once, even when there are more than m. If $\mathcal{N}_q^a = \varnothing$, no sampling occurs: $\widehat{Z}_q^a = \sum_{i \in T_q^a} e^{s_i} = Z_q$. For empty T_q^a, the original zero task mask applies; no positive target is created artificially.

For a fixed model,

$$\mathbb{E}\left[\frac{e^{s_{J_b}}}{Q_q^a(J_b)}\right] = \sum_{j \in \mathcal{N}_q^a} e^{s_j}, \qquad \mathbb{E}[\widehat{Z}_q^a] = Z_q. \tag{140}$$

The approximate loss for one request is

$$\widehat{\ell}_q^a = \log \widehat{Z}_q^a - \sum_{i \in T_q^a} t_{qi}^a s_i. \tag{141}$$

All positive targets participate in the average.

Set $\kappa_i = 1$ for $i \in T_q^a$ and $\kappa_j = C_{qj}^a / (m Q_q^a(j))$ for $j \in \mathcal{N}_q^a$. For a fixed sample,

$$\widehat{p}_i = \frac{\kappa_i e^{s_i}}{\widehat{Z}_q^a}, \qquad \frac{\partial \widehat{\ell}_q^a}{\partial s_i} = \widehat{p}_i - t_{qi}^a. \tag{142}$$

Outside T_q^a, set $t_{qi}^a = 0$. An unsampled negative item has $\kappa_i = 0$; $\sum_i \widehat{p}_i = 1$, but this is the distribution of the training computation, not the full catalog. Parameter derivatives follow the chain rule with Q_q^a and sample indices fixed.

To compute $\log \widehat{Z}_q^a$ stably, use scores $s_i + \log \kappa_i$ for $\kappa_i > 0$, subtracting the maximum. The expanded correction is

$$\widetilde{s}_i = \begin{cases} s_i, & i \in T_q^a, \\ s_i + \log C_{qi}^a - \log m - \log Q_q^a(i), & i \in \mathcal{N}_q^a,\ C_{qi}^a > 0. \end{cases} \tag{143}$$

For this scheme, $s_i - \log Q(i)$ is insufficient: it ignores multiplicity, sample size, and exact inclusion of positive targets. The different corrections for mandatory targets and sampled items matter [10]. Corrections apply only to the training denominator; candidate retrieval uses the original s^N_{qi}.

Unbiasedness in (140) does not imply an unbiased loss. Concavity of the logarithm gives

$$\begin{aligned} \mathbb{E}[\log \widehat{Z}^a_q] &\le \log Z_q, \\ \mathbb{E}[\nabla_\theta \widehat{\ell}^a_q] &\ne \nabla_\theta \ell^a_q \quad \text{in general.} \end{aligned} \tag{144}$$

Here $\theta = \theta_N$, and $\ell^a_q = \log Z_q - \sum_i t^a_{qi} s_i$. Exact and approximate losses cannot be compared directly when selecting an architecture. For fixed parameters, a finite catalog, and positive Q^a_q, the law of large numbers gives $\widehat{Z}^a_q \to Z_q$ as $m \to \infty$. The loss and its gradient at differentiable points converge as well. Convergence at a fixed model does not guarantee identical results from the two training processes.

A.3 Training integration and cost

Loss $\widehat{\mathcal{L}}^N_a$ is obtained by replacing the single-request loss in (125) with $\widehat{\ell}^a_q$, preserving w^N_q and a^a_q. In (131), only $\mathcal{L}^N_{\text{buy}}$ and, when used, $\mathcal{L}^N_{\text{next}}$ are replaced. Samples for these two tasks are constructed using their own targets. Contrastive loss, graph regularization, and the parameter penalty are unchanged. The approximation does not apply to the separate full-catalog SID loss.

The denominator requires at most $|T^a_q| + m$ dot products per request. Direct computation of the allowed mass additionally requires $O(|\mathcal{N}^a_q|)$ scalar operations per enabled task. History, positive and sampled items, and their graph neighbors are also recomputed using current parameters; stale vectors are not substituted. The graph regularizer over all rows and other shared components retain their full cost.

For the first approximate run, use $\delta = 0.1$ and $m = 512$; $m \in \{1024, 2048\}$ are tested if the selection budget on C_{select} allows it. Any m denotes sampling with replacement; full enumeration is exact mode. Comparing them on a small shared catalog and identical sessions checks computation and the cost of approximation, without guaranteeing transfer to the full catalog.

Modes follow protocol 22.1 with identical catalog, quotas, and evaluation labels; Hit@20 and $\text{Coverage}_{\text{cand}}$ are computed without negative-item sampling. The report includes epoch and total training time, update count, peak memory, duplicate rate in the sample, and purchase hit rate. Time savings are assessed together with the change in hit rate.

Hard negative items are allowed only as a separate hypothesis. Applying (139) requires the known actual sampling probability of every item and positive support over all of $\mathcal{N}^a_q$, for example through a uniform component. During gradient computation, the distribution is fixed; dependence on parameters and a different differentiation method require a separate derivation. Arbitrarily adding items retrieved by an index while retaining the old Q^a_q invalidates the justification of the denominator estimate.

A.4 Computational checks

Normative examples, derivatives, and boundary cases are collected in X02 (Section K.4).

B Sequential model for a controlled comparison

After the neural stage, weighted history aggregation is compared with a single causal attention block inspired by SASRec [8]. This compact adaptation replaces the session encoder while retaining the data, purchase target, catalog, and ranker type. The item graph remains in both models; its contribution is tested by separately disabling the graph branch.

B.1 Identical inputs and groups of simultaneous events

The model uses the history $\mathcal{D}_q^N$ of available `detail` events from Section 22.4, with the same L_H, handling of unknown identifiers, and assumption about static vectors. Sequences of cart, search, and page-view actions require a separate comparison: they change both the architecture and the available inputs. The ranker retains the cart features.

The item encoder is defined by (113)–(116), but is trained from scratch on A_N jointly with the sequential encoder. Denote its output by e_i^{seq}. The graph, paths, and catalog from A_G are retained; the trained parameters of the main model are not transferred. The search projection z_q^{seq} is obtained from (117) with its own trainable matrix T_{seq}, shared by the text branch and search.

Partition $\mathcal{D}_q^N$ into nonempty timestamp groups $\mathcal{G}_{q1}, \dots, \mathcal{G}_{qT_q}$ with times $t_{q1} < \dots < t_{qT_q} \le t_q$, where $T_q \le L_H$. Index b denotes a group, not a row of the source file. Define its input vector:

$$
\begin{aligned}
n_{qb,i}^{\text{detail}} &= \sum_{\ell:\, e_{s(q)\ell} \in \mathcal{H}_q,\ t_{s(q)\ell} \le t_{qb}} \mathbf{1}\{i_{s(q)\ell} = i,\ \kappa_{s(q)\ell} = \text{detail}\}, \qquad (145)\\
\bar{e}_{qb} &= \frac{1}{|\mathcal{G}_{qb}|} \sum_{\ell \in \mathcal{G}_{qb}} e^{\text{seq}}_{i_{s(q)\ell}},\\
f_{qb} &= \frac{1}{|\mathcal{G}_{qb}|} \sum_{\ell \in \mathcal{G}_{qb}} \log(1 + n^{\text{detail}}_{qb, i_{s(q)\ell}}),\\
x_{qb} &= F_{\text{seq}} \bar{e}_{qb} + p_b + a^t_{\text{seq}} \log\left(1 + \frac{t_q - t_{qb}}{t_{\text{unit}}}\right) + a^f_{\text{seq}} f_{qb} + a^m_{\text{seq}} \log|\mathcal{G}_{qb}|.
\end{aligned}
$$

Here $F_{\text{seq}} \in \mathbb{R}^{d \times d}$; $p_b, a^t_{\text{seq}}, a^f_{\text{seq}}, a^m_{\text{seq}} \in \mathbb{R}^d$ are trained on A_N, and $p_1, \dots, p_{L_H}$ is the group position table. Frequency includes events through the end of the group; age is measured at t_q. The contents of subsequent groups are unavailable. Row order within a group does not affect the input; zero learned vectors do not remove events.

B.2 A single causal attention block

For $v \in \mathbb{R}^d$, define coordinate normalization as

$$
\text{LN}_{\gamma,\beta}(v) = \gamma \odot \frac{v - \bar{v}\mathbf{1}}{\sqrt{d^{-1}\sum_{a=1}^d (v_a - \bar{v})^2 + \varepsilon_{\text{LN}}}} + \beta, \qquad \bar{v} = d^{-1}\sum_{a=1}^d v_a. \qquad (146)
$$

The symbol $\odot$ denotes elementwise multiplication; $\gamma, \beta \in \mathbb{R}^d$ are trainable; the initial $\varepsilon_{\text{LN}} = 10^{-5}$ is included in the full configuration of the sequential variant. The three applications $\text{LN}_1, \text{LN}_2, \text{LN}_3$ below have distinct trainable parameters.

For each existing group, compute the query, key, and value:

$$
\tilde{x}_{qb} = \text{LN}_1(x_{qb}), \qquad Q_{qb} = W_Q \tilde{x}_{qb}, \quad K_{qb} = W_K \tilde{x}_{qb}, \quad V_{qb} = W_V \tilde{x}_{qb}, \qquad (147)
$$

where $W_Q, W_K, W_V \in \mathbb{R}^{d\times d}$. A single attention head uses a causal mask: a group can access itself and preceding groups:

$$a_{qbc} = \frac{\exp(Q_{qb}^\top K_{qc}/\sqrt{d})}{\sum_{v=1}^{b}\exp(Q_{qb}^\top K_{qv}/\sqrt{d})}, \qquad 1 \le c \le b \le T_q, \tag{148}$$
$$a_{qbc} = 0, \qquad c > b.$$

Dummy groups introduced by batch padding are excluded from the computation. Attention to the group itself ensures a nonempty denominator. Residual connections and an elementwise nonlinearity are then applied:

$$y_{qb} = x_{qb} + W_O \sum_{c=1}^{b} a_{qbc} V_{qc}, \tag{149}$$
$$h_{qb}^{\text{seq}} = y_{qb} + W_2\,\mathrm{ReLU}(W_1\,\mathrm{LN}_2(y_{qb}) + b_1) + b_2.$$

Here $W_O \in \mathbb{R}^{d\times d}$, $W_1 \in \mathbb{R}^{d_f\times d}$, $W_2 \in \mathbb{R}^{d\times d_f}$, $b_1 \in \mathbb{R}^{d_f}$, $b_2 \in \mathbb{R}^d$, and $\mathrm{ReLU}(v)_a = \max(0, v_a)$. The starting value $d_f = 2d$ is a computational choice.

The history representation is read at the end of the observed prefix:

$$u_q^{H,\text{seq}} = \mathsf{n}_\varepsilon\Big(\mathrm{LN}_3(h_{qT_q}^{\text{seq}})\Big), \qquad T_q > 0. \tag{150}$$

If $T_q = 0$ and $M_q^S = 1$, set $u_q^{H,\text{seq}} = z_q^{\text{seq}}$; if both history and search are absent, set it to zero. Search is added using the same bounded mixing rule (120), replacing u_q^I with $u_q^{H,\text{seq}}$ and z_q^N with z_q^{seq}. The mixing coefficient is trained from scratch with the same $\bar{g}_N$ and initialization. The result u_q^{seq}, when only search is present, equals z_q^{seq}. Zero norms do not change the fixed mask of the training example.

B.3 Training, candidates, and comparison conditions

For the same $i \in \mathcal{I}_q$, define

$$s_{qi}^{\text{seq}} = \frac{\langle u_q^{\text{seq}}, e_i^{\text{seq}}\rangle}{\tau_N}. \tag{151}$$

Purchase targets, session weights, and auxiliary next views are the same as in Sections 22.6–22.7. Training minimizes the sum (131), with the encoder and its parameters replaced, following the recipe in 22.10. Graph regularization acts on this model's own item encoder. In the first comparison, both architectures use $\lambda_C = 0$, the same normalization method, number of negative items, and training budget. If the contrastive component is subsequently enabled, the example transformations and pair selection rules are also identical. The architectures may have different parameter counts.

After training, the item vectors are fixed and indexed. One query u_q^{seq} to the index returns up to $m_N = 90$ items according to (151) whose e_i^{seq} norm exceeds ε_N. This is the entire neural quota; the baseline sources retain 210 items under the overall limit $M = 300$. If the query norm is at most ε_N, the neural source is empty; deduplication and filling of the remainder follow the main policy. Candidate eligibility and the prohibition on inserting targets follow 9.

The ranker receives the score (151), masked search similarity, the mixing coefficient, and candidate provenance and rank. Multi-intent features are not attributed to this model. Ranker retraining and selection follow I; the architectures are compared using the same criterion and a predefined budget.

B.4 Cost and testable properties

Grouping requires on the order of $L_H d$ operations on precomputed vectors. For one prefix, the block requires $O(T_q d^2 + T_q^2 d + T_q d d_f)$ operations and $O(T_q^2 + T_q d + T_q d_f)$ intermediate memory. With the starting $d_f = 2d$, the latter expression is $O(T_q^2 + T_q d)$. Search projection, item vector reads, index lookup, and final ranking are accounted for separately. The item graph and its paths are not recomputed for each request. Training cost includes the same recomputation of item representations and the same selected method for computing the denominator. Standard key caching across requests is not assumed here: event age depends on t_q, and truncating the history changes positions.

Tests verify that the response is unchanged when rows within one timestamp group are permuted or events after t_q are modified, that attention ignores batch padding, and that the overall budget M is preserved. Permuting items across different times may change the result. Purchase coverage, training time, memory, and measured latency are published.

C Aging of relations and an updated graph

An updated graph is a separate policy that accounts for changes in relations between eligible items. The main comparison retains the A_G snapshot and the temporal protocol in 13.

C.1 Two distinct time scales

Parameter τ_G in (15) describes the distance between actions within one session. Define the aging of recorded sessions using a window $T_{\text{win}} > 0$ and a scale $\tau_{\text{age}} > 0$ in seconds.

Let c_k be the data cutoff for the kth update, and let $r_k > c_k$ be the actual publication time of the completed package. Denote by $\mathcal{S}^{\text{obs}}(c_k)$ the selected historical sessions satisfying $t_{0,s}+H+g < c_k$ and $t^{\text{obs}}_{\text{last},s} + g < c_k$. All published records of a selected session have been read and are available by c_k; a session with a later tail is excluded in its entirety. These conditions do not establish telemetry completeness. The same rule applies to sessions with and without a recorded purchase: a known target does not permit early inclusion.

Retain the pairs from (14) for which both events fall within the calendar window and both items belong to $\mathcal{I}_G$. For $i, j \in \mathcal{I}_G$, define

$$\mathcal{P}^{r,\text{fr}}_{sij}(c_k) = \{(\ell, m) \in \mathcal{P}^{r}_{sij}(c_k) : c_k - T_{\text{win}} \le t_{s\ell} < t_{sm} < c_k\}. \tag{152}$$

For an eligible pair, introduce

$$\xi_{s\ell m}(c_k) = \exp\left(-\frac{t_{sm} - t_{s\ell}}{\tau_G} - \frac{c_k - t_{sm}}{\tau_{\text{age}}}\right). \tag{153}$$

The weights and support are

$$w^{r,\text{fr}}_{sij}(c_k) = \begin{cases} \max\limits_{(\ell,m)\in\mathcal{P}^{r,\text{fr}}_{sij}(c_k)} \xi_{s\ell m}(c_k), & \mathcal{P}^{r,\text{fr}}_{sij}(c_k) \neq \varnothing, \\ 0, & \text{otherwise}, \end{cases} \tag{154}$$

$$W^{r,\text{fr}}_{ij}(c_k) = \sum_{s\in\mathcal{S}^{\text{obs}}(c_k)} w^{r,\text{fr}}_{sij}(c_k),$$

$$n^{r,\text{fr}}_{ij}(c_k) = \sum_{s\in\mathcal{S}^{\text{obs}}(c_k)} \mathbf{1}\{w^{r,\text{fr}}_{sij}(c_k) > 0\}. \tag{155}$$

The bound $0 \le w_{sij}^{r,\mathrm{fr}} \le 1$ is preserved and, consequently, $0 \le W_{ij}^{r,\mathrm{fr}} \le n_{ij}^{r,\mathrm{fr}}$. Support counts distinct sessions in the current window, not all sessions over the lifetime of the graph. Sparsification, threshold n_G, normalization, and row regularization apply to these new quantities. Equality (19) still holds. A larger τ_{age} weakens aging but does not remove the hard window boundary.

C.2 Publishing a consistent version

The catalog $\mathcal{I}_G$ is retained. New items require a separate eligibility rule based on first observation time, parameter initialization, and compatibility checks. A static metadata file does not establish past item availability.

The graph, bounded paths, item vectors, indexes, features, and a compatible ranker are published together. A new graph can change vectors even with unchanged neural weights. The previous package remains in use until the new one passes validation; a request selects a version by publication time $r_k < t_q$, not by data cutoff c_k.

In the first mode, the trained model parameters are fixed. The ranker is trained and validated in advance on earlier periods by replaying the adopted graph update rule. This validation is performed separately from that of the static model.

In the second mode, the required components are retrained on new historical blocks while preserving their roles. The package is published after all stages. Parameters, training targets, and validation results are available before r_k; calendar eligibility of targets retains the $H + g$ margin.

C.3 Comparison protocol and checks

The updating policy is evaluated on a separate subsequent interval D_{roll}. Before it is opened, the update schedule, $T_{\text{win}}, \tau_{\text{age}}$, training mode, publication delay rule, eligible data for each release, and fallback package are fixed. These quantities are selected on earlier intervals. Period D of the main static comparison is not used for their tuning or retraining. Training on early sessions in D_{roll} is allowed only after calendar eligibility is satisfied; the results are unavailable to preceding requests. The evaluation concerns a predefined learning policy.

The static and updating policies are replayed on the same requests from D_{roll} with identical K, M, H, Q_{max}. To separate the effect of decay from the benefit of new history, weights are also compared on the same data window. Published results include Hit@20, $\text{Coverage}_{\text{cand}}$, data age $t_q - c_k$, publication delay $r_k - c_k$, recomputation cost, and the number of missed updates. In an A/B test, training uses a predefined shared external stream or separate data from each group. Mixing these data requires an analysis of interactions between groups.

X01 checks time, weights, and the immutability of the published graph (Section K.4).

D Candidate generator based on semantic codes

A semantic identifier (SID) encodes an item as a sequence of indices into small codebooks. An additional source after the neural stage predicts these indices one at a time. The principles borrowed from DIGER and GRID are listed in 21.

The experiment uses Coveo vectors, positional soft quantization, and a recurrent decoder: each next index depends on the previous ones. It differs from DIGER in using no language encoder,

residual quantization, or adaptive noise adjustment. Code assignments are trained with a separate full-catalog recommendation loss; the graph encoder, ranker, and business criterion are retained.

D.1 Training data and fixed representations

The catalog $\mathcal{I}_G$ and frequencies are fixed using A_G. The neural module trained on A_N is frozen; its $e_i \in \mathbb{R}^d$ and $u_q^N \in \mathbb{R}^d$ become SID inputs. SID is also trained on A_N, which does not serve as a generalization check. Dimensions, epoch counts, and coefficients are selected on C_{select}, without fitting trainable parameters to this period.

Let x_i^{SID} combine normalized description and image vectors with availability masks; a missing modality yields a zero block. Categories and raw images are not required. The fixed input is

$$z_i^{\text{SID}} = [e_i; x_i^{\text{SID}}]. \tag{156}$$

Concatenation $[\,;\,]$ uses static data subject to the limitation in Section 3.

For a request q in the training block, set $Y_q^G = Y_q \cap \mathcal{I}_q \cap \mathcal{I}_G$. The set $\mathcal{Q}_N^{\text{SID}}$ contains the first requests of sessions $\mathcal{S}_{A_N}^+$ with $Y_q^G \neq \varnothing$. Set $w_q^{\text{SID}} = 1$ and $W_N^{\text{SID}} = \sum_{q \in \mathcal{Q}_N^{\text{SID}}} w_q^{\text{SID}}$. Each session has total weight one. If the catalog is empty or $W_N^{\text{SID}} = 0$, training is skipped with status `insufficient`. Requests without recorded purchases do not generate training targets and do not enter Hit@20. Purchases outside $\mathcal{I}_G$ do not expand the catalog and remain possible misses during evaluation.

D.2 Soft assignments and the hard code

Let L_{SID} be the code length, $V_{\text{SID}} \geq 2$ the size of each codebook, and d_{SID} the dimension of its vectors. For position ℓ, train a matrix A_ℓ^{SID}, a bias b_ℓ^{SID}, and vectors $c_{\ell b}^{\text{SID}} \in \mathbb{R}^{d_{\text{SID}}}$, $b = 1, \ldots, V_{\text{SID}}$. The assignment logits are

$$a_{i\ell b}^{\text{SID}} = -\left\| A_\ell^{\text{SID}} z_i^{\text{SID}} + b_\ell^{\text{SID}} - c_{\ell b}^{\text{SID}} \right\|^2. \tag{157}$$

Tied maxima are resolved by the smaller codebook entry index. At training step t, draw independent $U_{i\ell b} \sim \text{Uniform}(0,1)$ and $G_{i\ell b} = -\log(-\log U_{i\ell b})$. For numerical stability, U is clipped to $[\varepsilon_U, 1-\varepsilon_U]$, where $0 < \varepsilon_U < 1/2$. Soft assignments are given by

$$\rho_{i\ell b}^{\text{SID}}(t) = \frac{\exp\{(a_{i\ell b}^{\text{SID}} + \sigma_t^{\text{SID}} G_{i\ell b})/\tau_t^{\text{SID}}\}}{\sum_{v=1}^{V_{\text{SID}}} \exp\{(a_{i\ell v}^{\text{SID}} + \sigma_t^{\text{SID}} G_{i\ell v})/\tau_t^{\text{SID}}\}}. \tag{158}$$

Here $\tau_t^{\text{SID}} > 0$ controls the softness of the distribution, and $\sigma_t^{\text{SID}} \geq 0$ is the noise scale. All soft losses within a step use the same noise for an item. Noise is reproduced using stream $j = 3$ of the profile in 22.10: each joint step traverses the entire catalog in UTF-8 ID order, then ℓ, b, using `random()` and $\varepsilon_U = 10^{-7}$. Noise logarithms are computed in FP64 and the result is cast to FP32. Schedule (205) uses $t = 1, \ldots, T_{\text{SID}}$, where T_{SID} is the number of steps in three joint epochs, computed from the saved batch plan. During pretraining, $\sigma = 0, \tau = 1$; no noise is generated.

The positional vectors and their concatenation are

$$\widetilde{c}_{i\ell}^{\text{SID}} = \sum_b \rho_{i\ell b}^{\text{SID}} c_{\ell b}^{\text{SID}}, \qquad \widetilde{v}_i^{\text{SID}} = [\widetilde{c}_{i1}^{\text{SID}}; \ldots; \widetilde{c}_{iL_{\text{SID}}}^{\text{SID}}]. \tag{159}$$

Concatenation preserves the distinction between positions. The serving index uses only deterministic codes

$$k_{i\ell}^{\text{SID}} = \arg\max_b a_{i\ell b}^{\text{SID}}, \qquad \mathbf{k}_i^{\text{SID}} = (k_{i1}^{\text{SID}}, \ldots, k_{iL_{\text{SID}}}^{\text{SID}}). \tag{160}$$

Noise is disabled during serving and historical evaluation.

D.3 Losses and gradient flow to assignments

The trainable matrix R^{SID} maps the session vector to the space of concatenated codes. For $\tau_R^{\text{SID}} > 0$, define the full-catalog distribution

$$a_{qi}^{R,\text{SID}} = \frac{(R^{\text{SID}} u_q^N)^\top \widetilde{v}_i^{\text{SID}}}{\tau_R^{\text{SID}}}, \tag{161}$$

$$p_{qi}^{R,\text{SID}} = \frac{\exp(a_{qi}^{R,\text{SID}})}{\sum_{j \in \mathcal{I}_q \cap \mathcal{I}_G} \exp(a_{qj}^{R,\text{SID}})}. \tag{162}$$

The future-purchase loss is

$$\mathcal{L}_R^{\text{SID}} = -\frac{1}{W_N^{\text{SID}}} \sum_{q \in \mathcal{Q}_N^{\text{SID}}} \frac{w_q^{\text{SID}}}{|Y_q^G|} \sum_{i \in Y_q^G} \log p_{qi}^{R,\text{SID}}. \tag{163}$$

The remaining items form the normalization catalog; the absence of a purchase record does not imply a negative response to an impression.

For one request and target distribution $\pi_{qi}^{\text{SID}} = \mathbf{1}\{i \in Y_q^G\}/|Y_q^G|$, the derivative with respect to the logit is $p_{qi}^{R,\text{SID}} - \pi_{qi}^{\text{SID}}$. Since

$$\frac{\partial \rho_{i\ell b}^{\text{SID}}}{\partial a_{i\ell v}^{\text{SID}}} = \frac{\rho_{i\ell b}^{\text{SID}}}{\tau_t^{\text{SID}}} \big(\mathbf{1}\{b = v\} - \rho_{i\ell v}^{\text{SID}}\big), \tag{164}$$

there is a derivative chain $\mathcal{L}_R^{\text{SID}} \to \widetilde{v}_i^{\text{SID}} \to \rho_{i\ell b}^{\text{SID}} \to a_{i\ell b}^{\text{SID}} \to A_\ell^{\text{SID}}, c_{\ell b}^{\text{SID}}$. This derivative trains soft assignments but does not differentiate the discrete arg max. An improvement in the soft loss does not guarantee an improvement in hard retrieval.

Linear decoders D_m^{SID}, $m \in \{\text{text}, \text{image}\}$, are used to preserve modality information. Let $N_m = \sum_{i \in \mathcal{I}_G} M_i^m$. Define

$$\mathcal{L}_X^{\text{SID}} = \sum_{m : N_m > 0} \frac{1}{N_m} \sum_{i \in \mathcal{I}_G} M_i^m \left\| D_m^{\text{SID}} \widetilde{v}_i^{\text{SID}} - \bar{v}_i^m \right\|^2. \tag{165}$$

If a modality is entirely absent, its term is zero. The supplied vectors are reconstructed because the original text and images are unavailable.

Average code utilization and the balancing loss are defined as

$$\bar{\rho}_{\ell b}^{\text{SID}} = \frac{1}{|\mathcal{I}_G|} \sum_{i \in \mathcal{I}_G} \rho_{i\ell b}^{\text{SID}}, \qquad \mathcal{L}_B^{\text{SID}} = \frac{1}{L_{\text{SID}}} \sum_{\ell, b} \bar{\rho}_{\ell b}^{\text{SID}} \log(V_{\text{SID}} \bar{\rho}_{\ell b}^{\text{SID}}). \tag{166}$$

For soft assignments, gradient computation retains $r_{\ell b} = \text{LSE}_{i \in \mathcal{I}_G}(\log \rho_{i\ell b}^{\text{SID}}) - \log |\mathcal{I}_G|$, where $\log \rho$ is obtained directly via `logsoftmax`. The balancing term is computed as $\exp(r_{\ell b})(\log V_{\text{SID}} + r_{\ell b})$ following 22.3. In control variants with fixed codes, balance is computed from nonzero integer frequencies; zero cells are skipped, and the balancing-loss gradient is zero. The regularizer does not guarantee diversity of hard codes: uniform soft assignments yield zero loss, yet after tie-breaking the entire catalog may receive one code. Its utilization and collisions are checked under D.6; excessive regularization may destroy useful item groups.

D.4 Autoregressive decoder and training order

The decoder state $h_{q\ell}^{\mathrm{SID}} \in \mathbb{R}^{d_h^{\mathrm{SID}}}$ is initialized from the session vector. For a partially generated code $k_1, \dots, k_{\ell-1}$, define

$$h_{q1}^{\mathrm{SID}} = \tanh(U^{\mathrm{SID}} u_q^N + b_0^{\mathrm{SID}}), \tag{167}$$

$$P_\ell^{\mathrm{SID}}(b \mid q, k_{<\ell}) = \frac{\exp\{(O_\ell^{\mathrm{SID}} h_{q\ell}^{\mathrm{SID}} + o_\ell^{\mathrm{SID}})_b\}}{\sum_v \exp\{(O_\ell^{\mathrm{SID}} h_{q\ell}^{\mathrm{SID}} + o_\ell^{\mathrm{SID}})_v\}}, \tag{168}$$

$$h_{q,\ell+1}^{\mathrm{SID}} = \tanh(F^{\mathrm{SID}} h_{q\ell}^{\mathrm{SID}} + T^{\mathrm{SID}} c_{\ell k_\ell}^{\mathrm{SID}} + p_\ell^{\mathrm{SID}}). \tag{169}$$

The function tanh is applied to each coordinate. All matrices and biases in these formulas are trainable. The probability of a complete sequence is the product of the conditional probabilities of its positions.

At the start of an epoch, hard codes (160) are computed for all items and fixed as the targets for that epoch. When training the next position, the decoder receives the correct previous code. The loss is

$$\mathcal{L}_G^{\mathrm{SID}} = -\frac{1}{W_N^{\mathrm{SID}} L_{\mathrm{SID}}} \sum_{q \in \mathcal{Q}_N^{\mathrm{SID}}} \frac{w_q^{\mathrm{SID}}}{|Y_q^G|} \sum_{i \in Y_q^G} \sum_{\ell=1}^{L_{\mathrm{SID}}} \log P_\ell^{\mathrm{SID}}(k_{i\ell}^{\mathrm{SID}} \mid q, k_{i,<\ell}^{\mathrm{SID}}). \tag{170}$$

Loss (170) does not differentiate target indices; the connection to A_ℓ^{SID} and b_ℓ^{SID} is provided by (163). The gradient through the decoder input vectors changes tables $c_{\ell b}^{\mathrm{SID}}$ and possible future assignments, but is not a derivative with respect to a code index.

The joint objective is

$$\begin{aligned}\mathcal{L}^{\mathrm{SID}} = \mathcal{L}_G^{\mathrm{SID}} &+ \lambda_R^{\mathrm{SID}} \mathcal{L}_R^{\mathrm{SID}} + \lambda_X^{\mathrm{SID}} \mathcal{L}_X^{\mathrm{SID}} \\ &+ \lambda_B^{\mathrm{SID}} \mathcal{L}_B^{\mathrm{SID}} + \lambda_2^{\mathrm{SID}} \left\|\theta_{\mathrm{SID}}\right\|^2,\end{aligned} \tag{171}$$

where $\lambda_R^{\mathrm{SID}} > 0$, the other coefficients are nonnegative, and θ_{SID} contains only this module's parameters. The profile in 22.10 applies to all three phases. In pretraining, only A_ℓ^{SID}, b_ℓ^{SID}, $c_{\ell b}^{\mathrm{SID}}$, and D_m^{SID} are trained; all other SID parameters are frozen. One epoch is run on $\mathcal{L}_X^{\mathrm{SID}} + \lambda_B^{\mathrm{SID}} \mathcal{L}_B^{\mathrm{SID}}$: the catalog, initially sorted by UTF-8 ID, is shuffled by calling `permutation` on the ordering stream; batches contain up to 256 items. The sum of item reconstruction terms with global N_m is multiplied by the number of batches in the epoch; balance is computed over the full catalog. All parameters θ_{SID} are then trained for three joint epochs on (171), in batches of up to 64 requests from distinct sessions. Purchase and generative terms retain global weights and the factor T_{ep}, as in (132); reconstruction, balance, and L2 are computed in full at every step. At the end, the encoder and code tables are fixed, all hard codes are recomputed, and only the decoder is then trained further on (170) using the same A_N and the final targets. In this final phase, only $U^{\mathrm{SID}}, b_0^{\mathrm{SID}}, O_\ell^{\mathrm{SID}}, o_\ell^{\mathrm{SID}}, F^{\mathrm{SID}}, T^{\mathrm{SID}}, p_\ell^{\mathrm{SID}}$ from (167) are trained. It lasts two epochs with the same batching rule and global normalization; the other losses are disabled. At the start of each phase, the Adam moments and step counter are reset, while generator states continue. The lists of trainable and frozen tensors for each phase are saved in the run manifest; the original neural network remains frozen throughout all SID phases. After decoder training, the entire module is frozen before processing B.

D.5 Valid sequences, collisions, and output size

Different items may receive the same code. For a sequence $\mathbf{k}$, define its item bucket

$$\mathcal{B}^{\mathrm{SID}}(\mathbf{k}) = \{i \in \mathcal{I}_G : \mathbf{k}_i^{\mathrm{SID}} = \mathbf{k}\}. \tag{172}$$

In the mapping table, items are sorted by popularity from A_G, then by identifier. The served subset $\mathcal{B}_R^{\text{SID}}(\mathbf{k})$ contains the first R_{SID} items; the excluded fraction is reported separately. An item is uniquely identified by the pair "code, position in the table," since codes may repeat.

A prefix tree contains catalog codes: an edge specifies the next index, and a path to a leaf represents an existing code. The decoder uses only edges of this tree. At depth ℓ, at most B_{SID} paths are retained with the largest sums of original log probabilities from (167). Probabilities are not renormalized after an edge is prohibited, because this would change the path scores being compared. Ties are resolved lexicographically. Bounded beam width does not guarantee the globally best B_{SID} sequences.

Let $\mathcal{K}_q^{\text{SID}}$ be the retrieved leaves. From the set

$$\mathcal{U}_q^{\text{SID}} = \mathcal{I}_q \cap \bigcup_{\mathbf{k} \in \mathcal{K}_q^{\text{SID}}} \mathcal{B}_R^{\text{SID}}(\mathbf{k}) \tag{173}$$

select the first m_{SID} distinct items by score $\sum_\ell \log P_\ell^{\text{SID}}(k_{i\ell}^{\text{SID}} \mid q, k_{i,<\ell}^{\text{SID}})$, then by the same historical popularity and identifier. Consequently, $|\mathcal{C}_q^{(\text{SID})}| \le \min(m_{\text{SID}}, B_{\text{SID}} R_{\text{SID}})$. Quota m_{SID} is included in the existing limit M; the usual sources and the common filling rule are retained.

Additional features may include source ranks, the code log probability, and the mapping table size. Code probability does not estimate purchase probability or the causal effect of an impression. Ranker retraining follows I, candidate constraints follow 9, and failure handling follows 22.13.

The generator is enabled for serving only after measured validation of latency, memory, and cost on the target node. If requirements are violated, the source is excluded; research results are retained separately.

D.6 Diagnostics, comparisons, and computational limits

After noise is disabled, measurements include the proportions $f_{\ell b}^{\text{SID}} = |\mathcal{I}_G|^{-1} \sum_i \mathbf{1}\{k_{i\ell}^{\text{SID}} = b\}$, the effective number of codes $\exp\{-\sum_b f_{\ell b}^{\text{SID}} \log f_{\ell b}^{\text{SID}}\}$, the fraction of distinct complete codes, and the fraction of items beyond R_{SID}. Zero entropy terms are defined as zero. The soft loss is compared with the quality of the hard source. Utilization, collision, and runtime thresholds are fixed before the run; the regularizer does not guarantee compliance.

Diagnostics cover the entire $\mathcal{I}_G$ before applying the R_{SID} limit. Let $N_G = |\mathcal{I}_G| > 0$, $b_{\mathbf{k}} = |\mathcal{B}^{\text{SID}}(\mathbf{k})|$, with the sum below taken over all nonempty mapping tables:

$$\begin{aligned} \text{Col}_{\text{item}}^{\text{SID}} &= \frac{1}{N_G} \sum_{\mathbf{k}} b_{\mathbf{k}} \mathbf{1}\{b_{\mathbf{k}} > 1\}, \\ \text{Drop}_R^{\text{SID}} &= \frac{1}{N_G} \sum_{\mathbf{k}} \max(0, b_{\mathbf{k}} - R_{\text{SID}}). \end{aligned} \tag{174}$$

These measures distinguish nonunique codes from item exclusion; zero $\text{Drop}_R^{\text{SID}}$ does not rule out collisions. If $N_G > V_{\text{SID}}^{L_{\text{SID}}}$, at least $N_G - V_{\text{SID}}^{L_{\text{SID}}}$ assignments repeat; capacity is increased for a control without repetitions. The reverse inequality permits unique codes but does not guarantee them.

Three variants are required with the same $M, L_{\text{SID}}, V_{\text{SID}}$ and tuning budget: random fixed codes, codes obtained after reconstruction without recommendation fine-tuning, and jointly trained codes. In the first two variants, the assignment table is fixed: code vectors may be trained as decoder input representations, but changing them does not recompute the

assigned indices. The vector formulas for these variants replace soft assignments with $\rho^{\text{SID}}_{i\ell b} = \mathbf{1}\{b = k^{\text{SID}}_{i\ell}\}$. The random variant requires $V_{\text{SID}}^{L_{\text{SID}}} \geq |\mathcal{I}_G|$. Stream $m = 2, j = 6$ calls `choice(C,size=N_G,replace=False,shuffle=True)` once, where $C = V_{\text{SID}}^{L_{\text{SID}}} \leq 2^{63} - 1$ and $N_G = |\mathcal{I}_G|$. This uniformly samples distinct numbers from $0, \ldots, C - 1$ without replacement. The numbers are assigned to items in identifier order and represented in base V_{SID}, padded with leading zeros to length L_{SID}; one is added to each digit. This tests the value of semantic organization. When collision rates differ, the accessible fraction of the catalog is also compared.

X04–X05 check derivatives, training phases, output validity, and collisions (Section K.4).

Code table memory is $O(L_{\text{SID}} V_{\text{SID}} d_{\text{SID}})$, and index memory is $O(|\mathcal{I}_G| L_{\text{SID}})$. For a minibatch of b_N requests, full-catalog normalization requires $O(b_N |\mathcal{I}_G| L_{\text{SID}} d_{\text{SID}})$ operations after the soft item vectors have been computed. The catalog is processed in blocks with exact accumulation of the normalization sum: this saves memory but does not reduce the number of dot products. Decoding cost is bounded by $O(L_{\text{SID}} B_{\text{SID}} [(d_h^{\text{SID}})^2 + V_{\text{SID}} d_h^{\text{SID}} + d_h^{\text{SID}} d_{\text{SID}}])$, and the number of item mappings read is $B_{\text{SID}} R_{\text{SID}}$. Codebook size and beam width are varied under the same latency limit. Admission to the served configuration is checked under D.5; historical coverage and causal effect are evaluated separately.

E Additional retrieval and slate construction methods

These variants are not part of the first run. Each changes a separate part of the system and is compared with a fixed baseline under the same candidate limit.

E.1 Downweighting edges supported by few sessions

A separate research comparison may further reduce the contribution of edges supported by few sessions:

$$\chi^r_{ij} = \frac{n^r_{ij}}{n^r_{ij} + \lambda_{\text{edge}}}, \qquad \lambda_{\text{edge}} \geq 0,$$
$$g^{r,\text{edge}}_{qj} = \sum_{\ell \in \mathcal{D}_q} \alpha_{q\ell} \rho^r_{i_{s(q)\ell}} \chi^r_{i_{s(q)\ell},j} P^r_{i_{s(q)\ell},j}. \tag{175}$$

Coefficient χ^r_{ij} is computed only for retained edges, where $n^r_{ij} \geq 1$; setting $\lambda_{\text{edge}} = 0$ restores the original score. This variant retains the same P^r, topology, paths, and candidate quota, but changes graph scores and requires retraining the ranker on B. It may suppress useful rare relations, so it is not enabled automatically in the main configuration.

E.2 Additional slate diversity

An additional policy may reduce excessive similarity between items. The following probabilistic variant is outside the current formulation: it would require separately defined and validated estimates $\widehat{p}_{qi}$ of purchase probability. Scores $f(x_{qi})$ are not substitutes for them. Let $S_{ij} \in [0, 1]$ be the symmetric similarity of valid content vectors: negative cosine similarities are replaced by zero, and available modalities are combined in a normalized mixture. If no modality is shared, $S_{ij} = 0$ with an unknown-value mask. In this case, zero does not establish that the items differ.

For a set $L \subseteq \mathcal{C}_q$, define the surrogate

$$U_q(L) = \sum_{i \in L} \widehat{p}_{qi} - \frac{\lambda_D}{\max(1, K-1)} \sum_{\{i,j\} \subseteq L} S_{ij}, \qquad \lambda_D \geq 0. \tag{176}$$

The first term favors high observed purchase frequencies, while the second penalizes repeated inclusion of similar items. U_q is not measured in monetary units and is not conversion. The main version does not compute this objective: the list is sorted by the ranker score, with $\lambda_D = 0$. The probabilistic extension requires a separate training and selection plan and a new final period; tuning on the results of D is prohibited.

For $\lambda_D > 0$, items are added greedily:

$$i_k = \operatorname*{arg\,max}_{i \in \mathcal{C}_q \setminus L_{k-1}} \left[\widehat{p}_{qi} - \frac{\lambda_D}{\max(1, K-1)} \sum_{j \in L_{k-1}} S_{ij} \right], \quad L_k = L_{k-1} \cup \{i_k\}, \quad L_0 = \varnothing. \tag{177}$$

Addition continues through $k = K_q$, including when the increment is negative: the protocol fixes the output length. This algorithm is a heuristic; the global maximum of (176) is not guaranteed. Sequential construction costs on the order of $K|\mathcal{C}_q|$ similarity computations when accumulated sums are updated.

E.3 Why the sum of item probabilities is not conversion

Even if the true probabilities of a future purchase of every item under the same condition $\mathcal{H}_q$ were known, purchases of different items are dependent events. Only the general bounds below hold for the probability of at least one purchase of an item in L:

$$\max_{i \in L} p_i \leq \mathbb{P}\left(\bigcup_{i \in L} \{y_{qi} = 1\} \mid \mathcal{H}_q \right) \leq \min\left(1, \sum_{i \in L} p_i\right). \tag{178}$$

The formula $1 - \prod_{i \in L}(1 - p_i)$ requires conditional independence, which is generally not justified for substitute and complementary items. Estimates $\widehat{p}_{qi}$ are additionally conditional on the score and the candidate procedure, so substituting them into these bounds is only illustrative and does not produce guaranteed probability bounds. None of these formulas estimates the causal increase in purchases from changing the recommendation policy.

E.4 Multiple graph steps

The one-step graph admits a testable extension using a random walk with restart at the session history. Let T be a row-normalized mixture of relation matrices; zero rows are replaced by the historical distribution π^{pop}. Let h_q be the initial weight distribution over history items, obtained by summing $\alpha_{q\ell}$. On the shared snapshot catalog, define

$$z_q^{(n+1)} = \eta_R h_q + (1 - \eta_R) T^\top z_q^{(n)}, \qquad z_q^{(0)} = h_q, \quad 0 < \eta_R \leq 1. \tag{179}$$

At each step, part of the mass returns to the known history, while the remainder follows edges. If the known history is empty, this source is disabled. Items unknown to the graph receive no fictitious initial vertices; the remaining weights are renormalized.

Because T is row-stochastic, the transformation contracts distance in the sum-of-absolute-values norm by a factor of $1 - \eta_R$. Consequently, a unique limiting distribution exists, and for $n \geq 1$ it satisfies

$$\left\| z_q^{(n)} - z_q^* \right\|_1 \leq 2(1 - \eta_R)^n. \tag{180}$$

This bounds computational error, not recommendation quality. Transitions are performed on the sparse matrix; the contribution of replaced zero rows is computed by one multiplication of vector π^{pop} by their total mass. A full step with an explicit vector costs $O(N_G + E_{\text{sparse}})$, where N_G is the catalog size and E_{sparse} the number of retained edges before filling zero rows. A linear scan is needed even with no edges, so repeated scans of the full catalog may exceed the latency budget. The source is accepted only after measuring its additional purchase coverage and cost relative to one step.

E.5 Behavioral item vectors

Item2Vec can be studied as an additional source using co-occurring interactions [14]. Let $\mathcal{P}_A$ be item pairs from temporal and positional windows in period A_N, restricted to catalog $\mathcal{I}_G$. Each item has two learned vectors $u_i^B, v_i^B \in \mathbb{R}^{d_B}$: a center vector and a context vector. One possible training objective is

$$\begin{aligned} \mathcal{L}_B = &- \sum_{(i,j)\in\mathcal{P}_A} \left[\log \operatorname{sigmoid}((u_i^B)^\top v_j^B) + m_B \mathbb{E}_{k\sim Q_B} \log \operatorname{sigmoid}(-(u_i^B)^\top v_k^B) \right] \\ &+ \frac{\lambda_B}{2} \sum_i (\left\| u_i^B \right\|^2 + \left\| v_i^B \right\|^2), \qquad Q_B(k) = \frac{n_k^{3/4}}{\sum_j n_j^{3/4}}. \end{aligned} \tag{181}$$

Here n_k is the number of interactions with the item in A_G, m_B is the number of random contexts per positive pair, and $\lambda_B > 0$ is regularization. A random context helps distinguish characteristic neighborhoods from background frequency; it does not assert that the user rejected item k. The expectation is replaced by an average over independent samples from the fixed Q_B.

Let $\mathcal{D}_q^B \subseteq \mathcal{D}_q$ contain views of items with learned behavioral vectors. When this set is nonempty, the query representation is

$$u_q^B = \frac{\sum_{\ell\in\mathcal{D}_q^B} \alpha_{q\ell} u^B_{i_{s(q)\ell}}}{\sum_{\ell\in\mathcal{D}_q^B} \alpha_{q\ell}}, \qquad s_{qi}^B = (u_q^B)^\top v_i^B. \tag{182}$$

If $\mathcal{D}_q^B$ is empty, the source is unavailable. Using different center and context vectors is consistent with the training objective. Replacing the dot product with cosine similarity or averaging u_i^B, v_i^B defines a separate variant that requires evaluation. Behavioral and content vectors are not mixed coordinatewise without a specifically trained mapping.

Retrieval extensions are compared with the same overall budget M and the same request schedule. Added complexity is accepted only with a reproducible improvement in the selected purchase coverage measure and compliance with the serving budget. Their purpose is to measure the contribution of each retrieval method, not to increase the number of mandatory system components.

F Experiment with real impressions

This protocol is deferred for the public dataset. Before serving real impressions, it is required, together with checks of historical quality, technical limits on the target hardware, and outcome

recording. The control is the current policy or no recommendation block; it is fixed before launch [17].

F.1 Assignment unit and causal effect

Before launch, fix the assignment unit, eligible population, exposure horizon, and corresponding analysis unit. If the service has a stable permitted account/user/device key, assign variant $Z_u \in \{0,1\}$ by this key at first eligibility, with $\mathbb{P}(Z_u = 1) = 1/2$, and retain it throughout the experiment and outcome maturation. 1 denotes the new system and 0 the control. All sessions with the same key inherit the assignment; the assignment algorithm version, key, and result are recorded before calling the ranker. Rules for key changes, multiple devices, and cookie loss are fixed in advance; these cases must not allow hidden rerandomization. Eligibility is determined before exposure.

The primary session metric retains the unit from (185). The confirmatory cohort includes only the first eligible session $s(u)$ of each key new to the experiment; this rule is applied before assignment. The primary outcome $Y_{s(u)}(z)$ is a purchase after assignment in that same session before $t_{0,s(u)} + H$. Each key contributes exactly one session to the primary denominator:

$$\begin{aligned}\Delta^{\text{ITT}}_{1000,\text{entry}} &= 1000\big(\mathbb{E}[Y_{s(u)}(1)] - \mathbb{E}[Y_{s(u)}(0)]\big),\\ \widehat{\Delta}^{\text{ITT}}_{1000,\text{entry}} &= 1000\left(\frac{\sum_{u:Z_u=1} Y_{s(u)}}{N_1} - \frac{\sum_{u:Z_u=0} Y_{s(u)}}{N_0}\right), \quad N_0, N_1 > 0.\end{aligned} \tag{183}$$

Here N_z is both the number of assigned keys and the number of entry-cohort sessions; the unit is purchasing entry sessions per thousand, not all visits or people. A device key is not identical to a person. Later sessions inherit the variant but do not add to the primary denominator. The effect on any purchase by the key within H, and the ratio of purchasing sessions to all subsequent sessions, are separate secondary estimands only; the number of subsequent sessions may itself change under treatment. They do not replace the primary criterion S_{business}. For the entry cohort, the formulas below are applied with $s = s(u)$ and $n_z = N_z$ only if outcomes are independent and identically distributed across keys and there is no substantial interference between keys. If these conditions are not justified, an analysis by assignment units accounting for calendar dependence is specified in advance; an independent binomial SE is not used. The threshold, power, and covered traffic fraction are specified for this cohort.

Only when a stable key is unavailable is the limited short-term session protocol below permissible. At the first eligible request, assign $Z_s \in \{0,1\}$, with $\mathbb{P}(Z_s = 1) = 1/2$, and retain the variant through the end of the session and its outcome window. It does not estimate a user-level effect.

Let $Y_s(z)$ be the potential outcome: at least one purchase after assignment in the same session and before $t_{0,s} + H$, if the session is assigned variant z. For any specific session, the only observed outcome is

$$Y_s = Z_s Y_s(1) + (1 - Z_s) Y_s(0). \tag{184}$$

Assumption F.1 (Conditions for the session experiment)**.** Assignment is independent of potential outcomes. The policy within each variant is unambiguously defined. Assignment in one session does not substantially affect another session's outcome: for example, through users remembering recommendations, shared item scarcity, or popularity changed by the experiment. Purchase recording is equally complete in both variants. Independent binomial intervals additionally require independent and identically distributed outcomes across sessions within each group; randomization alone does not guarantee this condition.

Coveo has no stable person identifier across sessions, and this does not determine key availability in a future service. For session assignment, permitted privacy-safe aggregate signals of repeat visits and a sensitivity plan are defined before launch. The fraction of likely repeat visits is a diagnostic, not proof of the absence of carryover effects. If upper bounds r_z are justified for the fractions of outcomes potentially affected by cross-session interference in group z, then for binary outcomes the difference interval is expanded on both sides by $1000(r_0 + r_1)$, and intersected with $[-1000, 1000]$. Data-estimated bounds require separate accounting for their uncertainty and the overall error level. Without a justified bound and a plausible no-interference assumption, no confirmatory session effect is claimed; the result is descriptive.

Under Assumption F.1, for the entry cohort or the limited session protocol, the effect and estimator are

$$\Delta^{\text{ITT}}_{1000} = 1000\big(\mathbb{E}[Y_s(1)] - \mathbb{E}[Y_s(0)]\big), \tag{185}$$

$$\widehat{\Delta}^{\text{ITT}}_{1000} = 1000\left(\frac{k_1}{n_1} - \frac{k_0}{n_0}\right), \quad n_z = \sum_s \mathbf{1}\{Z_s = z\}, \quad k_z = \sum_s \mathbf{1}\{Z_s = z\}Y_s, \qquad n_0, n_1 > 0. \tag{186}$$

ITT is analysis by original assignment. Sessions with errors, empty responses, or no actual impression remain in n_z; Y_s is determined by a real purchase, not serving success. All purchases are counted without attributing an order to an impression. Selecting only those who clicked or saw the block breaks the original randomized comparison.

F.2 Interval for the business effect

The following formulas apply to the stated primary cohort with independent outcomes and a stopping point fixed in advance. For $n_0, n_1 > 0$ and sufficiently large counts of purchasing and non-purchasing sessions in each group, use the approximation

$$\widehat{p}_z = k_z/n_z, \qquad \widehat{\text{se}}_{\text{ITT}} = 1000\sqrt{\frac{\widehat{p}_1(1-\widehat{p}_1)}{n_1} + \frac{\widehat{p}_0(1-\widehat{p}_0)}{n_0}}, \tag{187}$$

$$I_{\text{normal}} = \left[\widehat{\Delta}^{\text{ITT}}_{1000} - z_{1-\alpha/2}\widehat{\text{se}}_{\text{ITT}}, \widehat{\Delta}^{\text{ITT}}_{1000} + z_{1-\alpha/2}\widehat{\text{se}}_{\text{ITT}}\right]. \tag{188}$$

Here z_u is a standard normal quantile. The confidence level is not the probability that a hypothesis is true. If a group is empty, the estimate and interval are undefined: status `insufficient`, with no substitution of a zero effect.

For rare outcomes, a conservative binomial interval can be selected in advance. For k purchasing sessions out of $n > 0$, $0 \le k \le n$, the bounds invert binomial tails using a validated implementation of beta-distribution quantiles:

$$\ell = \begin{cases} 0, & k = 0, \\ Q_{\text{Beta}(k,n-k+1)}(\alpha/4), & k > 0, \end{cases} \tag{189a}$$

$$u = \begin{cases} 1, & k = n, \\ Q_{\text{Beta}(k+1,n-k)}(1-\alpha/4), & k < n. \end{cases} \tag{189b}$$

Here $0 < \alpha < 1$, and Q is a quantile of the specified distribution; the result is checked for finiteness and $0 \le \ell \le u \le 1$. For two groups, the conservative effect interval is

$$I_{\text{exact}} = 1000[\ell_1 - u_0,\ u_1 - \ell_0]. \tag{190}$$

Noncoverage of each group's probability is at most $\alpha/2$, and that of the difference is at most α by the union bound. The independence conditions are given in F.1 and Assumption F.1; multiple sessions from one key require cluster analysis outside this binomial contract.

F.3 Business result acceptance rule

Let $[\ell_B, u_B]$ be the prespecified confidence interval for $\Delta^{\text{ITT}}_{1000}$ after the experiment is completed. Two claims are distinguished:

$$\ell_B > 0 \quad \Longrightarrow \quad \text{a positive effect is confirmed,} \tag{191}$$

$$\ell_B > \delta_{\text{accept}} \quad \Longrightarrow \quad \text{the business threshold is confirmed to be exceeded.} \tag{192}$$

Threshold $\delta_{\text{accept}} \geq 0$, in purchasing sessions per thousand, is set in advance. The claims hold under the assumptions and confidence level of the selected procedure; a point estimate exceeding the threshold is insufficient. The other acceptance conditions are specified in Section 20. With key-based assignment, these conditions apply to $\Delta^{\text{ITT}}_{1000,\text{entry}}$ and a threshold in purchasing entry sessions per thousand; a secondary user-level outcome does not replace this session criterion.

F.4 Experiment size and duration

Before launch, specify a detectable increase of $\delta_{\text{plan}} > \delta_{\text{accept}}$ purchasing sessions per thousand, error level α, and desired power $1-\beta$: the probability of detecting an effect of the assumed size. Let p_0 be the expected control rate, $\delta = \delta_{\text{plan}}/1000$, $p_1 = p_0 + \delta$, and $\bar{p} = (p_0 + p_1)/2$. For equal groups testing a zero effect, the approximate number of independent sessions in each group is

$$n \approx \frac{\left[z_{1-\alpha/2}\sqrt{2\bar{p}(1-\bar{p})} + z_{1-\beta}\sqrt{p_0(1-p_0)+p_1(1-p_1)}\right]^2}{\delta^2}. \tag{193}$$

The approximation is checked for applicability to the expected outcome rarity and analysis method. p_0 is obtained from a comparable stream or pilot; the Coveo rate does not transfer automatically to another store. With key-based assignment, plan the number of entry sessions, equal to the number of new keys; p_0 also refers to the entry cohort. If these outcomes are demonstrably independent, the formula applies, but substituting the count of all subsequent sessions is not allowed.

Formula (193) plans a test of no effect, not a lower bound above δ_{accept}. For the business threshold, test $H_0 : \Delta^{\text{ITT}}_{1000} \leq \delta_{\text{accept}}$ against $\delta_{\text{plan}} > \delta_{\text{accept}}$; the required size depends on the gap between these quantities.

The lower-bound rule for a normal interval gives a simple planning approximation. Denote $d_* = (\delta_{\text{plan}} - \delta_{\text{accept}})/1000$ and $v_* = p_0(1-p_0) + p_1(1-p_1)$, where $p_1 = p_0 + \delta_{\text{plan}}/1000$ must lie in $(0,1)$. With equal groups, the standard error of the difference in proportions is approximately $\sqrt{v_*/n}$. Therefore,

$$n \approx \frac{v_*(z_{1-\alpha/2} + z_{1-\beta})^2}{d_*^2}. \tag{194}$$

Condition $\sqrt{n}\, d_*/\sqrt{v_*} - z_{1-\alpha/2} = z_{1-\beta}$ plans for the lower bound to exceed the threshold when the standard error is close to its planned value. For rare outcomes, the power of the selected rule is checked by summing probabilities of pairs (k_0, k_1) or by simulation.

At v newly eligible primary-cohort units per second, the approximate enrollment time is $2n/v$, followed by $H + g$ of observation. The duration also covers the specified calendar cycles; varying traffic and session dependence affect power.

The date or rule for ending enrollment is fixed in advance; analysis is performed after all outcomes mature, following F.5.

F.5 Assignment check and stopping rule

Before the first assignment, save a plan of L calendar checks, the SRM control level $\alpha_{\text{SRM}} = 0.001$, and expected shares of $1/2$. Each check counts unique assigned units, including errors and missing impressions. If the plan includes G tested strata, membership $S_u = \phi(X_u^{\text{pre}})$ is computed before assignment from features available at that point. The value S_u is recorded in the assignment log and never changed. The plan specifies fields X_u^{pre}, the version and hash of rule ϕ, missing-value handling, and the expected share of each variant in each stratum. Clicks, impressions, fallback serving, and post-assignment outcomes do not define SRM strata; such breakdowns are permissible only for separately justified analysis.

The standard mode requires independent assignments with $\mathbb{P}(Z_u = 1 \mid X_u^{\text{pre}}) = 1/2$. Each scheduled check additionally requires that, conditional on the composition and size N of the overall cohort or each tested stratum, the number of assignments to variant 1 follows $\text{Bin}(N, 1/2)$. Composition is not adapted to assignment history or experimental exposure. Under these conditions, apply an exact two-sided binomial test; N includes all assigned members of the tested population regardless of serving and outcome. Without strata, each check uses threshold α_{SRM}/L; with G strata, the threshold for every test, including the overall test, is $\alpha_{\text{SRM}}/[L(G+1)]$. Dependent assignment or adaptive population composition requires a prespecified test of the actual mechanism with overall error control, not a binomial substitution. The plan contains the overall cohort composition rule, the test method, and justification of the conditional distribution and overall error control.

SRM means a mismatch in assignment proportions. When it is detected, new enrollment is paused, all assignments are retained, and missing logs, repeated keys, eligibility changes, and routing errors are investigated. Outcomes are not deleted to restore the proportion. Additional checks cover balance of prespecified pre-exposure features, integrity of assignment logs, and agreement between assignment and actual serving. Fallback does not change the ITT variant. Post-impression balance is not used to filter participants; the absence of SRM proves neither correct purchase recording nor the absence of interference.

The safety monitor receives only predeclared measures; before launch, fix their windows, denominators, minimum sample sizes, thresholds, response actions, and check schedule. Crossing an emergency threshold stops exposure; the availability of such a channel does not by itself make the final interval valid under data-dependent stopping. The main protocol has one final analysis with no interim efficacy decisions. Any data-dependent early stop, including a planned safety stop or a stop due to SRM, as well as any unplanned interruption, makes the causal result descriptive: ordinary intervals and p-values are not reported as confirmatory, and rules (191) and (192) are not applied. After investigation, a new preregistered experiment is required for a new confirmatory conclusion. Confirmatory analysis may be retained after planned interim stops only under a procedure separately specified before launch with overall error control (for example, alpha-spending or always-valid); it cannot be chosen retrospectively.

F.6 Incomplete outcomes and safety measures

A mature calendar horizon does not guarantee telemetry completeness. If group z contains m_z sessions with unknown outcomes, and k_z^{obs} purchases are reliably recorded among the remainder, then for $n_z > 0$, without additional assumptions,

$$\frac{k_z^{\text{obs}}}{n_z} \le p_z^{\text{sample}} \le \frac{k_z^{\text{obs}} + m_z}{n_z}. \tag{195}$$

The corresponding range of compatible sample effects is

$$1000 \left[\frac{k_1^{\text{obs}}}{n_1} - \frac{k_0^{\text{obs}} + m_0}{n_0}, \quad \frac{k_1^{\text{obs}} + m_1}{n_1} - \frac{k_0^{\text{obs}}}{n_0} \right]. \tag{196}$$

This is a range of compatible results, not a confidence interval: statistical uncertainty is assessed separately. Unknown outcomes are not replaced by zeros or removed from the ITT denominator without justification.

Before the experiment, specify limits on deterioration in error rate, latency, empty outputs, purchase recording, and the fraction of ineligible items. Stopping on these measures is permitted; the status of the causal analysis is governed by Section F.5. Unknown outcomes, maturity rules, and completeness are checked at the same assignment unit: with key-based assignment, $n_z, m_z, k_z^{\text{obs}}$ refer to the entry sessions of those keys, not all their sessions or purchases.

Session length, clicks, and cart additions are reported as secondary results. Multiple confirmatory effects require a prespecified multiplicity correction. Breakdowns by post-assignment features do not admit a causal interpretation without separate justification.

G Why historical data do not identify an increase in purchases

Nonidentifiability is established in Proposition 6.1. For example, Hit@20 of an empty output is zero, so a higher value from the new system does not prove an increase in purchases relative to no recommendations.

G.1 Conditions for evaluating another policy from an impression log

Evaluating a new policy from the old policy's log in a single-decision setting requires records $(X_s, A_s^{\text{act}}, Y_s)$: pre-assignment context, the list actually displayed, and the outcome. The old policy's probability $b(a \mid x)$ of choosing the list must be known.

This requires no unobserved causes of list selection that also affect the outcome, correct impression recording, and the support condition:

$$\pi(a \mid x) > 0 \implies b(a \mid x) > 0. \tag{197}$$

Under these conditions, the inverse-propensity estimator is

$$\widehat{V}_{\text{IPS}}(\pi) = \frac{1}{N} \sum_s \frac{\pi(A_s^{\text{act}} \mid X_s)}{b(A_s^{\text{act}} \mid X_s)} Y_s. \tag{198}$$

Rare lists with small probability b receive large weights, so variance may be unacceptable [15]. Weight clipping reduces variance at the cost of bias.

If an estimate $\widehat{m}(x,a) \approx \mathbb{E}[Y \mid X = x, A^{\text{act}} = a]$ has also been trained, the outcome model and weights can be combined:

$$\widehat{V}_{\text{DR}}(\pi) = \frac{1}{N}\sum_{s}\left[\sum_{a}\pi(a \mid X_s)\widehat{m}(X_s,a) + \frac{\pi(A_s^{\text{act}} \mid X_s)}{b(A_s^{\text{act}} \mid X_s)}\big(Y_s - \widehat{m}(X_s, A_s^{\text{act}})\big)\right]. \tag{199}$$

The outcome model is trained without the evaluated record. Double robustness means consistency when at least one of the two components is correctly specified, under causal identification and regularity conditions. It does not resolve a lack of support [16].

Formulas (198)–(199) apply to a single decision and a fixed subsequent regime. Sequential recommendations change future context; substituting individual requests with a shared session purchase requires a different justification.

Coveo lacks the required assignment probabilities and a complete recommendation impression log. Search results with clicks do not replace them, so IPS and DR are excluded from the main calculation. Predicted purchase probabilities also do not replace assignment probabilities b.

G.2 What event replay can establish

Replay checks feature availability, session state, candidate retrieval, latency, and constraints. A simulated user tests scenarios under a declared behavior model; neither method establishes an increase in real purchases.

Test K.18 checks the distinction between coverage and effect.

H Run configurations and parameter registry

H.1 Fixed configuration for the first run

The first run is GCS+Rk: three graph relations, descriptions, images, the search query, and LambdaMART. It uses direct relations and an unchanged graph under `observed_purchases_v1`, with no neural sources or parameter tuning. The settings below are initial settings, not measured optima on Coveo.

Data and time. Before training, fix the boundaries of the seven parts, session selection, file versions, and identifier order. The graph, catalog, and statistics are built on A_G; the ranker is trained on B. The reserved A_N is not used by the baseline. The first report is produced on C_{smoke}; C_{select}, E, D remain closed.

$$\begin{aligned} H &= 86\,400 \text{ s}, & g &= 1800 \text{ s}, & K &= 20, \\ Q_{\max} &= 1, & M &= 300, & d_G &= 100, \\ \Delta_G &= 1800 \text{ s}, & \tau_G &= 600 \text{ s}, & n_G &= 3, \\ \tau_H &= 300 \text{ s}, & \Delta_S &= 600 \text{ s}, & \lambda_G &= 10. \end{aligned} \tag{200}$$

The graph uses $B_G = 10$ preceding item timestamp groups; its session representation uses the $L_H = 20$ latest views under Section 7. Store $L_{\text{hist}} = 20$ timestamp groups; popularity smoothing is $\eta_P = 1$. The purchase-rate feature is not used.

Retrieval and features. Search over description, image, and query vectors is exact over the full eligible catalog. Quotas are fixed:

Source	Quota
Each of the three graph relations	50
Item description	40
Image	30
Search query	50
Recent views	20
Historical popularity	10
Total	300

Deduplication, unavailable-source masks, and filling by popularity follow (30). The ranker receives separate graph and content scores; mixtures with coefficients ω_r and γ_m are not used. The additional factor χ^r_{ij} is disabled. Double-precision tolerances are $\varepsilon_v = 10^{-12}$, $\varepsilon_f = 10^{-8}$. The ordering of tied scores and categorical encodings are fixed together with the feature schema.

Ranker and output. The LightGBM version, derivative function, and all settings are specified in 11.5. The first-run budget is not selected on C_{select}. The similarity penalty is disabled: $\lambda_D = 0$.

Validation and experiment completion. The first run ends with the checks in K.2 and the report on C_{smoke} specified in I.2. Compliance with load requirements under exact search is measured; approximate search is introduced through a separate comparison before D is opened.

Calibration is disabled. Final evaluation follows Sections 14.1 and 13.5. The initial $\alpha = 0.05$ and $B_{\text{boot}} = 2000$ do not replace the precision plan and conditions for applying temporal intervals.

H.2 Neural extension parameters

The graph, catalog, and statistics from A_G are retained; representations are trained on A_N. Quotas are redistributed under the overall limit $M = 300$:

Source	Baseline	Neural	With SID
Each of the three graph relations	50	30	30
Item description	40	30	30
Image	30	20	20
Search query	50	40	40
Recent views	20	20	20
Popularity	10	10	10
Mixed neural vector	0	30	20
Each of the two intents	0	30	10
Semantic code	0	0	50
Total	300	300	300

An inactive intent returns an empty list. After deduplication, vacant positions are filled by popularity according to formula (30); its initial quota does not limit additional filling. In a subsequent

evaluation with $J_I = 4$, each intent has a quota of 15 for the neural architecture and 5 for the SID variant; the total neural quotas of 90 and 40 are retained. With $J_I = 1$, the single direction receives 60 and 20 positions, respectively, in addition to the mixed source.

The initial configuration is

$$\begin{aligned} d &= 64, & J_I &= 2, & H_P &= 2, & d_P &= 50, \\ B_P &= 100, & R_P &= 20, & c_G &= 0.1, & \eta_N &= 1, \\ \tau_P &= 1, & \tau_N &= 0.1, & \tau_C &= 0.1, & \tau_{SN} &= 300 \text{ s}. \end{aligned} \tag{201}$$

For the three relations, initially $\zeta_r = 1/3$; the direct search mixing limit is $\bar{g}_N = 0.5$. The initial actual weight when search is available is $g_0 = 0.1$: initially, $v_g = w_g = 0$, $a_g^t = 0$, and the bias is

$$b_g = \log \frac{g_0}{\bar{g}_N - g_0} = \log \frac{1}{4}. \tag{202}$$

When the search signal is absent, the mask still gives $g_q^{SN} = 0$. These values do not separately limit the influence of β_S on the intent-weight distribution. Loss weights are

$$\lambda_{\text{next}} = 0.2, \qquad \lambda_C = 0.05, \qquad \lambda_G^N = 10^{-3}, \qquad \lambda_\theta = 10^{-5}. \tag{203}$$

In the initial extension, $p_S = 0$: random search dropout is not applied. The diagnostic threshold for intent agreement is fixed at $\delta_I = 0.05$. For 32-bit floating-point computation, the initial value is $\varepsilon_N = 10^{-6}$. The probability of dropping each available branch when constructing contrastive representations is 0.2, with branch restoration following Section 22.8. The optimizer, initialization, batches, and three training epochs are specified in 22.10. Checkpoints after one and five epochs are reserved for a separately planned continuation; evaluating them also counts toward the selection budget on C_{select}.

The cost of an exact epoch is measured using the actual $|\mathcal{I}_G|$ and number of training requests. If the budget is exceeded, use the separately identified mode in Appendix A: $m = 512$, $\delta = 0.1$. Values $m = 1024, 2048$ require a separate plan; a sampled denominator changes the training procedure.

H.3 Initial settings for the deferred SID study

The deferred study in Appendix D uses

$$\begin{aligned} L_{\text{SID}} &= 4, & V_{\text{SID}} &= 32, & d_{\text{SID}} &= 16, \\ d_h^{\text{SID}} &= 64, & B_{\text{SID}} &= 32, & R_{\text{SID}} &= 4, \end{aligned} \tag{204}$$

and $m_{\text{SID}} = 50$. Initial weights are $\lambda_R^{\text{SID}} = 1$, $\lambda_X^{\text{SID}} = 0.1$, $\lambda_B^{\text{SID}} = 0.01$, $\lambda_2^{\text{SID}} = 10^{-5}$; $\tau_R^{\text{SID}} = 1$. For T_{SID} joint steps specified in advance, the following schedule is permissible:

$$\sigma_t^{\text{SID}} = (1 - t/T_{\text{SID}})_+ , \qquad \tau_t^{\text{SID}} = 0.2 + 0.8\,(1 - t/T_{\text{SID}})_+ . \tag{205}$$

Training phases are specified in 22.10 and Section D.4. Hard-code utilization is checked over the entire catalog: $32^4 = 1\,048\,576$ possible codes do not rule out collisions with fewer items. Increasing L_{SID} or V_{SID} must be justified by measured retrieval losses.

H.4 Run manifest and stage checks

One run manifest stores the fixed inputs, code, and experiment settings in JSON; stage results are stored separately and refer to this manifest. A separate manifest snapshot and SHA-256 file for every stage are not mandatory. Before opening C_{select}, D, or assigning A/B variants, the corresponding plan is saved immutably; a version or checksum reference links it to the results. Before computation, verify that the code, settings, and inputs match the plan; a result cannot change a plan that has already been fixed. The normative template `documents/math/coveo_session_recommender_run_manifest.template.json` is not ready to run: unknown dates, hashes, and measurements are `null`. They must not be replaced with invented data. Template `manifest_version=2.10` specifies the mode through `evaluation_protocol`; the run manifest fixes the first-request rule, observed target, evaluation population, precision thresholds in proportions, and disabled components under Section 14.1. Unknown coverage is not replaced with complete coverage. Types, states, temporal relationships, input integrity, and conditions of the applicable stage are validated with ordinary data checks and tests. A separate test registration system is unnecessary; the validating code version is reproducible. Check B13 includes valid and invalid run manifests.

Field group	Required contents
Data	Versions and SHA-256 of the three tables, schema, vector dimensions, observation coverage and gaps; UTC and integer milliseconds, canonicalization and session subsampling rules.
Periods	Boundaries of all seven parts as $[a, b)$; nonoverlap, whole sessions, and eligibility (49). Calendar fields have the prefix `period_`; matrices have `matrix_`.
Code and models	Source code hash, pinned dependency file with package versions and hashes, feature schema, derivative function, full settings of active modules, hardware, numeric types, random-number generators and their states.
Selection and evaluation	`evaluation_protocol=observed_purchases_v1`, variants and selection rule, primary comparison, metadata sensitivity, precision and temporal dependence plan. Requirements are determined by the stage table below.
Serving	Load and limits, delivery and recovery rules, trusted sources, retention periods, monitoring, responsible role; required for service validation and admission to A/B testing.
Checks	Standard tests mapped to requirement IDs; their code defines the input or recipe, an independent reference, and tolerances. Separate stage results contain the actual outcome, code version, configuration, and seed. A separate hashed manifest for every test is unnecessary.

Stage conditions. Field `stage` specifies the stage being validated. The table determines required fields; data for future and inapplicable stages are not required.

Each result record contains the requirement/test ID, state, reason, actual report or a link to it, and reproducible code version, configuration, and seed. `null` is allowed before the execution deadline, but does not replace a required result when the stage is completed. For a disabled module, record `not_used` and a reason; the same deadlines apply when it is enabled. E is reserved and unused; calibration is disabled.

Stage	Completion conditions
`preflight`	Before training: data, periods, code, active model parameters, and numerical tolerances are filled in; applicable checks of data, canonicalization, eligibility, and mathematical preparation properties are completed before computation. Trained models, planned precision parameters for evaluation on D, and future load results are not required.
`post_training`	Trained models and their hashes are saved; applicable mathematical checks of training and data separation are completed. The real A/B protocol and B18 are not mandatory at this stage.
`post_load`	Separate service acceptance, not a prerequisite for local historical evaluation. Three load runs, memory, latency, failures, and replay and recovery checks are saved. The test result may be negative; this prohibits acceptance but not recording the result.
`pre_selection`	Before opening C_{select}, the variants, budget, and selection rule are fixed; training results, local cost measurements, and applicable checks of variants participating in selection are available. The plan includes the configuration of the variant without metadata. Service load acceptance is not required. Further development based on C_{select} is prohibited.
`pre_D`	The primary policy or pair and the full evaluation plan are fixed: half-width in proportions, planned variance, assumptions and dependence multiplier, provenance of the pilot on C_{smoke}, calendar grid, and blocks; selection conditions and applicable evaluation checks are satisfied, including the complete variant without metadata. Ranker errors and historical protocol violations block computation; service readiness is not required for `ideal_deterministic`. Results from D do not yet exist.
`post_D`	Final estimates, intervals, and states from Section 20 are saved, including negative and insufficient results. The plan is not changed retrospectively.
`pre_AB`	Before variant assignment, the experiment protocol, assignment key, sample size and duration calculation, SRM, emergency conditions, responsible roles, trusted keys, and production limits are filled in. The A/B plan is saved immutably; B18, serving, and applicable safety checks have passed, and readiness (100) is satisfied.
`post_AB`	After outcomes mature, results by assigned variant, integrity checks, and stopping information are saved.

Deadlines for applicable tests are specified in K.1. After selection, the final model is frozen and its link to the selection result is checked. Before each preparation, training, or evaluation computation, check the relevant data, mathematical properties, and immutability of the applicable plan. Saved checks are repeated when affected code or inputs change. For local evaluation, `post_load` is not a prerequisite for `pre_selection` or `pre_D`. A missing required report produces `not_run`; a test that runs and detects a violation produces `failed`. Data suitability, evaluation precision, and model quality are tracked separately under Section 20.

H.5 Preliminary calendar plan

Preliminary time shares for A_G, A_N, B, C, E, D are 20%, 20%, 25%, 15%, 10%, 10%. After applying calendar eligibility, publish the actual sizes N_J^{obs} and N_J^{+}; E remains reserved. Within C, equal calendar halves are initially allocated to C_{smoke} and C_{select}; eligibility with $H+g$ is applied to each separately. The shares are only an initial planning grid. If they do not yield eligible sessions or the required number of temporal blocks, the plan is changed before training using the early stream, or the research status is retained. Insufficient duration cannot be remedied by dividing a dependent period into artificially short independent blocks. A period shorter than $H + g$ is unsuitable. Boundaries and the rule for additional collection are set before training using preceding data; purchasing sessions are not selectively added, and results from D are not used for this purpose.

H.6 Parameter sources

The first run uses the recipe in H.1 without tuning. In subsequent comparisons, hyperparameters are selected only on C_{select}; model parameters are trained on the assigned period. The table identifies where each is defined without repeating its values.

Group	Where defined and fixed
Problem setup: $H, g, K, Q_{\max}$	Request rules 5; fixed before session selection.
Graph and history: $\Delta_G, B_G, \tau_G, \tau_H, n_G, d_G, \lambda_G, L_H, L_{\text{hist}}$	Section 7; statistics from A_G only. View limits and timestamp-group limits differ.
Sources: M, m_a, Δ_S, η_P	Recipe H.1; comparisons preserve the overall budget. Quotas do not replace filling after deduplication.
Ranker	Section 11.5; training on B. Penalty $\lambda_D = 0$ in the baseline.
Precision and intervals	Sections 15.2, 15.3. The plan and assumptions are fixed before opening D using the permitted pilot on C_{smoke}; they are not required for `preflight` or training.
Serving and numerical tolerances	Sections 17.5, 16.2. Thresholds are fixed before model selection.
Neural network: dimensions, paths, temperatures, loss weights	Appendix H.2; training on A_N with the graph, catalog, and statistics of A_G unchanged. Optimization rules are in 22.10.
Search mixing: $\bar{g}_N, g_0, p_S$	$0 \le \bar{g}_N \le 1$; with a positive bound, $0 < g_0 < \bar{g}_N$. Zero direct mixing does not disable the influence of search on intent weights. Initial values are in H.2.
Sampled denominator: m, δ	Appendix A; a separate training procedure, with the proposal built from A_G.
Sequence and aging	Appendices B, C; the history window and aging do not replace within-session decay.
SID: dimensions, codebooks, schedules, losses	Appendix H.3; training on A_N with the neural network fixed. In the trainable variant, $\lambda_R^{\text{SID}} > 0$; the other penalties are nonnegative.
Simple mixtures and rare-edge penalty	Section E; separate comparisons. The baseline does not use $\omega_r, \gamma_m, \lambda_{\text{edge}}$.

Group	Where defined and fixed
Impression experiment	Section F; α, power, the business threshold δ_{accept}, and the alternative $\delta_{\text{plan}} > \delta_{\text{accept}}$ are fixed before assigning variants.

H.7 Period sizes and data sufficiency

A sample of 200–500 thousand sessions may be sufficient for an engineering prototype, but the number of sessions with an observed target is unknown in advance. When a period with targets is opened, record

$$N_J^{\text{obs}}, \qquad N_J^{+} = |\mathcal{S}_J^{+}|, \qquad N_J^{\text{pair}} = \sum_{s \in \mathcal{S}_J^{+}} \mathbf{1}\{|\mathcal{J}_{q_s^{(1)}}| > 0\}. \tag{206}$$

The first two numbers count sessions; the third counts requests with training pairs. Candidate rows are not independent purchases. For A_N, also report the number of requests with $Y_q \cap \mathcal{I}_G \neq \varnothing$, the number of usable contrastive pairs, and the fraction of targets outside the catalog. Data sufficiency is checked for every stage.

Coverage precision conditions are specified in 15.2 and 15.4. Additional collection is allowed only under a prior plan, not until a convenient purchase count is reached.

I Comparison order and isolating component contributions

After the run in H.1, select the next comparison under Section 14.5. Before starting, fix the question, data, and budget. The tables below list permissible studies; running every row is not required. P and R are mandatory descriptive controls in the first evaluation; G–GCS provide diagnostics on C_{smoke}. The main control for improvement studies is GCS+Rk.

Variant	Components	Question tested
P	Historical popularity	Does the system outperform an available nonpersonalized response?
R	Repeat views and popularity	Is the result explained by a simple reminder of viewed items?
G	Graph relations and fallback sources	Are item-to-item transitions useful beyond repeat views?
GC	Graph and item features	Does content similarity improve coverage under the same candidate limit?
GCS	Graph, item features, and search query	Does available search context provide additional information?
GCS+Rk	The same sources and a trained ranker	Does ranking reduce the gap between candidate coverage and final-output coverage?

The second group of comparisons isolates the contributions of parts of the trainable module. Baseline sources are retained unless a row states otherwise.

Variant	Change relative to the full trainable model
Without graph branch	Remove e_i^G; retain the other branches and training on A_N.
Direct relations only	$H_P = 1$; test the contribution of indirect paths with the same number of retained neighbors.
One intent	$J_I = 1$; the total quota of intent sources is assigned to the single direction.
Without item content features	Remove branches e_i^T, e_i^V of the new encoder; retain identifiers, graph, and available search context.
Without search in the encoder	Set mask M_q^S to zero throughout the new module, including the search-only fallback representation; the baseline search source is unchanged.
Without direct search mixing	Only $\bar{g}_N = 0$; retain the influence of search on intent weights and all other rules.
Without contrastive component	$\lambda_C = 0$; retain the other targets and available modalities.
Without next view	$\lambda_{\text{next}} = 0$; retain purchase training and the selected regularizers.
Full model	All enabled components from Section 22; the main architecture's quotas from the initial plan.
Sequential control	Replace the session encoder with one causal attention block while retaining the graph item encoder; the same inputs, purchase target, and candidate budget.

Variant	Change relative to the full trainable model
Sampled denominator	The approximate loss from Appendix A; the exact version is a reference at a feasible scale.
Three SID variants	Random fixed codes, fixed codes after reconstruction, and jointly trained codes; quotas of the architecture with SID.

In a single-component comparison, retain A_N, the other targets, and the training schedule. Compare the sequential encoder with the main model using $\lambda_C = 0$ in both. When changing J_I, retain the total quota and M. The static-vector assumption is tested by separately removing all supplied vectors, including baseline content sources and search: this compares available information, not a single branch.

When comparing retrieval, fix the ranker algorithm and tuning budget, but retrain it on each variant's candidates in B. Compare the candidate bound and the output. When comparing only rankers, fix the same candidates; select simple-mixture coefficients on C_{select}. Intents are checked under Section 22.11; high weight entropy does not prove distinct directions, and the diversity penalty is initially disabled.

Build the graph, catalog, and statistics from the same A_G; note additional training on A_N in the report. Compare the contribution of architecture using identical training data. Equal M does not mean equal cost: before starting, limit the number of configurations, time, and memory; report hardware, completed runs, and reasons for failures.

The next stage, separate from the first run, permits at most 12 complete configurations in total. The initial plan contains nine: GCS+Rk and neural variants $J_I = 1, 2$, each with budget `num_boos t_round`$\in \{100, 200, 400\}$; the actual N_T does not exceed the selected budget. Neural variants are trained for three epochs using the same procedure; the random seed is 17. Additions are registered before evaluation. Seeds 29 and 43 are reserved for a separate stability study; the best run is not selected on D.

The report format is specified below. An NDCG improvement does not replace the primary criterion or operational constraints. Before opening D, fix one primary pair: the model selected on C_{select} and GCS+Rk. Other comparisons are exploratory; confirming several superiority claims requires a prespecified family of tests and a multiplicity correction. Selecting an improvement based on D requires a new final sample. The business effect is evaluated in a separate experiment with real impressions.

I.1 Required metadata sensitivity analysis

The main comparison using supplied vectors is conditional on their unknown construction date. The required variant `graph_recent_popularity_no_provided_metadata` excludes all supplied text, visual, and search vectors, category, price, their masks, and derived features from retrieval and the ranker. Search clicks/lists and query context are also unused. The remaining inputs are IDs, available events, the graph, repeat views, and popularity statistics built exclusively on A_G; the neural network and SID are disabled. Catalog $\mathcal{I}_G$, sessions, the request schedule, eligibility rules, K, and M are retained so the denominator does not change.

The three graph relations each have a quota of 50, repeat views have 20, and the initial popularity quota is 10; after deduplication, popularity fills the remaining positions up to $M = 300$. A new ranker is trained on its own candidates from B with the same budget and callback, using a feature schema without metadata fixed in advance; $\lambda_D = 0$. The configuration and its hash are

fixed before C_{select}. The complete implementation and applicable checks of this variant are required before its computations and by final evaluation, but do not block the first C_{smoke}. For D, the published results must include the chain $V, U,$ Hit, the difference from the baseline, temporal sensitivity intervals, and cost.

This is a required check of the result's dependence on information, not an automatically confirmatory second hypothesis. There remains a single primary pair; if several superiority claims are made, the family and multiplicity correction are fixed before D. Stability in this check does not prove the absence of leakage in the unknown snapshot; instability limits the applicability of the original result. Replacing the supplied metadata while events remain unchanged must not change the candidates, features, or response of the variant without metadata.

I.2 Required report for the first run and comparisons

The table defines report fields, not results on Coveo. For the first run, it is filled in for C_{smoke}; for D, only after the protocol is fixed. Unavailable quantities receive a state with a reason, not a substituted number. For local `ideal_deterministic`, cost means measured local time and memory; service load metrics remain unmeasured until `post_load` and do not block the historical report. Protocol `observed_purchases_v1` is stated explicitly. All quality measures refer to $\mathcal{S}_J^+$; costs also account for sessions without an observed target. Telemetry completeness remains unknown; conversion across all sessions is not estimated.

Block	Required quantities
Data and version	Period, policy version, N_J^{obs}, N_J^+, number of sessions without an observed target, and exclusion reasons.
Coverage chain	$\sum_s V_s$, $\sum_s U_s$, $\sum_s \text{Hit}_s$; the three losses (60), conditional coverage (61), and $\sum_s V_s / N_J^+$. A proportion is undefined when its denominator is zero.
Final output	Hit@20_J, $\text{Coverage}_{\text{cand},J}$; paired difference from the baseline and its confidence interval when comparing models.
Sources	Availability, list sizes, overlap, unique coverage; losses from quota limits and Δ_a^{fixed} from (64). Removal with fallback filling and retraining is performed only as a separate selected experiment (Section 14.4).
Request conditions	Session counts and coverage in prespecified groups: search, cart, modality availability, item history, and session history. Purchases from the observed cart and outside it are reported separately.
Cost	Training time, accelerator hours, peak RAM and GPU memory; hardware, load, P95, P99, error, deadline-exceedance, and fallback-response rates; agreement of approximate and exact search.
Next action	Reason for selecting the next comparison under Section 14.5, data to be fixed, and budget.

Save the short source lists before merging, item provenance, the output, and reasons for fallback responses. Quota losses are estimated from target ranks or with diagnostic lists of a depth bounded in advance; unknown ranks are not reconstructed by assumption. A technical failure

does not form a fourth term in the decomposition: it can affect several stages, while fallback may preserve a hit.

J Numerical example

Consider three eligible items a, b, c. Normalizing a row of one graph relation gives $P_{ab} = 0.8$, $P_{ac} = 0.2$; the total row weight is $d_a = 20$, and $\lambda_G = 10$. The history contains only a view of a, so

$$\rho_a = \frac{2}{3}, \qquad \alpha_{q1} = 1, \qquad g_{qb} = \frac{8}{15}, \qquad g_{qc} = \frac{2}{15}. \tag{207}$$

These numbers are not purchase probabilities. Let the normalized text vectors and query be

$$\bar{d}_a = (1, 0), \quad \bar{d}_b = (0.8, 0.6), \quad \bar{d}_c = (0, 1), \quad \bar{z}_q = (0, 1). \tag{208}$$

Text similarity to the viewed item is 0.8 for b and 0 for c, while search similarity is 0.6 for b and 1 for c. The graph and history favor b; search favors c. This is a meaningful disagreement between features, resolved by the learned ranker, not by an unspecified addition of different scales.

If the recorded future purchase is c and the candidates are $\{b, c\}$, then $y_{qb} = 0$, $y_{qc} = 1$. With equal initial scores, $\beta_R = 1$, and pair weight $\omega = 1$, the derivatives are

$$g^R_{qc} = -\frac{1}{2}, \quad g^R_{qb} = \frac{1}{2}, \qquad h^R_{qc} = h^R_{qb} = \frac{1}{4}. \tag{209}$$

A minimization step increases the score of c relative to b. The numbers illustrate the derivatives of one pair and do not describe a completed model on Coveo.

Now suppose that, of 1000 eligible recorded sessions, 30 have a recorded future purchase, and the first output covered such an item in 12 sessions. The first request's candidates contained it in 20 sessions. Then

$$N^{\text{obs}}_J = 1000, \quad N^+_J = 30, \quad \text{Hit@20}_J = \frac{12}{30} = 0.4, \quad \text{Coverage}_{\text{cand},J} = \frac{20}{30}. \tag{210}$$

If an eligible target existed in 24 sessions, the three losses as proportions are $(30 - 24)/30$, $(24 - 20)/30$, and $(20 - 12)/30$. Their sum, 0.6, complements Hit@20 to one. The other 970 sessions do not become confirmed negative outcomes. Conditional retrieval coverage is $20/24$; neither the gap nor this proportion measures additional purchases caused by recommendations.

In a separate randomized experiment, 3000 purchasing sessions out of 100 thousand in the control and 3200 out of 100 thousand in the new variant give

$$\widehat{\Delta}^{\text{ITT}}_{1000} = 2, \qquad \widehat{\text{se}}_{\text{ITT}} \approx 0.775. \tag{211}$$

The normal 95% interval is approximately $[0.48, 3.52]$. It supports a positive effect under the experiment's assumptions, but does not prove that the threshold of one additional purchasing session per thousand is exceeded: the lower bound is below one. All numbers in this appendix are invented to check the formulas.

K Testable implementation properties

Active modules are checked. Changes to data, features, the model, or output rules require rechecking the affected properties.

K.1 Requirement mapping and check order

Each requirement below has a permanent ID mapped to a standard test through its name or marker. Test code defines the input or the recipe for constructing it, an independent reference, tolerances, and the initial generator state. The result retains the actual report, code version, configuration, and seed; separate preregistration or a hashed manifest for every test is not mandatory. The deadline follows applicability under H.4. The absence of a reference or report does not count as a passed check.

Counters, sets, masks, versions, and order are compared exactly. Numbers must satisfy $|x - x_{\text{ref}}| \leq \text{atol} + \text{rtol}|x_{\text{ref}}|$; tolerances are specified in 16.2, and for tree derivatives in 11.5. An unexpected NaN/Inf is a failure; exceptions, such as latency $+\infty$, are listed explicitly. Generative checks contain at least 1000 cases per property, with initial generator seed 17. Save its algorithm, version, and minimal counterexample. The numerical examples below are mandatory.

Deadlines: B01, B02, B06, B09, B13, and `B10.data` are due at `preflight`; B04, B05, B07, B08, and `B10.ranker` are due before applying the respective computations, with actual artifacts checked at `post_training`. B14–B17 and `B03.service` (delivery and state) belong to `post_load`; `B03.data` (schema and canonicalization) belongs to `preflight`. Applicable replay properties for local evaluation are checked before computation, without mandatory implementation of service delivery and recovery. B11 checks that calibration is disabled before the first output calculation. `B10.metadata` is mandatory for the complete variant without metadata before its computations and before D; the variant configuration is fixed before C_{select}. B12 is performed before computing the reported temporal intervals and on actual inputs before D. For sampled training, `X02.support` is performed at `preflight`, and `X02.loss` at `post_training`. Other mathematical checks of enabled research components are performed before the respective computations are applied and confirmed at `post_training`. Applicable B10–B12 checks are repeated on actual artifacts before `pre_D`. B18 and the real experiment protocol are required by `pre_AB`, but not by `post_training`. Results on D and actual A/B outcomes are recorded only at `post_D` and `post_AB`, respectively. Tests of an enabled ANN are completed before it is admitted to serving.

K.2 Baseline system

Test K.1 (B01. Time and canonicalization)**.** On the millisecond grid, check events exactly at t_q and after it, a purchase in the first group and at $t_{0,s} + H$, a start at the eligibility boundary, and the recorded-history boundary $t^{\text{obs}}_{\text{last},s} + g = b_{A_G}$. Equality excludes the session from A_G; the calendar margin does not establish telemetry completeness. The reference is the inequalities in 4 and 5, without rounding. Permuting simultaneous rows preserves the result. Two identical rows with distinct IDs have raw multiplicity 2: `set_v1` gives 1, `multiset_v1` gives 2; repeating the same ID does not increase multiplicity. Future events do not change past inputs. For the last group with nine views of A and one of B (distinct event IDs) and candidate similarities of 1 and 0, the mean (35) is 0.5 under `set_v1` and 0.9 under `multiset_v1`. Without vector B, both means equal 1; without either vector, the value and mask are 0. Changing multiplicity semantics preserves the set of distinct items in the group and the maxima, but may change retrieval by averaged scores.

Test K.2 (B02. Search ambiguity)**.** Two vectors $(1, 0)$ and $(0, 1)$ with the same (s, t) remain ambiguous after canonicalization and any row permutation. Two identical usable vectors do not create false ambiguity. An unusable or missing vector in the latest search group disables context even if that group contains one usable vector. Replacing lists `clicked_skus_hash`, `product_skus_hash` does not change the inputs or response of the main variant.

Test K.3 (B03. Retries, closed blocks, and limits)**.** Accept 100 events and repeat the same block after 1 ms: insufficient tokens do not change counters, completeness, or the previous response. The same holds for a verifiable retry after δ_{late}. With $\delta_{\text{future}} = 0$, a new block at $t = a_{\text{block}}$ passes the time check; a lead of 1 ms or one hour produces `timestamp_future` without changing the history or $t_{\text{last},s}$. Check both calendar-range boundaries, incorrect units, overflow, and replay of an accepted block after the allowed range changes. Check a changed payload under the same ID, an ID list of the same length with different contents, a different owner, an empty block, an incorrect event count, and an incomplete envelope. An envelope consisting of two parts of 60 events is not accepted in parts; new IDs for an already closed time produce `closed_block_conflict`. With 950 accepted events, a new block of 100 is rejected in full. The accepted history is preserved; a gap in the new trusted portion is reflected under 16.8. Untrusted input does not corrupt another session. Two `remove` actions do not produce a negative quantity; simultaneous `add`/`remove` actions without an order leave the cart unknown. Only `add` yields the corresponding state; only `remove` yields the removed state; the absence of both leaves the state unknown.

Test K.4 (B04. Graph)**.** One hundred repetitions of a pair in one session yield support 1 and weight at most 1. A nonempty normalized row sums to 1; an empty row remains zero. For fixed $W^r_{ij} > 0$, increasing the other row weights decreases $\rho^r_i P^r_{ij} = W^r_{ij}/(d^r_i + \lambda_G)$. A separate edge penalty does not replace the reference path matrix.

Test K.5 (B05. Ranker numerics and derivatives)**.** A missing, zero, nonfinite, or incorrectly dimensioned external vector receives a zero mask and a separate reason; valid branches remain operational. A nonfinite internal feature or loss stops training and activates fallback during serving. Check opposite vectors, coordinates on the order of 10^{200}, empty masks, and scores $-10^4, -100, 0, 100, 10^4$. The reference uses a scaled norm and stable softplus/LSE. Original modality spaces are not mixed. For an enabled neural network, also check entropy and KL at saturation under N06; finite softmax values without finite derivatives are insufficient.

For pair $(1, 0)$ with score differences $-5, -0.1, 0, 0.1, 5$, compare derivatives with central differences, step 10^{-4}, at fixed ranks and weights: absolute tolerance 10^{-7} for the gradient and 10^{-6} for the Hessian. Check the direct sum of pairs, FP32 casting, a zero group, and a zero sum of pairwise gradients. On a small tree with $\gamma_T = 0$, check the leaf value and $\text{Gain}_{\text{LGBM}} = 2\,\text{Gain}_{\text{math}}$; external Dataset weights are disabled. A separate example contains at least five categories: one with $G = H = 0$, and the others with informative pairs and a positive Hessian sum. The check must exercise the grouped categorical split branch; with `cat_smooth=1`, internal keys and the result are finite.

Test K.6 (B06. Observed targets and eligibility)**.** Three eligible sessions have unknown coverage and observed targets $\{a, b\}$, $\{z\}$, $\varnothing$, where $z \notin \mathcal{I}_G$. The first output contains a only for the first session: $N^{\text{obs}}_J = 3$, $N^+_J = 2$, $\text{Hit@20}_J = 1/2$. Purchase z remains among the targets and counts as a miss; the future catalog is not substituted. An empty target does not become a confirmed negative outcome; a positive target does not establish telemetry completeness. A repeated purchase of the same SKU does not change the set; subsequent outputs do not change the metric. A purchase in the first-view group makes the session ineligible. For an eligible session, a purchase at $t_{0,s} + H$ is included in the target, while a later purchase is excluded. At $t_{0,s} + H + g = b_J$, the session is excluded regardless of purchase presence. Skipping published rows of a selected session is a read error, not unknown telemetry.

Test K.7 (B07. Candidates, delivery, and loss decomposition)**.** Baseline quotas sum to 300; after filtering and filling, $L_q \subseteq \mathcal{C}^{\text{served}}_q \subseteq \mathcal{I}_q$, $|\mathcal{C}^{\text{served}}_q| \le M$. A small catalog is not padded with ineligible items; the shared limit K does not require equal actual response lengths. A purchase outside the candidates is not inserted during training or evaluation.

Check the normal branch, fallback, failure before selection, and delivery failure after selection. In the last case, the list is empty and the selected candidates are preserved; before selection, both sets are empty. Branches are not merged retrospectively. Four purchasing sessions with one loss at each of the three stages and one hit yield $(N_J^+, \sum V_s, \sum U_s, \sum \mathrm{Hit}_s) = (4, 3, 2, 1)$, with three losses of $1/4$ each. A repeated hit does not add one. Always $\mathrm{Hit}_s \le U_s \le V_s \le 1$; if $\sum V_s = 0$, conditional coverage is undefined.

Test K.8 (B08. Sources and fallback)**.** A target with provenance $\{a, b\}$ remains after removing a: $\Delta_a^{\text{fixed}} = 0$. Removing its only source loses the target; fallback filling may restore it. These two calculations are recorded separately. A failure does not remove the session from the denominator; the response actually delivered is used.

Test K.9 (B09. Unavailable purchase rate)**.** The historical purchase-probability feature is absent from the feature schema; unknown outcomes are not replaced with zeros to calculate it.

Test K.10 (B10. Data separation and selection)**.** Parts `B10.data`, `B10.ranker`, and `B10.metadata` check the data, ranker, and variant without metadata by the applicable deadlines. Changing D does not change the model, variant list, block length, or retrieval rules; changing reserved E does not change the computations. Check that the seven parts share no sessions, further development on C_{select} is prohibited, and tie-breaking order is respected. Arbitrary replacement of supplied metadata does not change the variant in I.1.

Test K.11 (B11. Disabled calibration)**.** The run manifest and computations require no calibrator; data from E are not read, and output scores are not described as purchase probabilities. Penalty $\lambda_D = 0$; the probabilistic diversity procedure is disabled.

Test K.12 (B12. Temporal intervals)**.** For $J = 4, m = 2$, starts $(1, 3)$ produce indices $(1, 2, 3, 4)$; both policies are moved together, and the statistic is a ratio of sums. Empty calendar cells are retained; the $H + g$ calendar margin is not added. A zero denominator and too few blocks produce `insufficient`. The same state is required when all $d_s = 0$, all Hit=0, or all Hit=1, even if the formal interval has zero width. Check the boundaries of 99/100 sessions and 19/20 blocks under (76); sufficient counts do not permit a degenerate interval, for example with identically balanced cells. The single-coverage formula matches replacement of d_s by Hit. The resample denominator equals the number of sessions with an observed target; there is no factor of 1000; cells without such sessions remain on the calendar grid.

For B_0 independent equiprobable signs ± 1, each repeated m times, the variance of the mean is $1/B_0$, while an independence-based calculation incorrectly gives $1/(mB_0)$. The coverage check fixes the number of repetitions and the binomial tolerance for simulation error in advance. It is not required that every random interval be wider than the interval for independent observations.

Test K.13 (B13. Run manifest stages)**.** When prerequisites are filled in, missing future training/load results do not block `preflight`. Local `ideal_deterministic` permits selection and evaluation on D without `post_load`; the absence of a service does not establish pilot readiness. The first C_{smoke} does not require B18 or a completed variant without metadata. Omission of these checks by their required deadlines is rejected. Results are saved separately; changes to an already fixed plan or a mismatch between its version and the code, inputs, and settings in use are detected before computation. At the deadline for a test, a missing result produces `not_run`. A negative report does not become `passed` merely because it exists. Check conditional requirements for fields of enabled modules. The evaluation protocol is explicit; an unknown mode is rejected. Disabled calibration does not block `pre_D`, but a successful historical report does not establish complete outcome recording for A/B testing. An incorrect type, unknown state, inconsistent times, violated input integrity, or missed applicable check deadlines are rejected.

Test K.14 (B14. Versions and replay)**.** Batch and sequential processing of the same visible prefix produce identical masks, categories, candidates, and versions; numbers agree within the declared tolerance. A late event does not change a past response. For $t_q = 10{:}00$, $a_q = 10{:}05$, a package published at 10:03 is prohibited. A trace of a disabled image source reproduces the same mask and $\mathcal{C}_q^{\text{served}}$; without a trace, a healthy replay is not declared to be the measured U_s. Deleted inputs are not replaced by their hash.

Test K.15 (B15. Load, memory, and recovery)**.** Failed, overdue, and undelivered requests remain in the load and denominators. Input acceptance and queueing count toward the deadline; stage P99 values are not added. A timely fallback response remains a fallback. Exact search peaking at 30 GiB fails the overall 28 GiB limit; both packages are counted during an update. Check queue overflow, a Sybil/SKU burst, signatures, checkpoint loss, concurrent writes, and catalog failure. Insert more than L_{hist} search blocks between views: the queue of L_H views is preserved. Recovery limits, feature agreement, and RTO/RPO are checked by measurement.

Test K.16 (B16. Stopping and alert rates)**.** At one request per second, 20 consecutive technical failures of valid requests trigger protection, even though five minutes contain fewer than 1000 requests. Three failed internal checks also trigger protection. Invalid requests and an unauthentic new package do not roll back a healthy version. For $A = (2, 0.5, 2, 0.5)$ and $h = 1$, the exceedance fraction is $1/2$, but the rule requiring two adjacent windows never triggers. An empty window does not yield a zero error rate; a missing status counts as a failure.

Test K.17 (B17. Approximate search)**.** When ANN is enabled, compare it with exact search over the same eligible catalog. Dot products of 0.05 and 0.045 yield a loss of 0.005 regardless of the ranking-score temperature. An empty approximate list with a nonempty exact list fails acceptance. Duplicates, ineligible items, and incomplete repeatability are detected separately.

Test K.18 (B18. Causal outcome accounting)**.** Two sessions of one stable key receive one variant and contribute one entry session to the primary denominator. A service error does not remove it or a reliably recorded purchase. Check SRM and descriptive status after a safety stop; an empty group produces `insufficient`. A plan with a stratum based on post-assignment clicks is rejected before launch; a plan based on features available before assignment checks the missing-value rule, immutable membership, and retention of all assigned units in the denominator. With independent $Z_1, Z_2 \sim \text{Bernoulli}(1/2)$, the standard plan $S_1 = 1$, $S_2 = Z_1$ is also rejected: when the stratum size is 1, the number of variant-1 assignments is identically zero, rather than following $\text{Bin}(1, 1/2)$. For $k = 0$ and $k = n$, check the boundary beta intervals. With all other conditions satisfied, $\ell_B = \delta_{\text{accept}} < u_B$ produces `insufficient`, $\ell_B \le u_B = \delta_{\text{accept}}$ produces `failed`, and $\ell_B > \delta_{\text{accept}}$ produces `passed`. With predefined purchases independent of recommendations, an ideal output may have high Hit@20 while retaining $Y_s(1) = Y_s(0)$ and $\Delta_{1000}^{\text{ITT}} = 0$. Describing this result as conversion growth is an error in metric interpretation.

K.3 Neural extension

Test K.19 (N01. Graph reference and paths)**.** Changing A_N with A_G fixed preserves the catalog, $\mathcal{N}_i^P$, D_{ij}^G, and frequencies. With $\eta_N = 0$, nonempty rows have $A_{ij}^N = \nu_{ij}^P$, and $\mathcal{L}_G^N = 0$. On a small graph, exhaustive enumeration of up to H_P transitions matches search over states "vertex, number of transitions" with sufficient B_P. Truncated search returns a valid path and a truncation flag, but does not claim an exact minimum.

Test K.20 (N02. Masks and zero projections)**.** When branches are disabled, the remaining weights are normalized; without inputs, the source is empty. For a usable input, $Y_q^G \neq \varnothing$, and $u_q^N = 0$, the example is retained: $p_N(i \mid q) = 1/|\mathcal{I}_q|$, with loss $\log |\mathcal{I}_q|$. For $M_q^S = 1, m_q^{SN} = 0$,

search mixing is zero and history is not attenuated. Zeroing a learned projection does not remove the example.

Test K.21 (N03. Intents and search)**.** For $J_q = J_I$, jointly permuting intents, parameters, and quotas preserves representations, scores, and candidates. For identical $h_{q1} = h_{q2} = (1, 0)$ with equal weights, entropy is $\log 2$, even though the directions are identical. With one direction, pairwise diagnostics have a zero mask. For $h_{q1} = (1, 0)$, $z_q^N = (0, 1)$, and $g_q^{SN} = 1/4$, the result is $u_q^N = (3/4, 1/4)$ and the score of item $(0, 1)$ is $1/(4\tau_N)$. With $\bar{g}_N = 0$, search may still change the weights of distinct intents; disabling all its paths eliminates this influence. Mask p_S is not generated during serving.

Test K.22 (N04. Purchase and next-view targets)**.** Direct and blockwise computations of the full denominator agree. Two purchases have masses of $1/2$; score derivatives sum to zero; a nontarget score affects the denominator. For two views at 0 and 60 without a purchase, the last view enters the next-view target; also check $t_{0,s} + H$ and a view simultaneous with the first purchase under (127).

Test K.23 (N05. Contrastive comparisons)**.** A batch contains no two requests from the same session. Identical signatures and a shared purchase are excluded by the specified rule. Without eligible comparisons, the loss is zero and inputs and candidates are unchanged. Batch order and masks are preserved; variant `label_free_v1` removes only the shared-purchase filter, without reading C, E, D.

Test K.24 (N06. Initialization and training)**.** Repeating the specified profile and generator states reproduces initial tensors and batch order. The shared matrix is initialized once; with $U_N = 0$, initially $A^N = \nu^P$. Compare two Adam steps on a scalar with an independent calculation: norm clipping precedes moment updates, and L2 is counted once. At fixed parameters, the average purchase component of the loss over all batches, including the short final batch, matches the full loss under (132). For logits $(0, -200)$ in FP32, entropy is finite and close to zero, and KL relative to $(1/2, 1/2)$ is close to $\log 2$. Values and gradients with respect to logits are compared with a stable FP64 reference; a single position yields zero entropy and gradient, while an empty mask yields the specified zero branch. Rounding a probability to zero does not cause NaN. Check stream independence: a disabled optional consumer does not change batch order or other random transformations.

Test K.25 (N07. Quotas and baseline restoration)**.** The overall limit M is respected for any number of intents. Disabling trainable sources while restoring the original quotas, features, and ranker reproduces the baseline; merely zeroing the new score is insufficient. With an unrecorded neural-source failure, the result matches the common fallback: the branch, $\mathcal{C}_q^{\text{served}}$, and output order are unambiguous; fallback-output scores receive no probabilistic interpretation.

K.4 Additional variants

Test K.26 (X01. Aging, publication, and graph traversal)**.** Shifting an event pair into the past while keeping its interval unchanged preserves the within-session factor and decreases the aging factor; outside the window, the relation is excluded. The graph, paths, vectors, and index are published as a consistent package. With fixed c_k and eligible whole sessions, events outside the window or after c_k do not change the package. Pair repetitions do not increase a session's contribution beyond 1; on retained edges, $\rho_i^r P_{ij}^r = W_{ij}^r/(d_i^r + \lambda_G) \le P_{ij}^r$. A package with $r_k \ge t_q$ is unavailable; an update does not change an old response or its version. For enabled PPR on a graph without edges, check $z^{(1)} = \eta_R h_q + (1 - \eta_R)\pi^{\text{pop}}$ and measure the cost of a full step as the catalog grows.

Test K.27 (X02. Sampled denominator)**.** *Support (before training in sampled mode).* In order a, b, with frequencies $(1, 0)$ and $\eta_P = \delta = 10^{-20}$, the computed $Q_G \approx (1, 1.5 \cdot 10^{-20})$, but the saved $F = (1, 1)$. For target a, the directly computed allowed mass is positive, while $\rho_b = 0$; before training or consuming the stream, failure `negative_sampling_precision_unsupported` is required, not removal of b or waiting for a budget failure. Also check strictly increasing $F = (2^{-54}, 3 \cdot 2^{-55}, 1)$: its second interval is positive, but contains no generator-grid points and must be rejected. Check the initial parameters on the actual catalog against all conditions in A.1; the check does not consume the random stream.

Loss (after training). In catalog $\{a, b, c\}$, with $T_q^a = \{a\}$, zero scores, $m = 1$, $Q_q^a(b) = 1/4$, $Q_q^a(c) = 3/4$, we have $Z_q = 3$. The estimate takes values 5 and $7/3$ with these probabilities: $\mathbb{E}[\widehat{Z}_q^a] = 3$, but $\mathbb{E}[\log \widehat{Z}_q^a] < \log 3$. The mean loss gradient with respect to the positive score is $-22/35$, versus the exact $-2/3$. With a fixed sample, finite differences reproduce (142). Positive items enter once; negative items retain their sampling multiplicity. An empty complement yields exact mode. Check positive support of Q_q^a, exclusion of all targets from the negative sample, multiple targets, zero vectors, and a purchase outside the catalog: it does not expand the catalog and remains a possible miss. Compare the mean $\widehat{Z}$ with exact Z, accounting for random sampling error and the allowed distribution discretization error; exhaustive enumeration of the complement with unit weights reproduces the exact loss and gradients. A finite random sample is not required to do so. Repeating stream $j = 4$ preserves sampled items and the number of rejected attempts; a zero task and an empty complement do not change the generator state.

Test K.28 (X03. Group sequence)**.** Permutation within a group does not change the representation; the causal mask prevents later groups from influencing earlier ones. Moving items between groups may change the response. The control retains action types and the purchase target; an action-enriched variant is identified separately. For $\gamma = 1, \beta = 0$, input $(0.001, -0.001)$, and $\varepsilon_{\text{LN}} = 10^{-5}$, the first LN coordinate is $0.001/\sqrt{0.000011}$, approximately 0.3015113.

Test K.29 (X04. SID: derivatives and stages)**.** With fixed noise, finite differences reproduce (164). Check the path from $\mathcal{L}_R^{\text{SID}}$ to A_ℓ^{SID} through soft assignments (a nonzero gradient on a non-degenerate example), and from $\mathcal{L}_G^{\text{SID}}$ to the decoder's input code vectors. With target codes fixed, $\mathcal{L}_G^{\text{SID}}$ gives no derivative with respect to A_ℓ^{SID}; e_i, u_q^N are fixed. For each stage, check the set of updated parameters, the reset of Adam state, and noise repeatability. Only the decoder changes in the final stage. Saturated assignments retain finite balancing loss and gradients; finite-difference checks do not apply to the discrete arg max.

Test K.30 (X05. SID collisions)**.** For $L_{\text{SID}} = 2$, two items with code $(2, 1)$ and $R_{\text{SID}} = 1$ yield the first item in the specified order. A purchase of the second does not change the mapping and remains a miss for the SID source; another source may retrieve the item. Without noise, codes and candidates repeat; all returned items belong to $\mathcal{I}_q$. The random control with the same stream $m = 2, j = 6$ reproduces unique codes. Inconsistent codebooks, decoder, and tree are not admitted to serving.

L Contents of the final research deliverable

Deliver the run manifest, checks, and report I.2, including data sources and evaluation denominators, as well as the A/B plan. Without the corresponding observations, causal and monetary results have state `not_observed`.

References

[1] Coveo. *SIGIR eCom Data Challenge: dataset documentation*. https://github.com/coveooss/SIGIR-ecom-data-challenge. Description of the schema, anonymization, features, and search data.

[2] Tagliabue J., Greco C., Roy J.-F., Bianchi F., Cassani G., Yu B., Chia P. J. *SIGIR 2021 E-Commerce Workshop Data Challenge*. 2021. https://arxiv.org/abs/2104.09423.

[3] Peintner A., Mohammadi A. R., Zangerle E. *Efficient Session-based Recommendation with Contrastive Graph-based Shortest Path Search*. ACM Transactions on Recommender Systems, 2025. https://doi.org/10.1145/3701764. Author version: https://evazangerle.at/publication/peintner-tors-2025/.

[4] Choi M., Kim H.-Y., Cho H., Lee J. *Multi-intent-aware Session-based Recommendation*. SIGIR, 2024. https://doi.org/10.1145/3626772.3657928. https://arxiv.org/abs/2405.00986.

[5] Zhang X., Xu B., Ma F., Wang Z., Yang L., Lin H. *Rethinking Contrastive Learning in Session-based Recommendation*. Pattern Recognition. Vol. 169, 2026. Article 111924. https://doi.org/10.1016/j.patcog.2025.111924. https://arxiv.org/abs/2506.05044.

[6] Ju C. M., Collins L., Neves L., Kumar B., Wang L. Y., Zhao T., Shah N. *Generative Recommendation with Semantic IDs: A Practitioner's Handbook*. 2025. https://arxiv.org/abs/2507.22224. Authors' repository: https://github.com/snap-research/GRID.

[7] Fu J., Ge X., Karatzoglou A., Arapakis I., Verberne S., Jose J. M., Ren Z. *Differentiable Semantic ID for Generative Recommendation*. SIGIR, 2026. https://doi.org/10.1145/3805712.3809641. https://arxiv.org/abs/2601.19711.

[8] Kang W.-C., McAuley J. *Self-Attentive Sequential Recommendation*. ICDM, 2018. https://arxiv.org/abs/1808.09781.

[9] Chen J., Zhang J., Huang X., Yang Y., Lian D., Chen E. *Adaptive Sampled Softmax with Inverted Multi-Index: Methods, Theory and Applications*. 2025. https://arxiv.org/abs/2501.08563.

[10] Khrylchenko K., Baikalov V., Makeev S., Matveev A., Liamaev S. *Correcting the LogQ Correction: Revisiting Sampled Softmax for Large-Scale Retrieval*. RecSys, 2025. https://doi.org/10.1145/3705328.3748033. https://arxiv.org/abs/2507.09331.

[11] TensorFlow. *Candidate Sampling Algorithms*. https://www.tensorflow.org/extras/candidate_sampling.pdf. Comparison of training methods with sampled classes and noise examples.

[12] Burges C. J. C. *From RankNet to LambdaRank to LambdaMART: An Overview*. Microsoft Research Technical Report MSR-TR-2010-82, 2010. https://www.microsoft.com/en-us/research/publication/from-ranknet-to-lambdarank-to-lambdamart-an-overview/.

[13] LightGBM 4.6.0. *Parameters; Python training API; custom derivatives and split gain implementation*. https://lightgbm.readthedocs.io/en/v4.6.0/Parameters.html; https://lightgbm.readthedocs.io/en/v4.6.0/pythonapi/lightgbm.train.html; https://github.com/microsoft/LightGBM/blob/v4.6.0/python-package/lightgbm/basic.py; https://github.com/microsoft/LightGBM/blob/v4.6.0/src/treelearner/feature_histogram.hpp. Version `lightgbm==4.6.0`, binary package hashes, and the callback are saved in the run lockfile.

[14] Barkan O., Koenigstein N. *Item2Vec: Neural Item Embedding for Collaborative Filtering*. 2016. https://arxiv.org/abs/1603.04259.

[15] Schnabel T., Swaminathan A., Singh A., Chandak N., Joachims T. *Recommendations as Treatments: Debiasing Learning and Evaluation*. 2016. https://arxiv.org/abs/1602.05352.

[16] Dudík M., Langford J., Li L. *Doubly Robust Policy Evaluation and Learning*. 2011. https://arxiv.org/abs/1103.4601.

[17] Kohavi R., Tang D., Xu Y. *Trustworthy Online Controlled Experiments: A Practical Guide to A/B Testing*. Cambridge University Press, 2020. https://doi.org/10.1017/9781108653985.

[18] Holm S. *A Simple Sequentially Rejective Multiple Test Procedure*. Scandinavian Journal of Statistics. 1979. Vol. 6, no. 2. P. 65–70. https://www.jstor.org/stable/4615733.

[19] Efron B. *Bootstrap Methods: Another Look at the Jackknife*. The Annals of Statistics. 1979. Vol. 7, no. 1. P. 1–26. https://doi.org/10.1214/aos/1176344552.

[20] Künsch H. R. *The Jackknife and the Bootstrap for General Stationary Observations*. The Annals of Statistics. 1989. Vol. 17, no. 3. P. 1217–1241. https://doi.org/10.1214/aos/1176347265.